

有界测向误差下干扰源的连续几何规划与清除

摘 要

无线电干扰源的快速清除既需要保证发现全部目标，也需要控制测向与行走代价。本文以最坏角锥交的最小包围圆为定位判据，把集合估计、保证接收、剩余覆盖证书及真实控制器续跑连接起来，检验几何信息改善能否转化为实际时间收益。

问题一，以前向角锥表示有界示向观测，以半平面交求多边形，以顶点对求区域直径。直径为 40 米的等边三角形不能被半径 20 米圆覆盖，故以最小包围圆半径不超过 20 米作为保证清除判据。

问题二，首次收到信号还给出了接收半径下界。完整首扇区的保证接收域为四个圆盘的交；复核得到全域最坏后验半径上界 55.739 米、物理量化模型下界 55.6528 米。这直接说明，对完整首扇区，一次补测保证半径不超过 35 米的区域为空。被场地边界裁剪后的区域仍可另行搜索。

问题三，沿用 1124 米七点环与有限测向加光学兜底，检验最坏半径连续选点及方法 56 的单源策略续跑。开发 40 局中，半径贪心平均增加 165.79 秒；真实续跑筛选后仅节省 1.10 秒，因此未作为默认策略。冻结配置保留安全提前停车，独立 200 局全部清除，总均时 3003.37 秒，相对方法 56 节省 1.48 秒。

问题四，以“历史全频道扫描点与剩余发现点共同覆盖所有位置和朝向”为不变量，采用向外舍入区间算术认证每一次连续点替换。独立有理数复核通过 4564 个空间单元。实验候选 84 独立 200 局全部清除，总均时 5704.99 秒，相对方法 56 节省 29.62 秒，配对 95% 区间为 $[-4.69, 63.92]$ 秒。五组压力测试均完成清除。动态整点删除在开发集中未触发，故不声称减少固定 21 点或达到新增 10% 收益。

推荐默认保留方法 78，候选 84 未推广。默认策略 Q4 独立总均时 5733.40 秒；几何证书证明覆盖可行性，配对实验检验时间效果。本文全部成绩为本地模拟，六次官方正式测试及加密日志尚待补入。

关键词：有界误差；主动测向；最小包围圆；剩余覆盖；策略续跑；邻域巡游

1 问题分析与统一目标

1.1 物理约束及时间结构

目标位置固定在 $\Omega = \{z : \|z\| \leq 1800\}$ ，总数为 10 至 16，每个源独占一个频道。机器人从原点、频道 1 出发，移动速度 5 米/秒。每次换频 1 秒、检测 5 秒、失败光学 3 秒、成功光学与清除共 5 秒。移动时不能检测；在同一地点重复测向不能消除固定误差。真实接收半径在 1000 至 1500 米之间，定向源另有未知闭辐射半平面。任何合法源距均可能小于 1000 米；接收半径下界不等于源距下界。

用 L, N_s, N_m, N_f, N_c 分别表示移动距离、换频数、检测数、失败清除数和成功清除数，任务总时间严格为

$$T = L/5 + N_s + 5N_m + 3N_f + 5N_c. \quad (1)$$

研究目标是在全部清除约束下减小 T 。正式题设另要求报告 T/N_c ，本文同时提供该指标，比较策略时坚持同种子配对的**总时间均值**。先对每局计算 T/N_c 再取平均，不等于总体总时间除以总体源数。

1.2 文献的可迁移内容

Kaya[3] 用恒速运动的 Fisher 信息矩阵行列式作为目标，其局部最优路径并不保证最坏角锥交满足 20 米清除阈值。Vander Hook 等 [1] 的谨慎贪心同时考虑移动与测量，但原文采用初始高斯位置假设和 EKF 高斯噪声，处理双向示向歧义；不能把该文直接称为本题的有界集合收缩定理。本文采用其重视不利观测与时间代价的思路，重新构造几何目标。

Bayram 等 [2] 把保证定位精度的测点布置和巡游结合，其不确定度定义采用 GDOP，28.9 倍成本与 5.5 倍不确定度是双准则结论。本文只迁移“访问可行区域”的结构，不移植其常数。Dumitrescu-Mitchell[8] 和 Peng 等 [7] 提供邻域巡游的建模背景；清除邻域须由本题覆盖约束重新推导。

Hoffmann 等 [4] 通过策略续跑评价首个动作直至最终定位。本文将其落实为当前控制器的单源续跑。Sunberg-Kochenderfer[5] 指出，单纯双重渐进加宽会导致连续观测树的单粒子退化，并提出 POMCPOW 与 PFT-DPW 加以修正；该结论不是对全部 POMCPOW 的否定，也不证明旧连续搜索失败的因果原因。本文未实现完整 POMDP 搜索树。

2 集合状态及问题一的求解

2.1 前向角锥与多边形交

设检测点 s 、示向度 θ ，令 $u = (\cos \theta, \sin \theta)$ ， $v = (-\sin \theta, \cos \theta)$ 。半角为 $\bar{\varepsilon}$ 的前向角锥为

$$W(s, \theta) = \{z : |v^\top(z - s)| \leq \tan \bar{\varepsilon} u^\top(z - s)\}. \quad (2)$$

两个线性不等式还隐含 $u^\top(z - s) \geq 0$ ，因此必须保留射线的前向性。理论物理误差为 $\pm 1^\circ$ ；本地公开协议模拟器保留 0.01 度读数，在线采用 $\bar{\varepsilon} = 1.005^\circ$ 包含四舍五入误差。若实际接口不量化，使用这一角度仍为保守外包。

每个频道保存正常示向交形成的凸多边形 P_k 及全部历史响应。初始化包含 Ω 的外

切多边形；每次正常示向与 $W(s, \theta)$ 、接收半径 1500 米的外切多边形相交，并加轴向上界。无信号不直接裁掉定向源的位置多边形，其位置与朝向耦合保留在历史状态和剩余覆盖逻辑中。有限假设用于排序，不能作为全清除的集合证据。

若仅给出半平面、不额外给出场地边界，先判断空集、无界集或有界集。非空无界集直径为无穷大；空集提示观测不一致。退化点和线段分别按零直径和端点距处理。有界多边形可以用逐半平面裁剪得到，本文小规模实现扫描全部顶点对，计算

$$D(P) = \max_{i,j} \|p_i - p_j\|. \quad (3)$$

凸函数在多面体上的最大值可取于极点，因此顶点对已覆盖区域内任意点对。该实现耗时 $O(m^2)$ ；如顶点规模增大可换用旋转卡壳。

2.2 直径不足以判断单点清除

定义

$$\rho(P) = \min_c \max_{z \in P} \|z - c\|. \quad (4)$$

等边三角形边长 40 米时， $D = 40$ 米而 $\rho = 40/\sqrt{3} \approx 23.094$ 米。因此半径 $D/2$ 的圆未必能覆盖定位区域，不能以 $D \leq 40$ 替代 $\rho \leq 20$ 。最小包围圆由至多三个顶点支持：一个点、两个点的中点圆，或三个点的外接圆。通用几何库计算圆心后再次取全部顶点距离的最大值，返回的是普通浮点估计，并不构成严格向外舍入半径。连续候选模块另加数值余量，保证清除检查使用阈值裕量；这些工程处理不包含全部坐标裁剪误差的形式证明。精确有理数证书单独标明。

保证清除区域实际上是

$$\mathcal{C}(P) = \bigcap_{p_i \in V(P)} \overline{B}(p_i, 20), \quad (5)$$

其非空当且仅当 $\rho(P) \leq 20$ 。若最小包围圆为 (c, r) ，则 $\overline{B}(c, 20 - r) \subseteq \mathcal{C}(P)$ 。这给出可供路径优化的邻域，而不是必须访问的唯一圆心。

3 问题二的保证接收域与第二测的能力边界

3.1 四圆盘完整接收域

把首检测点和示向方向分别归一化为原点、 x 轴，源位置写成 $z = ru_\phi$ ，其中 $5 < r \leq 1500$ 、 $|\phi| \leq \bar{\epsilon}$ 。首次收到信号意味着其未知真实接收半径满足 $R_s \geq \max(1000, r)$ 。第二测点 q 必能收到的充要条件是

$$\|q - ru_\phi\| \leq \max(1000, r), \quad 5 < r \leq 1500, |\phi| \leq \bar{\epsilon}. \quad (6)$$

对 $r \leq 1000$ ，距离平方关于 r 凸，只需检查闭包端点 5、1000。对 $r \geq 1000$ ，约束化为 $\|q\|^2 \leq 2rq^\top u_\phi$ ，在 $r = 1000$ 成立后随 r 增大不再加严。由角端点取最坏方向，得到

$$G = \bigcap_{r \in \{5, 1000\}, \sigma \in \{-1, 1\}} \overline{B}(r(\cos \bar{\epsilon}, \sigma \sin \bar{\epsilon}), 1000). \quad (7)$$

令 $q = (a, b)$ 、 $d = a \cos \bar{\varepsilon} - |b| \sin \bar{\varepsilon}$ ，等价代数式为

$$a^2 + b^2 \leq 2000d, \quad a^2 + b^2 \leq 999975 + 10d. \quad (8)$$

此前经验点 $(750, \pm 300)$ 具有良好接收裕量；它不是最优定位点。完整扇区的已证候选为 $(843.1178, \pm 545.4003)$ ，本轮再次运行精确有理数脚本，得到表1。

表 1: 完整首扇区的第二测最坏半径证书，单位米

模型	全域下界	候选上界	差距上界
连续保守半角 1.005 度	55.73823	55.739	0.00077
物理误差及 0.01 度量化	55.6528	55.739	0.0862

上界用第二读数区间的加宽角锥外包，并对外包顶点精确核验包围圆。下界分别用近共线区固定双点见证和其余区域的远端不可区分双点见证覆盖完整 G 。相关三角函数以有理区间界计算，接受判断不依赖浮点搜索结果。脚本与输出随支撑材料给出。

全域下界的双点构造如下。由对称性只考虑 $q = (a, b)$ 、 $b \geq 0$ 。当 $b \leq 30$ 时，首轴上的 $(1340, 0)$ 与 $(1500, 0)$ 可产生相同的两次正常示向读数，给出 80 米半径下界。其余区域取 $A = 1500u_{-e_1}$ ，从 q 指向 A 的射线顺时针转 $2e_2$ 后与首扇区上边界相交于 $B = tu_{e_1}$ 。记 $k = \sin(e_1 + 2e_2)$ 、 $h = \cos(e_1 + 2e_2)$ 、 $s = \sin(2e_2)$ 、 $l = \sin(2e_1 + 2e_2)$ ，则

$$t = \frac{1500(ka + hb) - s(a^2 + b^2)}{1500l - ka + hb}. \quad (9)$$

证书先核验分母正、 $t > 5$ 、两次读数均不会触发近距离响应，再将 $t \leq T$ 化成凹二次式在包含 G 的 1000 米圆盘上的最大值非正。两模型的 T 分别为 1400.796371、1400.99，故不可区分双点的半距离至少为

$$\frac{1}{2} \sqrt{1500^2 + T^2 - 3000T \cos(2e_1)}. \quad (10)$$

取 $e_1 = 1.00499999^\circ$ ，连续模型的 $e_2 = 1.00499999^\circ$ 、物理量化模型的 $e_2 = 0.99999999^\circ$ ，保留严格读数余量，得到表中的两项下界。构造和区间余量均由有理数脚本逐项检查，而非只抽样验证 q 。

3.2 最近保证定位区域可以为空

欲以最近点代替固定坐标，可求

$$\min_{q \in G(P)} \|q - x\|, \quad R(P, q) := \sup_{\theta \in \Theta(P, q)} \rho(P \cap W(q, \theta)) \leq \tau. \quad (11)$$

阈值取 20 米时保证下一次正常示向后可清除；取 35 米时只允许尝试清除，不能保证成功。表1表明，**完整首扇区在这两个阈值下均无可行点**。边界裁剪或已有更多测向可能使 P 变小，此时应重新求域；不能把完整域下界搬到裁剪域，也不能假定保证区域永远非空。

本轮在线局部搜索使用更强的充分接收条件 $\max_{z \in P} \|q - z\| \leq 950.7$ 米，以保持原经验接收余量。它并非式(7)的完整域。最近保证域实验检查当前多边形的 35 米阈值，

未找到被门槛采纳的替换动作；这一结果仅说明本次搜索未触发，不证明每个裁剪实例的完整域都为空。

4 可独立复现的联合控制流程

方法编号只是配置索引。默认策略由“有限深度动作规划、发现与定位联合调度、有限光学后备、安全提前停车”组成，比较基线为关闭提前停车的同一控制器。表2列出决定其行为的参数。

表 2: 默认控制器的主要参数与终止门槛

参数	Q3	Q4
发现点	原点及 1124 米六点环	认证 21 点双环
单源决策深度/节点预算	2 层/6000	2 层/6000
状态假设上限/局部动作上限	32/6	63/6
机会补测上限	每源 4 次	关闭
基线补测偏移	首示向坐标 (750, ± 300)	动作菜单中保留同坐标
主动阶段后备	至多 3 轮中心试探后光学	直接自适应条带光学
尝试清除/保证清除	35 米/20 米	按规划动作/20 米
路由精确解阈值	节点数不超过 12	节点数不超过 12

1. 初始化 20 个频道的未知状态与发现点集。从当前频道开始循环扫描未知频道；正常示向建立保守多边形，近距响应立即在原地清除。成功后永久退出该频道。
2. 对每个已发现未清除源，从与全部历史响应一致的有限位置、半径和朝向假设构造动作表。测点包括当前位置、包围圆心、首示向偏移点、长轴与横轴偏移点；清除动作包括圆心和长轴上的试探点，Q3 还加入保证清除内域。排除历史同地点同类动作。
3. 在深度 2 的有限状态树上比较“行走、换频、测量、成功或失败清除”总秒数。节点上限 6000，叶端是有限假设的 28 米蛇形期望代价；它仅作基线排序，不提供覆盖或最优性证明。基线测向结果仍按 2 度分组，本轮不对这项旧代理进行连续加密。任何时候真实集合均用完整角锥更新。
4. 把未访问发现点与各已知源的下一动作同时放入开放巡游问题。Q3 允许同一源的若干候选服务入口，Q4 使用选中的单个动作点。节点不超过 12 时用子集动态规划，否则多起点最近邻、2-opt 和重定位，辅以最小生成树及指派松弛下界；下界只约束这一次路由代理。每次仅执行首个任务，响应后重建全部节点。
5. 每源最多执行 6 次局部动作。Q3 随后允许保证域中的第二测及至多 3 轮中心附近试探，35 米半径时可以试清，失败继续；Q4 转入自适应条带覆盖。光学把区域切成可被半径 20 米圆覆盖的有限子块，按有限假设命中时间选取巡游顺序；逐点发出清除命令，直到成功。
6. Q3 在已停车点可为其它已知源补测：每源最多 4 次，要求离估计圆心不超过 1000 米、相对首示向横向间距至少 50 米且未重复。它只增加信息，不参与全清除证明。发现数达到 16 后可跳过剩余发现点；否则完成证书要求的未知频道扫描，再清除所有已知源才退出。

方法 78 在该控制器外包一层安全提前停车适配器：仅把已验证可成功的清除动作沿当前路段提前，控制器仍保留原路标，所有测向位置保持不变。实验候选 84 在 Q3 与 78 相同，在 Q4 才启用后文的连续发现点证书替换。局部动作和机会补测数都有上

限，光学子块数有限；在理想几何、接口按题设响应且无传输故障时，全发现证书与逐块清除共同给出有限完成性。程序异常或超时会报告失败，不把它当成全清除。

5 问题三的最坏剩余半径规划

5.1 对全部读数取最坏而非仅枚举无噪声顶点

若以 $z \in V(P)$ 的真实方向直接切一次 $\pm 1^\circ$ 角锥，既遗漏测量误差偏移，也没有证明极值只在这些方向出现。本文覆盖全部可能示向区间 $\Theta(P, q)$ ：当 q 在 P 外时，用顶点方位的最大圆周空隙确定可见角区间，再向两端扩张 $\bar{\varepsilon}$ ；当 $q \in P$ 时使用整个圆周。

把读数区间分成 $[\theta_j - \delta_j, \theta_j + \delta_j]$ 。只要 $\bar{\varepsilon} + \delta_j < 90^\circ$ ，有

$$\bigcup_{|\theta - \theta_j| \leq \delta_j} W_{\bar{\varepsilon}}(q, \theta) \subseteq W_{\bar{\varepsilon} + \delta_j}(q, \theta_j). \quad (12)$$

故

$$R(P, q) \leq \max_j \rho(P \cap W_{\bar{\varepsilon} + \delta_j}(q, \theta_j)). \quad (13)$$

优先二分当前半径上界最大的区间，直到预算或上下界间距阈值满足。近距响应的真实不确定度至多 5 米；将这些几何方向一并纳入角锥外包只会更保守。在线半平面与圆运算仍采用浮点加容差；角区间的连续覆盖论证不等于整个浮点几何库已获形式化验证。25005 个密集读数检查未超程序外包，另由问题二有理数证书复核关键理论阈值。

候选点用确定性单纯形局部搜索，接收域由凸盘交投影构造。新候选仅保留一个；不使用旧 2 度粒子分箱目标，也不再进行八方向加密。半径目标的非凸性意味着这是局部候选搜索，而不是测点全域最优证书。

5.2 把支配门槛与时间保证分开

设原动作后半径上界为 r_0 ，新点为 q ，其上界 r_q 。本轮筛选条件为

$$\frac{2(r_0 - r_q)}{5} > \frac{\max(0, \|x - q\| - \|x - q_0\|)}{5} + 5, \quad r_q < 0.85\rho(P). \quad (14)$$

左侧是把直径收缩换算成行走秒数的启发式，右侧包含额外移动和一次检测。**这不是后续真实总时间的支配定理**。半径变小可能增加入口代价、改变联合调度，或延后一次本可成功的试清。开发实验的负结果证实仅靠该门槛不足以消除退步。

5.3 用当前单源控制器续跑筛选首动作

对通过几何门槛的连续候选与原动作，构造与已观测历史一致的有限场景，包括内部假设、可行边界点和 $\pm 1^\circ$ 误差。每个首动作均由同一单源控制器继续运行，直至真实模拟清除或执行实际的光学覆盖函数；没有把 28 米旧蛇形估价作为停止叶端。

续跑时保留当前位置、频道、多边形、历史和已用动作预算，同地点响应复用原值。仅当场景平均剩余时间至少下降 5 秒且最大场景退步不超过 5 秒时才替换。这里的“最坏”只针对列出的有限续跑场景，不是对连续源位置的时间证明。它是单源控制器续跑，未模拟其它源的后续联合调度或机会补测，因此不能等同整局策略改进定理。开发集总共执行 34624 次场景续跑，仅采纳 22 次动作替换，均时改善仍很小。

续跑中的“真实”专指调用同一单源控制器直至实际清除成功，未来观测仍由有限假设生成：每个场景固定取 +1 或 -1 度误差，不执行 0.01 度量化。它不复用评测内核完整的位置相关误差场；同位置历史读数则重放保存值。因此有限场景的门槛不能升级为连续误差场下的鲁棒保证，方法 82 没有推广。

5.4 全向发现与有限清除机制

七点集由原点和半径 1124 米的六等分环构成。原点覆盖半径 999.6 米圆域；其余源的方位距离最近环点角差至多 30 度。距离平方

$$d^2(r) = r^2 + 1124^2 - 2r \cdot 1124 \cos 30^\circ \quad (15)$$

关于 r 凸，在 $[999.6, 1800]$ 端点检查即可证明小于 999.6^2 。有理区间核验 $\sqrt{3}$ 和实际二进制坐标，保证真实最小接收半径 1000 米下覆盖整个场地。

每个正常发现源最多采用有限次局部试探；Q3 随后进入原单源定位后备，Q4 直接进入有限光学扫掠。每一光学子区域都有 20 米覆盖证据，因此逐个访问必有一个成功。若已发现 16 个源，可停止发现并处理已知剩余源；否则必须完成覆盖证书规定的各频道扫描。10 至 15 个源的局不能凭“已有许多空频道”提前退出。

6 问题四的负信息与剩余覆盖

6.1 未知朝向必须进入可能状态

未发现频道的状态应写为 $\omega = (z, R, h, d)$ ，其中 d 表示是否定向， h 为单位朝向。令 $C(q)$ 表示测点 q 可收到的状态集合，则无信号更新为

$$\Omega_k^{t+1} = \Omega_k^t \setminus C(q_t). \quad (16)$$

在不知道 R 的情况下，只能保证 $R \geq 1000$ ；定向源的必收状态还需 $h^\top(q - z) \geq 0$ 。单纯在位置平面切一个 1000 米圆盘对 Q4 不成立。

一次正响应也不足以给出不同位置的普遍必收点。固定一个可能源位置 z ，已知 $h^\top(s - z) \geq 0$ ，若要求对所有这样的 h 都满足 $h^\top(q - z) \geq 0$ ，则 $q - z$ 必须是 $s - z$ 的非负倍数。当首角锥包含两个与 s 不共线的可能源位置时，两个必要射线的共同测点只有 s 。所以不能不加条件地把 Q3 保证第二测推广到 Q4。多个正测点的凸包给出一个共同必收充分域，但它仍可能很小。

6.2 剩余覆盖不变量及证明

记 H_t 为截至当前对所有仍未发现频道都已完整扫描的实际点集， F_t 为计划保留的未来发现点。对源位置 z ，定义保证距离内点集

$$S_t(z) = \{q \in H_t \cup F_t : \|q - z\| \leq 1000\}. \quad (17)$$

命题 1. 若对所有 $z \in \Omega$ 有 $z \in \text{conv } S_t(z)$ ，则每个仍未发现频道的任意合法源状态都能在 F_t 中至少一个点被收到。

证明. 对任意单位朝向 h ，若所有 $q \in S_t(z)$ 都满足 $h^\top(q - z) < 0$ ，则其任何凸组合也

具有负投影，与 $z \in \text{conv } S_t(z)$ 矛盾。因此至少一个点在距离 1000 米以内且投影非负，能接收该源。若这个点属于 H_t ，此前完整频道扫描就会发现它，与“仍未发现”矛盾，故必有可接收点在 F_t 。全向源不需要投影条件，结论同样成立。 $\square$

该命题等价于用连续几何充分条件认证 $\Omega_k^t \subseteq \bigcup_{q \in F_t} C(q)$ ，不需要把四维状态离散成粒子。所有未发现频道共享完整扫描历史；频道一旦发现，退出剩余发现证书，转入已知源定位。假如将来加入只扫部分频道的机会扫描，必须分别保存 $H_{k,t}$ ，不能套用共享历史。

初始 F_0 使用已认证 21 点双环布局。每次欲删除一个点或将其移向当前位置，都先重新验证命题1；失败或计算预算耗尽时保留原点。一次阴性响应只是把某点从未来集合移入历史集合， $H_t \cup F_t$ 并未变化，**不会自动让另一个本来必需的静态点变成冗余点**。真正的删除需要额外实测位置、已证覆盖冗余或其它合法先验。因此“21 减到 15”不是本题局内必然可实现的省空间。

6.3 连续单元与向外舍入认证

从包围场地的正方形递归二分矩形单元 B 。若单元与场地无交则略去；否则令

$$I_B = \{q : \max_{v \in V(B)} \|q - v\| \leq 999.999\}. \quad (18)$$

当 $V(B) \subseteq \text{conv } I_B$ 时，距离凸性保证 I_B 中的每一点对整个 B 都在 1000 米以内，凸包条件也延伸到整个 B 。未通过则沿长边二分，最多 30 层、100000 次检查；未决即拒绝该修改。

每个距离平方及叉积均使用向外舍入区间：只有距离上界小于阈值、叉积下界非负时才接受。凸包构造用于提出见证，其每条边及凸性再用区间检查；它本身的浮点排序不作为证明。实际二进制坐标通过 17 位有效数字传入接口，保证认证坐标与发送坐标一致。初始布局本轮认证 4552 个叶单元；一个实际接受的 3.595 米点替换产生 4564 个叶单元，独立 Python 有理数程序同时核验单元几何及整个场地分区无缺口。

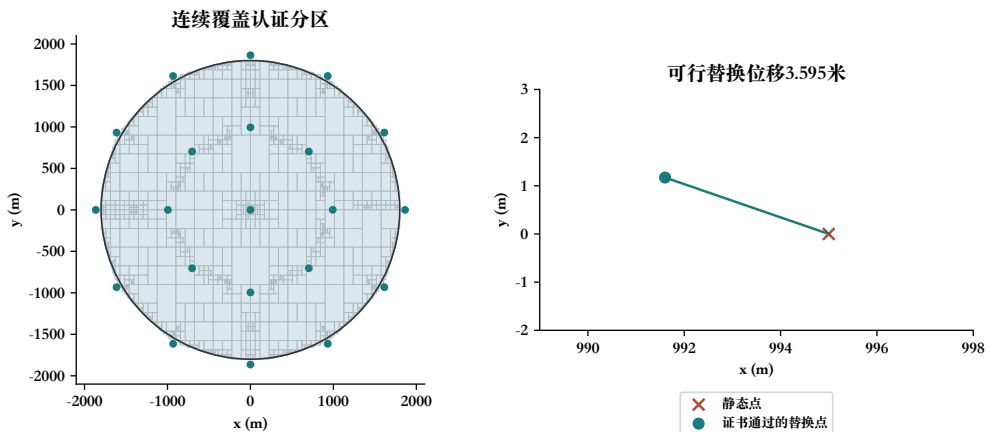


图 1: 固定 21 点及一个连续替换的认证空间单元。细分密度反映距离和凸包边界的紧约束；放大图显示内环点允许移动的范围很小。

6.4 实施范围及固定点数下界

方法 81 在已选中的下一发现点到当前位置的线段上寻找通过证书的较近点。七次二分是有可行性复核的局部搜索，不宣称该约束沿线单调，也不宣称找到了完整可行域的最近点。方法 85 进一步尝试在现有定位位置扫描后删除一个未来点，仍逐次认证；开发集没有成功删除整点，候选 84 关闭这一额外计算。

固定通用布点下界的适用对象是出发前固定、对任意位置和朝向保证接收的有限点集，不包括适应性信息和光学扫描。下面补出面积与边界弧证书之间的分析连接。

命题 2. 对半径 $R = 1800$ 米场地及最小接收半径 $L = 1000$ 米，任意固定通用定向发现点集至少含 15 点。

证明. 设点 q_i 到原点距离为 r_i ，其接收圆盘在场地内的面积为 A_i 。全朝向可接收等价于源位置属于其距离 L 内测点的凸包；若仅有两个测点，则源必须落在两点连线上。有限条连线的面积为零，因此几乎处处至少有三重圆盘覆盖，积分得 $\sum_i A_i \geq 3\pi R^2$ 。

再考虑位于场地边界且径向向外发射的源。内点 $r_i < R$ 不可能接收， $r_i = R$ 只能覆盖零长弧， $r_i > R + L$ 完全无效。其余点能覆盖的边界弧半角为

$$\delta_i = \min \left\{ \arccos(R/r_i), \arccos \frac{R^2 + r_i^2 - L^2}{2Rr_i} \right\}. \quad (19)$$

圆周全部覆盖要求 $\sum_i \delta_i \geq \pi$ 。定义面积损失 $d_i = 1 - A_i/(\pi L^2)$ 。对 $r_i \geq R$ ，两圆交面积为

$$A(r_i) = 2 \int_0^L [\sqrt{R^2 - y^2} + \sqrt{L^2 - y^2} - r_i]_+ dy. \quad (20)$$

被积函数随 y 单调下降，1 米步长左矩形和给出面积上界。脚本在径向分段 $[1800, 1900, 1950, 2000, 2025, 2050, 2800]$ 上，以有理数平方根上界和余弦泰勒界逐段认证 $d_i \geq K\delta_i$ ，其中 $K = 1.38$ 。内点的 $\delta_i = 0$ 且 $d_i \geq 0$ ，外于 $R + L$ 的点也满足。因此

$$N = \sum_i \frac{A_i}{\pi L^2} + \sum_i d_i \geq \frac{3R^2}{L^2} + K\pi > 14.0553 > 14, \quad (21)$$

由整数性得到 $N \geq 15$ 。每段最小余量及有理数运算输出均在支撑材料中。 $\square$

当前关系为 $15 \leq N^* \leq 21$ ，既不证明 21 最优，也不证明动态方案可省 6 次扫描。

7 光学覆盖和邻域巡游的严格转换

Fekete 等 [6] 的割草模型要求连续路径上的刀具覆盖区域；本题光学必须显式发出清除命令，不能把沿线经过算成自动光学成功。先将 P 分成有限凸子区域 P_i ，再为每一块选择清除位置，才能保留覆盖保证。

若 $P_i \subseteq B(c_i, r_i)$ 且 $r_i \leq 20$ ，可行清除点是式(5)给出的 $\mathcal{C}(P_i)$ ，其内含 $B(c_i, 20 - r_i)$ 。不能仅要求路径碰到 $B(c_i, 20)$ 。例如源在该圆右端、机器人在左端，二者相距 40 米。本文对每块全部顶点执行 20 米距离复核，交替优化访问坐标与 2-opt 顺序，并在整条扫描长度和有限假设平均发现时间都改善时接受。

原条带覆盖、可变条带覆盖和连续邻域巡游分别保留为方法 83、86 的消融。对多

个已满足 $\rho \leq 20$ 的源，实验实现沿全局访问顺序联合调整清除位置；该实现并未求解所有排列上的全局 CETSP。某一固定扫掠全长缩短，也不保证每个未知源的首次命中时间缩短。开发集 Q4 仍出现净退步，因而没有启用这些模块。

推荐默认方法 78 保留的安全提前停车只修改已经保证成功的清除动作，在原路段上找到最早能覆盖全部 P 的位置，未来测向路标保持不变。其路程不变量为 $L + \|x - a\| \leq L_0$ ，其中 a 为原计划路标、 x 为真实位置。由线段关系与三角不等式可归纳证明。该机制不会产生本轮声称的结构性规划收益。

8 配对实验及策略选择

8.1 开发与独立验证严格隔离

开发使用种子 2110001 至 2110040，每问 40 局。历史 CSV 在本机同种子标签下未能逐行重现，因此本轮重新编译原始未修改方法 56，并与新代码重跑的 80 行开发基线逐行核对一致；所有比较均使用本轮同一编译器和标准库。历史 CSV 不参与配对或合并，不能据种子标签相同声称跨平台场景相同。冻结后使用 2610001 至 2610200，每问 200 局，比较方法 56、上一轮默认 78 和本轮候选 84。压力五组分别为边界外向、共线端点、近重合、相关角误差、分区端点误差，各 40 局/问，起始种子 2620001 至 2660001，步长 10000。独立集不用于继续调整参数。

本地各局独立生成 10 至 16 个源及不重复频道；局内位置在圆内按面积均匀提议，但拒绝任意两源距离小于 40 米的位置，故位置间不独立。接收半径在 1000 至 1500 米、朝向在整圆均匀提议；Q4 定向源数在 1 至源数减 1 之间生成。默认误差由厘米舍入位置和种子的散列映射到 $[-1, 1]$ 度，再把读数量化至 0.01 度；同位置误差固定。统计均值仅适用于这一公开的本地场景分布。不同方法可走不同位置，配对共享环境和误差场，而非强行共享一串独立测量噪声。压力组合用于暴露失效结构，其分布与 IID 不同，不与 IID 合并估计总体均值。

表 3: 开发 40 局每问的配对总时间，正值表示节省；未混入独立集

方法	变化	Q3 节省秒	Q4 节省秒
79	最坏半径与行走门槛	-165.79	0.00
80	最近保证第二测	0.00	0.00
81	连续发现点替换	0.00	31.83
82	几何门槛及单源续跑	1.10	0.00
83	原覆盖的邻域巡游	1.72	-5.21
85	连续替换及机会删点	0.00	31.83
86	可变覆盖及邻域巡游	1.72	-4.09
84	冻结候选组合	1.91	32.67

方法 79 的 Q3 表现为 7 局更快、33 局更慢，明显偏向不利；因跨平台历史样本不可逐行对齐，不把这一胜负结构与旧 67–69 作严格同场景比较。不能据半径目标更贴物理，就断言时间必然改善。方法 82 用真实单源续跑后将平均退步消除，但仅节省 1.10 秒；Q3 最后选择方法 78 的安全执行优化。Q4 开发平均节省约 32.67 秒的候选 84 由连续发现点替换和提前停车组成，其它模块关闭。方法 81 与 85 的动作结果相同，后者仅增加无效认证计算。

8.2 独立同种子配对结果

表 4: 独立 200 局每问，候选 84 与方法 56 比较；时间单位为秒

指标	Q3	Q4
方法 56 总均时	3004.85	5734.60
默认 78 总均时	3003.37	5733.40
默认 78 单源均时	235.00	452.52
候选 84 总均时	3003.37	5704.99
候选 84 单源均时	235.00	450.62
总均时节省	1.48	29.62
CPU 平均时间	0.10	1.48
CPU 最大时间	0.20	2.68
84 对 78 新增节省	0.00	28.41
节省率/%	0.049	0.516
配对 95% 区间	[1.21,1.76]	[-4.69,63.92]
更快/持平/更慢	185/15/0	119/0/81
全部清除	200/200	200/200

对每局配对差 $d_i = T_{56,i} - T_{84,i}$ ，以 $\bar{d} \pm t_{0.975,n-1} s_d / \sqrt{n}$ 报告均值区间。收益率是 $100\bar{d}/\bar{T}_{56}$ ，不是各局百分比的平均值。退步局以 10^{-4} 秒容差判定，避开微秒级累计舍入。以上数字来自完整 CSV，保持失败或不利结果可追踪。

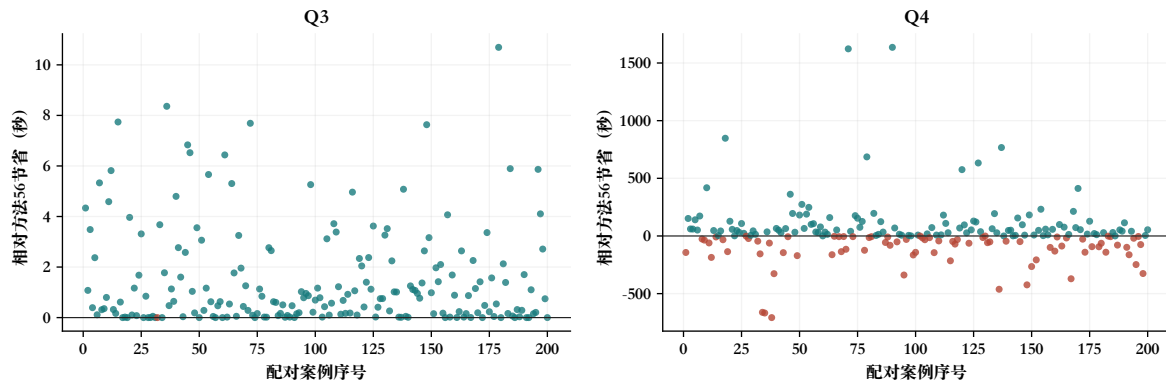


图 2: 冻结独立集每问 200 个环境的配对时间差。正值为候选 84 节省，横线为零；异常值与负收益全部保留。

Q3 与方法 78 逐局相同；其方法 56 的 1.48 秒收益全部来自已有提前停车，不能记为本轮新增规划收益。Q4 相对方法 78 平均节省 28.41 秒、0.496%，95% 区间为 $[-5.92, 62.75]$ 秒，86 局变慢。相对方法 56 的最大单局退步为 707.85 秒。两个 Q4 比较的区间均跨零，按预声明规则不推广候选 84，推荐默认保留方法 78。实验候选可由 geometric 策略名显式运行。

8.3 成本分解及压力测试

式(1)使配对节省分成移动、测向、换频和失败清除四项；同局成功清除数相同，其成本相消。这是计时恒等分解，不是各模块的独立因果贡献。

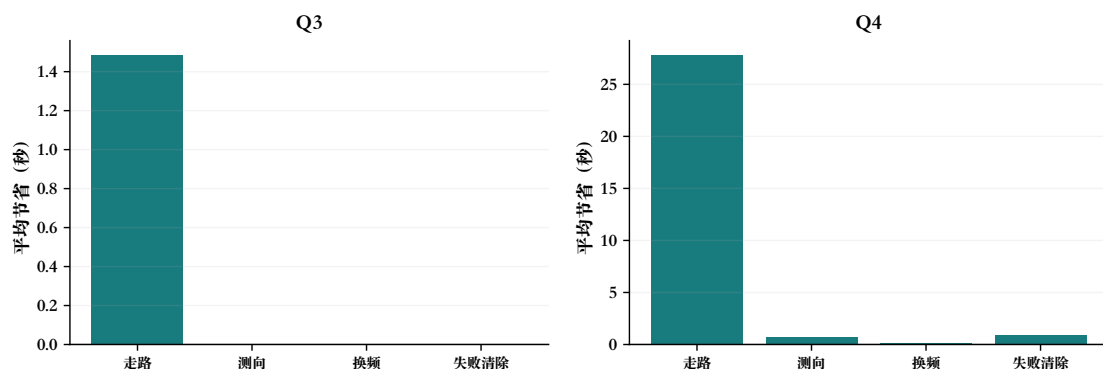


图 3: 独立集平均节省的计时分解。几何点的小幅移动可改变后续响应和控制分支，不能把全部差异直接归为移动量。

表 5: 五组压力测试，每组每问 40 局，候选 84 相对 56；全部清除

组别	Q3 节省秒	Q4 节省秒	Q3 更慢局	Q4 更慢局
边界外向	11.89	-23.98	0	15
共线端点	7.45	-8.02	0	17
近重合	0.17	-0.28	0	18
相关角误差	2.20	14.45	0	13
分区端点误差	4.29	60.23	0	15

五组均全部清除并不能推出每组都更快。最大 CPU 时间是本机本地计算观测值，不能据此证明官方网络下 20 分钟最坏墙钟约束；桥接器另设真实截止计时并在请求结果不确定时停止发送新动作。

8.4 公开协议内核和 HTTP 验证

候选 84 通过 10 个本地 Python 物理核案例和 6 个本地 HTTP 案例，逐局检查清除数。该内核由队内按照公开协议实现，**不是官方模拟器**。另执行 9 个 C++ 几何与控制回归程序、有理数剩余覆盖复核及传输超时测试。Q2 精确证书、Q3 七点覆盖证书和 Q4 固定点数下界均重新运行。在线机器人只链接传感器接口，不链接快速模拟器的真值结构。

8.5 官方正式结果的独立证据栏

题目要求 Q3 和 Q4 各三次官方正式测试及原始加密日志。当前目录中没有可核验的这些日志，故表6不填入本地种子或本地运行时间。正式运行应在官方界面就绪后使用桥接器进入；有限正式机会不由本地复现脚本触发。

表 6: 官方正式结果栏，当前尚无可核验记录

问题及次数	测试案例编码	清除数	单源均时	程序时间
Q3 第 1 次	待正式日志	待填	待填	待填
Q3 第 2 次	待正式日志	待填	待填	待填
Q3 第 3 次	待正式日志	待填	待填	待填
Q4 第 1 次	待正式日志	待填	待填	待填
Q4 第 2 次	待正式日志	待填	待填	待填
Q4 第 3 次	待正式日志	待填	待填	待填

9 模型评价与结论

本轮贡献主要是把连续几何目标与时间目标的差别落实为可执行检验：全部可能读数的角区间外包，揭示保证第二测邻域的空集边界，建立含历史负信息的连续覆盖不变量，以及把光学巡游邻域正确写成覆盖子区域的清除位置交集。

这些结构并未产生预期的两位数时间下降。半径贪心、有限续跑和光学邻域巡游均有明确负结果或微小收益；动态删除整点尚未成功。冻结独立集候选 Q4 虽平均节省 29.62 秒，但区间跨零，因此只将其作为进一步研究的可行算法交付，默认仍为方法 78。覆盖证书只证明安全可行；时间效率仍取决于观测场、发现顺序与后续联合控制。当前论文的可复现数值为本地研究结果，正式测试栏完成前不构成可直接提交的最终正式成绩版本。

参考文献

- [1] Vander Hook J, Tokekar P, Isler V. Cautious Greedy Strategy for Bearing-Only Active Localization: Analysis and Field Experiments[J]. Journal of Field Robotics, 2014, 31:296–318. https://josh.vanderhook.info/media/pdf/JoshV_JFR_2013_Localization.pdf.
- [2] Bayram H, Vander Hook J, Isler V. Gathering Bearing Data for Target Localization[J]. IEEE Robotics and Automation Letters, 2016, 1(1):369–374. doi:10.1109/LRA.2016.2521387.
- [3] Kaya C Y. Observer Path Planning for Maximum Information[J]. Optimization, 2021. doi:10.1080/02331934.2021.2011868. <https://arxiv.org/abs/2103.02059>.
- [4] Hoffmann F, Schily H, Krestel M, et al. Non-myopic Sensor Path Planning for Emitter Localization with a UAV[C]//FUSION 2023. doi:10.23919/FUSION52260.2023.10224174.
- [5] Sunberg Z, Kochenderfer M. Online algorithms for POMDPs with continuous state, action, and observation spaces[C]//ICAPS, 2018. Extended version: <https://arxiv.org/abs/1709.06196>.
- [6] Fekete S P, Krupke D, Perk M, et al. The Lawn Mowing Problem: From Algebra to Algorithms[C]//ESA, 2023, LIPIcs 274:45:1–45:18. doi:10.4230/LIPIcs.ESA.2023.45.
- [7] Peng C, Wei M, Isler V. Stochastic Traveling Salesperson Problem with Neighborhoods for Object Detection[C]//ICRA, 2023. Author archive deposited 2024: <https://arxiv.org/abs/2407.06366>.
- [8] Dumitrescu A, Mitchell J S B. Approximation algorithms for TSP with neighborhoods in the plane[J]. Journal of Algorithms, 2003, 48(1):135–159. <https://arxiv.org/abs/1703.01640>.

A 支撑材料与复现

文件位置	内容
src/ 与 review/	可编译 C++ 在线机器人、几何、规划、覆盖证书及验证脚本
results/ artifacts/	每局开发、独立和压力 CSV 及本地 HTTP 动作日志 冻结配置、执行命令、哈希、区间统计、几何证书及复核结果
evidence/ paper/ 与 figures/ vendor/mocksim/ run_all.py	文献核验、旧版本哈希与原数学推导 本文 LaTeX 源码、自动生成数表与独立科研图 来自所给压缩包的非官方公开协议模拟内核 重新编译、运行测试并重建实验汇总

编译使用 C++17 与优化级别 O2，不允许 fast-math；该选项会破坏向外舍入认证的前提。新会话由原点开始；非零位置误差采用旧后备策略。支撑包不包含官方正式日志，待正式测试导出后保持原文件名加入。

B 完整计算源程序

此附录列出在线机器人、实验内核、消融模块和几何证书的完整源码。历史方法分支仅供复核，不表示默认启用。模块标题与支撑包同名。

src/active.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 namespace active {
4 // Envelope over bearing intervals; only a ranking surrogate, not a certified
5 // time bound. Real updates continue using the original conservative polygon.
6 inline double posterior_radius(const geo::Poly&p,geo::Pt q){
7   auto c=geo::mec(p);double reference=atan2(c.c.y-q.y,c.c.x-q.x)*180/geo::pi;
8   double lo=1e99,hi=-1e99;
9   for(auto v:p){double a=atan2(v.y-q.y,v.x-q.x)*180/geo::pi-reference;a=std::remainder(a,360.);lo=std::min(
10     lo,a);hi=std::max(hi,a);}
11   if(hi-lo>=179)return c.r;
12   lo-=1.005;hi+=1.005;double step=(hi-lo)/24,worst=0;
13   for(int j=0;j<24;++j){auto post=geo::observe(p,q,reference+lo+(j+.5)*step,1.005+step/2);if(!post.empty())
14     worst=std::max(worst,geo::mec(post).r);}
15   return std::min(c.r,worst);
16 }
17 inline geo::Pt choose(const geo::Poly&p,geo::Pt pos,geo::Pt first,double angle,double A,double B,int mode){
18   auto c=geo::mec(p);auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};
19   geo::Pt q1=first+u*A+v*B,q2=first+u*A-v*B;
20   geo::Pt best=geo::dist(pos,q1)<geo::dist(pos,q2)?q1:q2;
21   // Longest chord is an uncertainty-axis estimate, not a stochastic covariance.
22   double longest=0;geo::Pt axis=u;
23   for(auto a:p)for(auto b:p){double dd=geo::dist(a,b);if(dd>longest){longest=dd;axis=(a-b)/dd;}}
24   geo::Pt transverse{-axis.y,axis.x};
25   geo::Poly candidates{q1,q2};
26   for(double along:{-.2,0.,.2})for(double side:{-1.,1.})for(double d:{50.,100.,200.,300.})candidates.
27     push_back(c.c+axis*(along*longest)+transverse*(side*d));
28   double bestscore=1e100;double weight=mode==1?1.:3.;
29   for(auto q:candidates){
30     if(geo::contains(p,q)||geo::norm(q)>2671.)continue;
31     double nearest=1e100,farthest=0;

```

```

29   for(size_t i=0;i<p.size();++i){nearest=std::min(nearest,geo::segment_distance(q,p[i],p[(i+1)%p.size()]));
        farthest=std::max(farthest,geo::dist(q,p[i]));}
30   if(nearest<10||farthest>999.9)continue;
31   double r=posterior_radius(p,q);
32   double score=geo::dist(pos,q)/5+6+geo::dist(q,c.c)/5+weight*r/5;
33   if(score<bestscore){bestscore=score;best=q;}
34 }return best;
35 }
36 }

```

src/adaptive_optical.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include <stdexcept>
4 namespace adaptive_optical {
5 #define B_VARIABLE_COVER 1
6 inline geo::Poly variable_cover(const geo::Poly&p,geo::Pt first,double angle,std::vector<geo::Poly>*
        certificate=nullptr){
7   if(certificate)certificate->clear();if(p.empty())return {};
8   auto whole=geo::mec(p);if(whole.r<20-1e-4){if(certificate)certificate->push_back(p);return {whole.c};}
9   auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};geo::Poly local;
10  double xmin=INFINITY,xmax=-INFINITY,ymin=INFINITY,ymax=-INFINITY;
11  for(auto z:p){auto d=z-first;local.push_back({geo::dot(d,u),geo::dot(d,v)});xmin=std::min(xmin,local.back
        ().x);xmax=std::max(xmax,local.back().x);ymin=std::min(ymin,local.back().y);ymax=std::max(ymax,local.
        back().y);}
12  struct Packing {bool valid=true;geo::Poly centers;std::vector<geo::Poly>cells;};
13  auto pack=[&](double left,double right,int rows){Packing result;
14    auto strip=geo::clip(geo::clip(local,-1,0,-left+1e-6},{1,0,right+1e-6});
15    if(strip.empty()){result.valid=false;return result;}
16    double bottom=INFINITY,top=-INFINITY;for(auto z:strip){bottom=std::min(bottom,z.y);top=std::max(top,z.y)
        ;}
17    for(int j=0;j<rows;++j){double a=bottom+(top-bottom)*j/rows,b=bottom+(top-bottom)*(j+1)/rows;
18      auto cell=geo::clip(geo::clip(strip,{0,-1},-a+1e-6},{0,1},b+1e-6);if(cell.empty())continue;
19      auto circle=geo::mec(cell);if(!(circle.r<20-1e-4)){result.valid=false;return result;}
20      result.centers.push_back(circle.c);result.cells.push_back(std::move(cell));
21    }return result;
22  };
23  geo::Poly out;double left=xmin;int strips=0;
24  do{
25    double remaining=xmax-left,chosen_width=-1,efficiency=-1;Packing chosen;
26    std::vector<int> row_counts{1,2};int fallback_rows=std::max(1,int(ceil((ymax-ymin)/28)));
27    if(fallback_rows>2)row_counts.push_back(fallback_rows);
28    for(int rows=row_counts){double low=0,high=std::min(39.99,std::max(0.,remaining));
29      auto start=pack(left,left,rows);if(!start.valid)continue;
30      auto full=pack(left,left+high,rows);
31      if(full.valid)low=high;else for(int iteration=0;iteration<14;++iteration){double mid=(low+high)/2;if(
        pack(left,left+mid,rows).valid)low=mid;else high=mid;}
32      auto candidate=pack(left,left+low,rows);if(!candidate.valid||candidate.centers.empty())continue;
33      double score=(remaining<=1e-7?1.:low)/candidate.centers.size();
34      if(score>efficiency){efficiency=score;chosen_width=low;chosen=std::move(candidate);}
35    }
36    if(chosen_width<0||remaining>1e-7&&chosen_width<1e-7)throw std::runtime_error("variable cover made no
        progress");
37    if(strips%2){std::reverse(chosen.centers.begin(),chosen.centers.end());std::reverse(chosen.cells.begin(),
        chosen.cells.end());}
38    for(size_t i=0;i<chosen.centers.size();++i){auto c=chosen.centers[i];out.push_back(first+u*c.x+v*c.y);
39      if(certificate){geo::Poly cell;for(auto z:chosen.cells[i])cell.push_back(first+u*z.x+v*z.y);certificate
        ->push_back(std::move(cell));}
40    }
41    left+=chosen_width;if(++strips>10000)throw std::runtime_error("variable cover exceeds strip budget");
42  }while(left<xmax-1e-7);
43  return out;
44 }

```

```

45 inline geo::Poly cover(const geo::Poly&p,geo::Pt first,double angle,std::vector<geo::Poly>*certificate=
    nullptr){
46   if(certificate)certificate->clear();
47   if(p.empty())return {};auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};geo::Poly local;
48   double xmin=1e99,xmax=-1e99;for(auto z:p){auto d=z-first;local.push_back({geo::dot(d,u),geo::dot(d,v)});
    xmin=std::min(xmin,local.back().x);xmax=std::max(xmax,local.back().x);}
49   int nx=std::max(1,int(ceil((xmax-xmin)/28)));geo::Poly out;
50   for(int i=0;i<nx;++i){double left=xmin+i*(xmax-xmin)/nx,right=xmin+(i+1)*(xmax-xmin)/nx;auto cell=geo::
    clip(geo::clip(local,{-1,0},-left+1e-5},{1,0},right+1e-5);if(cell.empty())continue;
51   auto c=geo::mec(cell);if(c.r<20-1e-5){out.push_back(first+u*c.c.x+v*c.c.y);if(certificate){geo::Poly
    global;for(auto z:cell)global.push_back(first+u*z.x+v*z.y);certificate->push_back(global);}continue;}
52   double bottom=1e99,top=-1e99;for(auto z:cell){bottom=std::min(bottom,z.y);top=std::max(top,z.y);}int ny=
    std::max(1,int(ceil((top-bottom)/28)));
53   for(int jj=0;jj<ny;++jj){int j=i%2?ny-1-jj:jj;double a=bottom+j*(top-bottom)/ny,b=bottom+(j+1)*(top-
    bottom)/ny;
54   auto part=geo::clip(geo::clip(cell,{0,-1},-a+1e-5},{0,1},b+1e-5);if(part.empty())continue;auto circle=
    geo::mec(part);
55   if(circle.r>=20-1e-5)throw std::runtime_error("adaptive optical subcell radius exceeds20");out.push_back
    (first+u*circle.c.x+v*circle.c.y);if(certificate){geo::Poly global;for(auto z:part)global.push_back(
    first+u*z.x+v*z.y);certificate->push_back(global);}
56  }
57 }return out;
58 }
59 }

```

src/bearing_types.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 namespace dynamic_selection {
4 struct Event {geo::Pt q;int kind;double angle=0;};
5 struct Hypothesis {geo::Pt z;double radius;bool directional;double heading;double weight;};
6 }

```

src/belief_dp.hpp

```

1 #pragma once
2 #define B_CONTEXT_TERMINAL 1
3 #include <vector>
4 #include <functional>
5 #include <cstdint>
6 #include <stdexcept>
7 #include <map>
8 #include <tuple>
9 #include <cmath>
10 #include <algorithm>
11 namespace belief_dp {
12 struct Action {bool measure;std::vector<int>outcome;std::vector<double>seconds;};
13 struct Model {std::vector<double>weights;std::vector<Action>actions;std::vector<std::vector<double>>travel;
    std::function<double(uint64_t,int)>terminal;std::function<int(int,int,int)>advance;std::function<
    double(uint64_t,int,int)>terminal_context;};
14 struct Result {int action=-1;double value=0;int expanded=0;bool exact=false,budget_hit=false;std::vector<
    double>action_values;};
15 inline Result solve(const Model&m,int depth,bool tuned,int limit){
16   const int n=m.weights.size(),a=m.actions.size();
17   if(n<1||n>63||depth<0||depth>5||limit<0||!m.terminal||m.travel.size()!=size_t(a+1))throw std::
    invalid_argument("invalid DP dimensions");
18   for(double w:m.weights)if(!(w>0)||!std::isfinite(w))throw std::invalid_argument("invalid hypothesis weight
    ");
19   for(const auto &row:m.travel){if(row.size()!=size_t(a+1))throw std::invalid_argument("invalid travel
    matrix");for(double x:row)if(x<0||!std::isfinite(x))throw std::invalid_argument("invalid travel cost")
    ;}
20   for(const auto &x:m.actions){if(x.outcome.size()!=size_t(n)||x.seconds.size()!=size_t(n))throw std::
    invalid_argument("invalid action dimensions");for(double c:x.seconds)if(c<0||!std::isfinite(c))throw

```

```

        std::invalid_argument("invalid action cost");}
21 if(bool(m.advance)!=bool(m.terminal_context))throw std::invalid_argument("both context callbacks required
    ");
22 Result result;result.action_values.assign(a,INFINITY);std::map<std::tuple<uint64_t,int,int,bool,int>,std::
    pair<double,int>>memo;
23 std::map<std::tuple<uint64_t,int,int>,double>leaves;std::map<uint64_t,double>masses;
24 auto mass=[&](uint64_t mask){auto it=masses.find(mask);if(it!=masses.end())return it->second;double w=0;
    for(int i=0;i<n;++i)if(mask>>i&1)w+=m.weights[i];masses[mask]=w;return w;};
25 auto terminal=[&](uint64_t mask,int at,int context){auto key=std::make_tuple(mask,at,context);auto it=
    leaves.find(key);if(it!=leaves.end())return it->second;double v=m.terminal_context?m.terminal_context(
    mask,at,context):m.terminal(mask,at);if(v<0||!std::isfinite(v))throw std::invalid_argument("invalid
    terminal policy cost");leaves[key]=v;return v;};
26 std::function<std::pair<double,int>(uint64_t,int,int,bool,int)>value;
27 value=[&](uint64_t mask,int at,int h,bool tunedNow,int context){
28     auto key=std::make_tuple(mask,at,h,tunedNow,context);auto it=memo.find(key);if(it!=memo.end())return it->
        second;
29     std::pair<double,int>best{terminal(mask,at,context),-1};if(h==0)return best;
30     if(result.expanded>=limit){result.budget_hit=true;return best;}++result.expanded;
31     double total=mass(mask);
32     for(int j=0;j<a;++j){const auto &act=m.actions[j];std::map<int,uint64_t>groups;
33         double cost=m.travel[at][j]+(act.measure&&!tunedNow?1.:0.);
34         for(int i=0;i<n;++i)if(mask>>i&1){cost+=m.weights[i]/total*act.seconds[i];groups[act.outcome[i]]|=
            uint64_t(1)<<i;}}
35         // No state reduction and no terminal success: extra action is uninformative.
36         if(!m.advance&&groups.size()==1&&groups.begin()->first>=0)continue;
37         for(auto [out,sub]:groups)if(out>=0)cost+=mass(sub)/total*value(sub,j,h-1,tunedNow||act.measure,m.
            advance?m.advance(context,j,out):0).first;
38         if(at==a&&h==depth)result.action_values[j]=cost;
39         if(cost<best.first-1e-10)best={cost,j};
40     }
41     memo[key]=best;return best;
42 };
43 auto best=value((uint64_t(1)<<n)-1,a,depth,tuned,0);result.value=best.first;result.action=best.second;
    result.exact=!result.budget_hit;return result;
44 }
45 }

```

src/certified_route.hpp

```

1 #pragma once
2 #include <vector>
3 #include <algorithm>
4 #include <cmath>
5 #include <limits>
6 #include <numeric>
7 #include <stdexcept>
8 namespace cert_route {
9 struct Problem{std::vector<double>start;std::vector<std::vector<double>>edge;};
10 struct Solution{std::vector<int>path;double lower=0,upper=0;bool exact=false,local_optimal=false;};
11 inline double length(const Problem&p,const std::vector<int>&r){if(r.empty())return 0;double c=p.start[r
    [0]];for(size_t i=1;i<r.size();++i)c+=p.edge[r[i-1]][r[i]];return c;}
12 inline double mst_lower(const Problem&p){int n=p.start.size();std::vector<double>d(n+1,1e100);std::vector<
    bool>done(n+1);d[0]=0;double total=0;
13 for(int k=0;k<n;++k){int u=-1;for(int i=0;i<n;++i)if(!done[i]&&(u<0||d[i]<d[u]))u=i;done[u]=true;total+=
    d[u];
14 for(int j=0;j<n;++j)if(!done[j]){double w=u==0?p.start[j-1]:j==0?p.start[u-1]:std::min(p.edge[u-1][j-1],
    p.edge[j-1][u-1]);d[j]=std::min(d[j],w);}}
15 return total;
16 }
17 inline double assignment_lower(const Problem&p){int n=p.start.size()+1;double inf=1e12;std::vector<double>u
    (n+1),v(n+1);std::vector<int>match(n+1),way(n+1);
18 auto cost=[&](int i,int j){--i;--j;if(i==j)return inf;if(j==0)return 0.;if(i==0)return p.start[j-1];return
    p.edge[i-1][j-1];};
19 for(int i=1;i<n;++i){match[0]=i;int j0=0;std::vector<double>minv(n+1,inf);std::vector<bool>used(n+1);
20 do{used[j0]=true;int i0=match[j0],j1=0;double delta=inf;

```

```

21   for(int j=1;j<=n;++j){if(!used[j]){double cur=cost(i0,j)-u[i0]-v[j];if(cur<minv[j]){minv[j]=cur;way[j]=j0
    ;}if(minv[j]<delta){delta=minv[j];j1=j;}}
22   for(int j=0;j<=n;++j){if(used[j]){u[match[j]]+=delta;v[j]-=delta;}else minv[j]-=delta;j0=j1;
23   }while(match[j0]);
24   do{int j1=way[j0];match[j0]=match[j1];j0=j1;}while(j0);
25   }
26   // Return a conservatively adjusted feasible assignment dual, not a cycle
27   // cover primal rounded upward. The adjustment exceeds arithmetic residuals.
28   double violation=0;for(int i=1;i<=n;++i)for(int j=1;j<=n;++j)violation=std::max(violation,u[i]+v[j]-cost(i
    ,j));
29   double dual=0;for(int i=1;i<=n;++i)dual+=u[i]+v[i];return std::max(0.,dual-n*(violation+1e-8));
30   }
31   inline std::vector<int> nearest(const Problem&p,int first=-1){int n=p.start.size();std::vector<bool>used(n)
    ;std::vector<int>r;
32   if(first>=0){r.push_back(first);used[first]=true;}
33   while((int)r.size()<n){int j=-1;double best=1e100;for(int k=0;k<n;++k){if(!used[k]){double c=r.empty()?p.
    start[k]:p.edge[r.back()][k];if(c<best){best=c;j=k;}}r.push_back(j);used[j]=true;}return r;
34   }
35   inline bool improve(const Problem&p,std::vector<int>&r,bool relocate){
36   for(int round=0;round<100;++round){double best=length(p,r)-1e-7;std::vector<int>choice;int n=r.size();
37   for(int i=0;i<n;++i)for(int j=i+1;j<n;++j){auto test=r;std::reverse(test.begin()+i,test.begin()+j+1);
    double c=length(p,test);if(c<best){best=c;choice=test;}}
38   if(relocate)for(int i=0;i<n;++i)for(int j=0;j<n;++j){if(i!=j){auto test=r;int x=test[i];test.erase(test.
    begin()+i);test.insert(test.begin()+j,x);double c=length(p,test);if(c<best){best=c;choice=test;}}
39   if(choice.empty())return true;r=std::move(choice);
40   }return false;
41   }
42   inline Solution solve(const Problem&p){Solution s;int n=p.start.size();if(!n){s.exact=s.local_optimal=true;
    return s;}
43   if(n<=12){int states=1<<n;std::vector<double>dp(states*n,1e100);std::vector<signed char>parent(states*n
    ,-1);
44   for(int j=0;j<n;++j)dp[(1<<j)*n+j]=p.start[j];
45   for(int mask=1;mask<states;++mask)for(int j=0;j<n;++j){if(mask&(1<<j)){double c=dp[mask*n+j];for(int k=0;k
    <n;++k){if(!(mask&(1<<k))){int index=(mask|(1<<k))*n+k;double v=c+p.edge[j][k];if(v<dp[index]){dp[index
    ]=v;parent[index]=j;}}}
46   int mask=states-1,last=0;for(int j=1;j<n;++j){if(dp[mask*n+j]<dp[mask*n+last])last=j;s.upper=dp[mask*n+
    last];
47   while(mask){s.path.push_back(last);int prev=parent[mask*n+last];mask^=1<<last;last=prev;}std::reverse(s.
    path.begin(),s.path.end());s.lower=s.upper;s.exact=s.local_optimal=true;return s;
48   }
49   s.path=nearest(p);improve(p,s.path,false);s.upper=length(p,s.path);
50   for(int first=0;first<n;++first){auto r=nearest(p,first);improve(p,r,false);double c=length(p,r);if(c<s.
    upper-1e-7){s.upper=c;s.path=r;}}
51   s.local_optimal=improve(p,s.path,true);s.upper=length(p,s.path);
52   s.lower=std::max(assignment_lower(p),std::max(0.,mst_lower(p)-1e-6));
53   if(s.lower>s.upper+1e-5)throw std::runtime_error("route lower bound exceeds feasible route");return s;
54   }
55   }

```

src/configurations.hpp

```

1 #pragma once
2 #include "policy.hpp"
3 inline Options configuration(int method){
4   Options o;o.fixedLayout=21;o.routing=2;o.opportunistic=true;
5   if(method==1){o.secondA=850;o.secondB=400;}
6   if(method==2){o.secondA=843.1178;o.secondB=545.4003;}
7   if(method>=3)o.stop_when_found16=true;
8   if(method==4)o.route_model=1;
9   if(method==5)o.route_model=2;
10  return o;
11 }

```

src/continuous_points.hpp

```
1 #pragma once
2 #include "geometry.hpp"
3 #define B_CONTINUOUS_POINTS 1
4 namespace continuous_points {
5 template<class Score, class Valid>
6 geo::Poly refine(const geo::Poly& seeds, Score score, Valid valid, int budget, double scale){
7     std::vector<std::pair<double, geo::Pt>> evaluated;
8     int calls=0;
9     auto evaluate=[&](geo::Pt q){if(calls>=budget||!valid(q))return;
10     for(auto x:evaluated)if(geo::dist(q,x.second)<1e-7)return;
11     ++calls;double value=score(q);if(!std::isfinite(value))return;evaluated.push_back({value,q});
12 };
13 for(auto q:seeds)evaluate(q);if(evaluated.empty())return {};
14 auto best=[&]() {return std::min_element(evaluated.begin(),evaluated.end(),[](auto a,auto b){return a.first
    <b.first;})->second;};
15 for(int level=0;level<6&&calls<budget;++level){
16     for(int repeat=0;repeat<2&&calls<budget;++repeat){auto center=best();
17         for(int k=0;k<8;++k)evaluate(center+geo::polar(scale,45.*k));
18         if(geo::dist(center,best())<1e-7)break;
19     }scale*=.5;
20 }
21 std::stable_sort(evaluated.begin(),evaluated.end(),[](auto a,auto b){return a.first<b.first;});
22 geo::Poly out;for(auto x:evaluated){out.push_back(x.second);if(out.size()==6)break;}return out;
23 }
24 }
```

src/discovery.hpp

```
1 #pragma once
2 #include "dynamic_selection.hpp"
3 namespace discovery {
4 class Belief {
5     std::vector<dynamic_selection::Hypothesis> particles;
6 public:
7     Belief()=default;
8     explicit Belief(int problem){for(int i=1;i<=2048;++i){double r=1800*sqrt(dynamic_selection::radical(i,2)),
        a=360*dynamic_selection::radical(i,3);particles.push_back({geo::polar(r,a),1000+500*dynamic_selection
        ::radical(i,5),problem==4&&dynamic_selection::radical(i,11)<.5,360*dynamic_selection::radical(i,7),1})
        ;}}
9     size_t size()const{return particles.size();}
10    double probability(geo::Pt q)const{if(particles.empty())return 0;int yes=0;for(auto h:particles)yes+=
        dynamic_selection::receives(h,q);return double(yes)/particles.size();}
11    void negative_scan(geo::Pt q){particles.erase(std::remove_if(particles.begin(),particles.end(),[&](auto h)
        {return dynamic_selection::receives(h,q);}),particles.end());}
12 };
13 }
```

src/dynamic_selection.hpp

```
1 #pragma once
2 #include "geometry.hpp"
3 #include "belief_dp.hpp"
4 #include "bearing_types.hpp"
5 #include "optical_terminal.hpp"
6 #include "continuous_points.hpp"
7 #include "robust_geometry.hpp"
8 #include <numeric>
9 namespace dynamic_selection {
10 #define B_REMAINING_DEPTH 1
11 #define B_BEARING_BINS 1
12 inline int bearing_outcome(double theta,int width=2){if(width<=0||360%width)throw std::invalid_argument("
    invalid bearing bin");double a=fmod(theta+720.,360.);if(a<0)a+=360.;return 10+int(floor(a/width));}
```

```

13 inline int remaining_depth(int requested,int allowed,int used){return std::max(0,std::min(requested,allowed
-used));}
14 inline double finite_terminal_cost(const std::vector<Hypothesis>&hypotheses,uint64_t mask,geo::Pt now,geo::
Pt first,geo::Pt u,geo::Pt v){
15
16 double xmin=1e99,xmax=-1e99,ymin=1e99,ymax=-1e99,total=0;
17 for(int i=0;i<(int)hypotheses.size();++i){if(mask>>i&1){auto d=hypotheses[i].z-first;double x=geo::dot(d,u
),y=geo::dot(d,v);xmin=std::min(xmin,x);xmax=std::max(xmax,x);ymin=std::min(ymin,y);ymax=std::max(ymax
,y);total+=hypotheses[i].weight;}
18 int nx=std::max(1,int(ceil((xmax-xmin)/28))),ny=std::max(1,int(ceil((ymax-ymin)/28)));geo::Poly sweep;
19 for(int i=0;i<nx;++i)for(int jj=0;jj<ny;++jj){int j=i%2?ny-1-jj:jj;sweep.push_back(first+u*(xmin+(i+.5)*(
xmax-xmin)/nx)+v*(ymin+(j+.5)*(ymax-ymin)/ny));}
20 double best=1e100;
21 for(int reverse=0;reverse<2;++reverse){if(reverse)std::reverse(sweep.begin(),sweep.end());double
cumulative=0,expected=0;uint64_t remaining=mask;auto position=now;
22 for(auto q:sweep){cumulative+=geo::dist(position,q)/5;position=q;for(int i=0;i<(int)hypotheses.size();++
i){if((remaining>>i&1)&&geo::dist(q,hypotheses[i].z)<=20+1e-7){expected+=hypotheses[i].weight/total*(
cumulative+5);remaining&=~(uint64_t(1)<<i);if(!remaining)break;cumulative+=3;}
23 if(remaining)throw std::runtime_error("finite terminal sweep failed");best=std::min(best,expected);
24 }return best;
25
26 }
27 struct Alternative {geo::Pt point;bool measure;double value;};
28 struct Choice {bool available=false,measure=true,budget_hit=false,exact=false;geo::Pt point;double value=0;
int depth=0,hypotheses=0,expanded=0;std::vector<Alternative>alternatives;};
29 inline bool receives(const Hypothesis&h,geo::Pt q){return geo::dist(h.z,q)<=h.radius+1e-9&&(!h.directional
||geo::dot(q-h.z,geo::polar(1,h.heading))>=-1e-9);}
30 inline bool consistent(const Hypothesis&h,const std::vector<Event>&events){
31 for(auto e:events){double distance=geo::dist(h.z,e.q);
32 if(e.kind==3){if(distance<=20)return false;continue;}
33 if(e.kind==4){if(distance>20)return false;continue;}
34 bool heard=receives(h,e.q);
35 if(e.kind==0){if(heard)return false;}
36 else if(e.kind==1){if(!heard||distance>5)return false;}
37 else if(e.kind==2){double angle=atan2(h.z.y-e.q.y,h.z.x-e.q.x)*180/geo::pi;if(!heard||distance<=5||std:::
abs(std::remainder(angle-e.angle,360.))>1.0050001)return false;}
38 }return true;
39 }
40 inline double radical(int i,int base){double a=0,b=1.;while(i){b/=base;a+=b*(i%base);i/=base;}return a;}
41 inline std::vector<Hypothesis> make_hypotheses(const geo::Poly&p,const std::vector<Event>&history,int
problem,int cap){
42 if(p.empty()||cap<1||cap>63)return {};
43 geo::Pt center;for(auto q:p)center=center+q;center=center/double(p.size());
44 std::vector<double>areas;double area=0;
45 for(size_t i=0;i<p.size();++i){area+=std::abs(geo::cross(p[i]-center,p[(i+1)%p.size()]-center));areas.
push_back(area);}
46 geo::Poly positions{center};
47 for(int i=1;i<cap;++i){
48 if(area<1e-10){positions.push_back(p.front()*(1-i/double(cap))+p.back()*(i/double(cap)));continue;}
49 size_t tri=std::lower_bound(areas.begin(),areas.end(),radical(i,2)*area)-areas.begin();double a=sqrt(
radical(i,3)),b=radical(i,5);
50 positions.push_back(center*(1-a)+p[tri]*(a*(1-b))+p[(tri+1)%p.size()]*(a*b));
51 }
52 std::vector<Hypothesis>all;
53 for(auto z:positions){
54 double low=1000;for(auto e:history){if(e.kind==1||e.kind==2)low=std::max(low,geo::dist(z,e.q));if(low
>1500+1e-7)continue;low=std::min(low,1500.);
55 std::vector<double>headings;for(int i=0;i<12;++i)headings.push_back(i*30.);
56 // Event boundary midpoints sample even very narrow feasible heading arcs.
57 std::vector<double>cuts{0};for(auto e:history){if(e.kind<=2){double a=atan2(e.q.y-z.y,e.q.x-z.x)*180/geo::
pi;for(double d:{-90.,90.})cuts.push_back(fmod(a+d*720.,360.));}
58 std::sort(cuts.begin(),cuts.end());for(size_t i=0;i<cuts.size();++i){double b=i+1<cuts.size()?cuts[i+1]:
cuts[0]+360.;headings.push_back(fmod((cuts[i]+b)/2,360.));}
59 for(double r:{low,(low+1500)/2,1500.}){
60 Hypothesis h{z,r,false,0,problem==3?1.:.5};if(consistent(h,history))all.push_back(h);

```

```

61     if(problem==4)for(double a:headings){h={z,r,true,a,.5/headings.size()};if(consistent(h,history))all.
        push_back(h);}
62 }
63 }
64 if(all.size()<=size_t(cap))return all;
65 double total=0;for(auto h:all)total+=h.weight;
66 std::vector<Hypothesis>selected;double cumulative=all[0].weight;size_t j=0;
67 for(int i=0;i<cap;++i){double target=total*(i+.5)/cap;while(j+1<all.size()&&cumulative<target)cumulative+=
        all[++j].weight;auto h=all[j];h.weight=1.;selected.push_back(h);}
68 return selected;
69 }
70 inline geo::Poly retreat_points(geo::Pt center,const std::vector<Event>&history){
71     geo::Poly points;for(auto it=history.rbegin();it!=history.rend();++it)if(it->kind==2){double length=geo::
        dist(center,it->q);if(length<1)continue;auto ray=(it->q-center)/length;for(double d
        :{25.,50.,100.,200.})points.push_back(center+ray*std::min(d,.8*length));break;}return points;
72 }
73 inline geo::Poly safe_clear_points(const geo::Poly&p,geo::Pt now){
74     if(p.empty())return {};auto c=geo::mec(p);double r=20-c.r-1e-6;if(r<0)return {};
75     double d=geo::dist(now,c.c);geo::Poly out{d<=r?now:c.c+(now-c.c)*(r/d)};
76     for(int i=0;i<16;++i)out.push_back(c.c+geo::polar(r,i*22.5));return out;
77 }
78 inline Choice plan(const geo::Poly&p,geo::Pt now,geo::Pt first,double angle,const std::vector<Event>&
        history,int problem,int depth,bool tuned,int limit,bool retreat=false,int samples=32,bool
        clear_neighborhood=false,int terminal_mode=0,int refinement=0,int bearing_bin=2,int robust_mode=0){
79     Choice choice;choice.depth=depth;
80     auto hypotheses=make_hypotheses(p,history,problem,samples);choice.hypotheses=hypotheses.size();if(
        hypotheses.empty())return choice;
81     geo::Pt center=geo::mec(p).c,axis=geo::polar(1,angle);double longest=0;
82     for(auto a:p)for(auto b:p){double d=geo::dist(a,b);if(d>longest){longest=d,axis=(b-a)/d;}}
83     geo::Pt transverse{-axis.y,axis.x},u=geo::polar(1,angle),v{-u.y,u.x};
84     std::vector<geo::Pt>locations;std::vector<bool>measure;
85     auto add=[&](geo::Pt q,bool sensing){
86         if(!std::isfinite(q.x)||!std::isfinite(q.y)||geo::norm(q)>2671.99)return;
87         for(auto e:history)if((sensing?e.kind<=2:e.kind>=3)&&geo::dist(e.q,q)<1e-6)return;
88         for(size_t i=0;i<locations.size();++i)if(measure[i]==sensing&&geo::dist(locations[i],q)<1e-6)return;
89         locations.push_back(q);measure.push_back(sensing);
90     };
91     add(now,true);add(center,true);
92     if(retreat)for(auto q:retreat_points(center,history))add(q,true);
93     for(double side:{-1.,1.}){add((first+u*750+v*(side*300),true);for(double d:{50.,150.,300.})add((center+
        transverse*(side*d),true));}
94     for(double along:{-.25,.25})for(double side:{-1.,1.})add((center+axis*(along*longest)+transverse*(side*80),
        true);
95     add(center,false);for(double along:{-.4,-.2,.2,.4})add((center+axis*(along*longest),false);
96     if(clear_neighborhood)for(auto q:safe_clear_points(p,now))add(q,false);
97     if(refinement){geo::Poly seeds;for(size_t i=0;i<locations.size();++i)if(measure[i])seeds.push_back(
        locations[i]);
98     double total=0;for(auto h:hypotheses)total+=h.weight;
99     auto valid=[&](geo::Pt q){if(!std::isfinite(q.x)||!std::isfinite(q.y)||geo::norm(q)>2671.99)return false;
        for(auto e:history)if(e.kind<=2&&geo::dist(e.q,q)<1e-6)return false;return true;};
100     auto score=[&](geo::Pt q){double cost=geo::dist(now,q)/5+(tuned?0:1);std::map<int,uint64_t>groups;
101     for(size_t i=0;i<hypotheses.size();++i){auto h=hypotheses[i];double distance=geo::dist(q,h.z);int out=0;
        double seconds=5;
102         if(receives(h,q)){if(distance<=5){out=-1;seconds=10;}else{double theta=atan2(h.z.y-q.y,h.z.x-q.x)*180/
            geo::pi;out=bearing_outcome(theta,bearing_bin);}}
103         cost+=h.weight/total*seconds;groups[out]|=uint64_t(1)<<i;
104     }
105     for(auto [out,mask]:groups)if(out>=0){double mass=0;for(size_t i=0;i<hypotheses.size();++i)if(mask>>i&1)
        mass+=hypotheses[i].weight;cost+=mass/total*finite_terminal_cost(hypotheses,mask,q,first,u,v);}
106     return cost;
107 };
108 for(auto q:continuous_points::refine(seeds,score,valid,refinement==1?80:160,std::clamp(longest
        *.15,20.,180.)))add(q,true);
109 }
110 if(locations.empty())return choice;

```

```

111 belief_dp::Model model;for(auto h:hypotheses)model.weights.push_back(h.weight);
112 int n=locations.size();model.travel.assign(n+1,std::vector<double>(n+1));geo::Poly at=locations;at.
    push_back(now);
113 for(int i=0;i<n;++i)for(int j=0;j<n;++j)model.travel[i][j]=geo::dist(at[i],at[j])/5;
114 for(int j=0;j<n;++j){belief_dp::Action a;a.measure=measure[j];
115     for(auto h:hypotheses){double distance=geo::dist(h.z,locations[j]);int out;double seconds;
116         if(!a.measure){bool hit=distance<=20;out=hit?-1:0;seconds=hit?5:3;}
117         else if(!receives(h,locations[j])){out=0;seconds=5;}
118         else if(distance<=5){out=-1;seconds=10;}
119         else{double theta=atan2(h.z.y-locations[j].y,h.z.x-locations[j].x)*180/geo::pi;out=bearing_outcome(theta
            ,bearing_bin);seconds=5;}
120         a.outcome.push_back(out);a.seconds.push_back(seconds);
121     }model.actions.push_back(a);
122 }
123 model.terminal=[&](uint64_t mask,int from){return finite_terminal_cost(hypotheses,mask,at[from],first,u,v)
    ;};
124 if(terminal_mode&&bearing_bin!=2)throw std::invalid_argument("context terminal requires 2-degree bins");
125 optical_terminal::Contexts contexts(p,locations,measure,hypotheses,first,angle,terminal_mode>=2);
126 if(terminal_mode){model.advance=[&](int context,int action,int outcome){return contexts.advance(context,
    action,outcome);};model.terminal_context=[&](uint64_t mask,int from,int context){return contexts.value
    (mask,at[from],context);};}
127 auto result=belief_dp::solve(model,depth,tuned,limit);choice.value=result.value;choice.expanded=result.
    expanded;choice.budget_hit=result.budget_hit;choice.exact=result.exact;
128 if(result.action>=0){choice.available=true;choice.measure=measure[result.action];choice.point=locations[
    result.action];}
129 if(choice.available){for(int j=0;j<n;++j){if(std::isfinite(result.action_values[j]))choice.alternatives.
    push_back({locations[j],measure[j],result.action_values[j]});std::stable_sort(choice.alternatives.
    begin(),choice.alternatives.end(),[](const auto&a,const auto&b){return a.value<b.value;});if(choice.
    alternatives.size()>6)choice.alternatives.resize(6);}
130 if(robust_mode&&choice.available){
131     auto old=geo::mec(p);
132     int positive=0;for(auto e:history)positive+=e.kind==2;
133     if(old.r>35&&(robust_mode!=2||positive==1)){
134         auto fresh=robust_geometry::candidate(p,now,first,angle,history,problem,robust_mode==2?2:1);
135         if(fresh.valid){
136             double oldrho=choice.measure&&robust_geometry::reception(p,choice.point,history,problem)?
                robust_geometry::worst_radius(p,choice.point,128).upper:old.r;
137             double extra=std::max(0.,geo::dist(now,fresh.q)-geo::dist(now,choice.point))/5+5;
138             // Time-scaled diameter reduction is a selection heuristic, not a theorem
139             // about total time. The held-out paired test is the promotion gate.
140             bool accept=robust_mode==2?(fresh.rho<=35&&geo::dist(now,fresh.q)+25<geo::dist(now,choice.point)):(.4*(
                oldrho-fresh.rho)>extra&&fresh.rho<old.r*.85);
141             if(accept){choice.point=fresh.q;choice.measure=true;choice.alternatives={{fresh.q,true,choice.value}};}
142         }
143     }
144 }
145 return choice;
146 }
147 }

```

src/geometry.hpp

```

1 #pragma once
2 #include <algorithm>
3 #include <cmath>
4 #include <limits>
5 #include <string>
6 #include <vector>
7 namespace geo {
8 constexpr double pi=3.14159265358979323846;
9 struct Pt {double x=0,y=0; Pt operator+(Pt b)const{return{x+b.x,y+b.y};} Pt operator-(Pt b)const{return{x-b
    .x,y-b.y};} Pt operator*(double t)const{return{x*t,y*t};} Pt operator/(double t)const{return{x/t,y/t
    };}};
10 inline double dot(Pt a,Pt b){return a.x*b.x+a.y*b.y;}
11 inline double cross(Pt a,Pt b){return a.x*b.y-a.y*b.x;}

```

```

12 inline double norm(Pt a){return std::hypot(a.x,a.y);}
13 inline double dist(Pt a,Pt b){return norm(a-b);}
14 inline Pt polar(double r,double deg){double t=deg*pi/180;return{r*cos(t),r*sin(t)};}
15 using Poly=std::vector<Pt>;
16 struct HP{Pt a;double b;}; // a dot x <= b
17 inline Poly hull(Poly p){
18     std::sort(p.begin(),p.end(),[](Pt a,Pt b){return a.x!=b.x?a.x<b.x:a.y<b.y;});
19     p.erase(std::unique(p.begin(),p.end(),[](Pt a,Pt b){return dist(a,b)<1e-9;}),p.end());
20     if(p.size()<3)return p;
21     Poly h(2*p.size());int k=0;
22     for(auto x:p){while(k>=2&&cross(h[k-1]-h[k-2],x-h[k-1])<=1e-10)--k;h[k++]=x;}
23     int t=k+1;for(int i=(int)p.size()-2;i>=0;--i){auto x=p[i];while(k>=t&&cross(h[k-1]-h[k-2],x-h[k-1])<=1e-10)--k;h[k++]=x;}h.resize(k-1);return h;
24 }
25 inline Poly clip(const Poly&p,Pt a,double b){
26     Poly q;if(p.empty())return q;
27     for(size_t i=0;i<p.size();++i){Pt x=p[i],y=p[(i+1)%p.size()];double fx=dot(a,x)-b,fy=dot(a,y)-b;
28         bool ix=fx<=1e-9,iy=fy<=1e-9;if(ix&&iy)q.push_back(x);
29         if(ix!=iy){double t=fx/(fx-fy);q.push_back(x+(y-x)*t);}}
30     return q;
31 }
32 inline Poly outer_disk(Pt c,double r,int n=64){Poly p;double rr=r/cos(pi/n);for(int i=0;i<n;++i)p.push_back
    (c+polar(rr,(i+.5)*360/n));return p;}
33 inline Poly disk_clip(Poly p,Pt c,double r,int n=64){for(int i=0;i<n;++i){Pt a=polar(1,i*360./n);p=clip(p,a
    ,r+dot(a,c));}return p;}
34 inline Poly observe(Poly p,Pt s,double theta,double eps=1.005){
35     Pt u=polar(1,theta),v={-u.y,u.x};double t=tan(eps*pi/180);
36     for(Pt a:{v-u*t,v*(-1-u*t)})p=clip(p,a,dot(a,s));
37     return p;
38 }
39 inline bool contains(const Poly&p,Pt z){if(p.empty())return false;if(p.size()==1)return dist(p[0],z)<1e-7;
40     if(p.size()==2)return std::abs(cross(p[1]-p[0],z-p[0]))<1e-6&&dot(z-p[0],z-p[1])<=1e-6;
41     for(size_t i=0;i<p.size();++i)if(cross(p[(i+1)%p.size()]-p[i],z-p[i])< -1e-6)return false;
42     return true;}
43 inline double diameter(const Poly&p){double d=0;for(auto a:p)for(auto b:p)d=std::max(d,dist(a,b));return d
    ;}
44 struct Circle{Pt c;double r=-1;};
45 inline bool inside(Circle c,Pt p){return c.r>=0&&dist(c.c,p)<=c.r+1e-7;}
46 inline Circle paircircle(Pt a,Pt b){return{(a+b)/2,dist(a,b)/2};}
47 inline Circle triplecircle(Pt a,Pt b,Pt c){
48     Pt u=b-a,v=c-a;double d=2*cross(u,v);
49     if(std::abs(d)<1e-12){Circle best{};1e100;for(auto q:{paircircle(a,b),paircircle(a,c),paircircle(b,c)})
        if(inside(q,a)&&inside(q,b)&&inside(q,c)&&q.r<best.r)best=q;return best;}
50     double uu=dot(u,u),vv=dot(v,v);Pt o=a+Pt{(uu*v.y-vv*u.y)/d,(u.x*vv-v.x*uu)/d};return{o,dist(o,a);}
51 }
52 // Incremental enclosing circle; final radius enlarged to actual max vertex distance.
53 inline Circle mec(const Poly&p){Circle c;
54     for(size_t i=0;i<p.size();++i)if(!inside(c,p[i])){c={p[i],0};for(size_t j=0;j<i;++j)if(!inside(c,p[j])){c=
        paircircle(p[i],p[j]);for(size_t k=0;k<j;++k)if(!inside(c,p[k]))c=triplecircle(p[i],p[j],p[k]);}}
55     for(auto x:p)c.r=std::max(c.r,dist(x,c.c));
56     return c;
57 }
58 struct HResult{std::string kind;Poly p;};
59 inline HResult halfplanes(const std::vector<HP>&h){
60     auto feasible=[&](Pt x){for(auto q:h)if(dot(q.a,x)>q.b+1e-8)return false;return true;};
61     Poly v;bool exists=feasible({0,0});
62     for(auto q:h)if(dot(q.a,q.a)>0&&feasible(q.a*(q.b/dot(q.a,q.a))))exists=true;
63     for(size_t i=0;i<h.size();++i)for(size_t j=0;j<i;++j){auto a=h[i],b=h[j];double d=cross(a.a,b.a);if(std::
        abs(d)<1e-12)continue;
64     Pt x=((a.b*b.a.y-a.a.y*b.b)/d,(a.a.x*b.b-a.b*b.a.x)/d);if(feasible(x)){v.push_back(x);exists=true;}}
65     if(!exists)return{"empty",{}};
66     Poly dirs={{1,0},{-1,0},{0,1},{0,-1}};for(auto q:h){dirs.push_back({-q.a.y,q.a.x});dirs.push_back({q.a.y,-
        q.a.x});}
67     for(auto d:dirs)if(norm(d)>1e-12){d=d/norm(d);bool ok=true,uncertain=false;
68         for(auto q:h){double v=dot(q.a,d);if(v>1e-10)ok=false;if(v>0)uncertain=true;}}

```

```

69 // A positive violation accepted only by tolerance is not an unboundedness
70 // certificate: report uncertainty instead of inventing a recession ray.
71 if(ok)return{uncertain?"numerically_uncertain":"unbounded",{}};
72 }
73 return{"bounded",hull(v)};
74 }
75 inline double segment_distance(Pt z,Pt a,Pt b){Pt d=b-a;double t=dot(d,d)>0?dot(z-a,d)/dot(d,d):0;t=std::
    clamp(t,0.,1.);return dist(z,a+d*t);}
76 inline bool meets_disk(Poly p,double r){if(contains(hull(p),{0,0}))return true;for(size_t i=0;i<p.size();++
    i)if(segment_distance({0,0},p[i],p[(i+1)%p.size()])<=r+1e-7)return true;return false;}
77 inline Poly coverage(int problem,double h=950,bool square=false){
78 if(problem==3){Poly p={{0,0}};for(int i=0;i<6;++i)p.push_back(polar(1200,i*60));return p;}
79 Poly p={{0,0}};int n=(int)ceil(4000/h)+2;
80 auto v=[&](int i,int j){return square?Pt{i*h,j*h}:Pt{h*(i+.5*j),h*sqrt(3.)/2*j}};};
81 for(int i=-n;i<n;++i)for(int j=-n;j<n;++j){Pt a=v(i,j),b=v(i+1,j),c=v(i,j+1),d=v(i+1,j+1);
82 if(square){Poly q={a,b,d,c};if(meets_disk(q,1800))p.insert(p.end(),q.begin(),q.end());}
83 else for(auto q:{Poly{a,b,c},Poly{d,c,b}})if(meets_disk(q,1800))p.insert(p.end(),q.begin(),q.end());}
84 Poly unique;for(auto x:p){bool duplicate=false;for(auto y:unique)if(dist(x,y)<1e-6)duplicate=true;if(!
    duplicate)unique.push_back(x);}return unique;
85 }
86 }

```

src/joint_config.hpp

```

1 #pragma once
2 #include "configurations.hpp"
3 #define B_SHORTCUT_CONFIG 1
4 inline Options joint_configuration(int method,int problem){
5 if(method==84){auto o=joint_configuration(problem==3?78:81,problem);o.shortcut=true;return o;}
6 if(method==86){auto o=joint_configuration(83,problem);if(problem==4)o.neighborhood_variable=true;return o
    ;}
7 if(method==85){auto o=joint_configuration(81,problem);if(problem==4)o.residual_opportunistic=true;return o
    ;}
8 if(method>=79&&method<=84){auto o=joint_configuration(56,problem);if(method==79)o.robust_mode=1;if(method
    ==80)o.robust_mode=2;if(method==81&&problem==4)o.residual_cover=true;if(method==82)o.robust_mode=3;if(
    method==83)o.neighborhood_tour=true;return o;}
9 if(method==78){auto o=joint_configuration(56,problem);o.shortcut=true;return o;}
10 if(method>=74&&method<=77){auto o=joint_configuration(56,problem);if(problem==4)o.optical_portfolio=method
    -73;return o;}
11 if(method>=70&&method<=73){auto o=joint_configuration(56,problem);o.bearing_bin=method==70?4:method==71?3:
    method==72?6:8;return o;}
12 if(method>=67&&method<=69){auto o=joint_configuration(56,problem);o.refinement=method==69?2:1;if(method
    ==68&&problem==3)o.planning_samples=63;return o;}
13 if(method==66){auto o=joint_configuration(56,problem);o.budget_aware=true;return o;}
14 if(method>=61&&method<=65){auto o=joint_configuration(method==61|method==63?56:60,problem);if(problem==4)
    o.optical_search=method<=62?1:method<=64?2:3;return o;}
15 if(method==60){auto o=joint_configuration(56,problem);if(problem==4)o.variable_optical=true;return o;}
16 if(method==59){auto o=joint_configuration(56,problem);if(problem==4)o.terminal_mode=1;return o;}
17 if(method==58){auto o=joint_configuration(58,problem);if(problem==4)o.terminal_mode=2;return o;}
18 if(method>=55&&method<=57){auto o=joint_configuration(method==55?38:39,problem);if(problem==4)o.
    adaptive_optical=true;if(method==57)o.optical_weight=.5;return o;}
19 if(method==53|method==54){auto o=joint_configuration(39,problem);if(problem==4)o.dynamic_limit=method
    ==53?10:14;return o;}
20 if(method>=50&&method<=52){auto o=joint_configuration(39,problem);o.discovery_weight=method==50?.25:method
    ==51?.5:1.;return o;}
21 if(method>=45&&method<=49){auto o=joint_configuration(method==48?40:method==49?42:39,problem);o.
    immediate_fee_weight=method==45?.25:method==47?1:.5;return o;}
22 if(method>=40&&method<=44){auto o=joint_configuration(method==41?38:39,problem);if(method==40|method
    ==41|method==44)o.use_lns=true;if(method==42|method==44)o.optical_weight=.5;if(method==43)o.
    optical_weight=.25;return o;}
23 if(method==39){auto o=joint_configuration(38,problem);o.weighted_optical=true;return o;}
24 if(method==38)return joint_configuration(problem==3?32:27,problem);
25 if(method>=34&&method<=37){auto o=joint_configuration(method==34|method==37?29:method==35?21:27,problem);
    o.defer_single=true;if(method==36)o.auxiliary=1;if(method==37)o.q3_radius=1200;return o;}

```

```

26 if(method==32||method==33){auto o=joint_configuration(method==32?29:27,problem);o.clear_neighborhood=true;
    return o;}
27 auto o=configuration(5);o.dynamic_depth=2;o.active_mode=problem==3?1:0;
28 if(method==1)o.auxiliary=1;
29 if(method==2)o.joint_actions=true;
30 if(method==3){o.auxiliary=1;o.joint_actions=true;}
31 if(method==4)o.route_model=1;
32 if(method==5){o.auxiliary=2;o.joint_actions=true;}
33 if(method==6){o.auxiliary=2;}
34 if(method>=7){o.auxiliary=2;o.joint_actions=true;}
35 if(method==7)o.start_residual=.5;
36 if(method==8)o.start_residual=1;
37 if(method==9)o.start_residual=2;
38 if(method==10)o.retreat=true;
39 if(method==11){o.retreat=true;o.start_residual=1;}
40 if(method==12){o.auxiliary=0;o.direct_optical=true;}
41 if(method==13){o.direct_optical=true;o.optical_rotate=true;}
42 if(method==14){o.candidate_menu=true;}
43 if(method==15){o.candidate_menu=true;o.complete_source=true;}
44 if(method==16){o.candidate_menu=true;o.complete_source=true;o.direct_optical=true;o.optical_rotate=true;}
45 if(method==17){o.auxiliary=0;o.direct_optical=true;o.optical_rotate=true;}
46 if(method==18){o.auxiliary=0;o.direct_optical=true;o.retreat=true;}
47 if(method==19){o.auxiliary=3;o.joint_actions=false;}
48 if(method==20){o.auxiliary=3;}
49 if(method==21){o.auxiliary=4;o.direct_optical=true;}
50 if(method==22){o.auxiliary=4;o.joint_actions=false;}
51 if(method>=23){o.auxiliary=0;o.direct_optical=true;}
52 if(method==23||method==24||method==25||method==26||method==28)o.dynamic_depth=3;
53 if(method==24)o.optical_rotate=true;
54 if(method==25)o.candidate_menu=true;
55 if(method==26){o.candidate_menu=true;o.complete_source=true;}
56 if(method==27||method==28)o.planning_samples=63;
57 if(method==29){o.q3_radius=1124;o.candidate_menu=true;o.auxiliary=2;o.direct_optical=false;}
58 if(method==30){o.q3_radius=1124;o.dynamic_depth=3;o.optical_rotate=true;}
59 if(method==31){o.q3_radius=1150;o.candidate_menu=true;o.auxiliary=2;o.direct_optical=false;}
60 return o;
61 }

```

src/lns.hpp

```

1 #pragma once
2 #include "certified_route.hpp"
3 #include <random>
4 namespace new_route {
5 // Deterministic, non-neural destroy/repair prototype. This is not NLNS or LKH.
6 inline std::vector<int> repair(const cert_route::Problem&p,std::vector<int> r,std::vector<int> missing){
7     while(!missing.empty()){
8         int chosen=-1,at=-1;double regret=-1,bestCost=1e100;
9         for(int a=0;a<(int)missing.size();++a){double best=1e100,second=1e100;int index=0;
10             for(int j=0;j<=(int)r.size();++j){auto q=r;q.insert(q.begin()+j,missing[a]);double c=cert_route::length(
11                 p,q);if(c<best){second=best;best=c;index=j;}else second=std::min(second,c);}
12             double diff=second<1e99?second-best:0;
13             if(diff>regret+1e-8||(std::abs(diff-regret)<1e-8&&best<bestCost)){regret=diff;bestCost=best;chosen=a;at=
14                 index;}
15         }
16         r.insert(r.begin()+at,missing[chosen]);missing.erase(missing.begin()+chosen);
17     }return r;
18 }
19 inline cert_route::Solution solve(const cert_route::Problem&p){
20     auto s=cert_route::solve(p);int n=p.start.size();if(s.exact)return s;
21     std::mt19937 rng(120926+n);
22     for(int iteration=0;iteration<10;++iteration){
23         auto r=s.path;std::vector<int>missing;int k=3+iteration%4;
24         for(int t=0;t<k;++t){int j=(iteration%2==0)?int(rng())%r.size():int((iteration+t)%r.size());missing.
25             push_back(r[j]);r.erase(r.begin()+j);}
26     }
27 }

```

```

23 r=repair(p,r,missing);
24 if(cert_route::length(p,r)<s.upper-1e-7||iteration%3==0)cert_route::improve(p,r,true);
25 double cost=cert_route::length(p,r);if(cost<s.upper-1e-7){s.path=r;s.upper=cost;}
26 }
27 s.local_optimal=cert_route::improve(p,s.path,true);s.upper=cert_route::length(p,s.path);return s;
28 }
29 }

```

src/neighborhood_tour.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include "bearing_types.hpp"
4 namespace neighborhood_tour {
5 inline bool valid(gео::Pt q,const гео::Poly&cell){for(auto z:cell)if(geo::dist(q,z)>20-1e-6)return false;
    return true;}
6 inline гео::Pt project(geo::Pt q,const гео::Poly&cell){for(int pass=0;pass<24;++pass){double shift=0;for(
    auto z:cell){double d=geo::dist(q,z);if(d>20-1e-5){auto next=z+(q-z)*((20-1e-5)/d);shift=std::max(
    shift,geo::dist(q,next));q=next;}}if(shift<1e-8)break;}return q;}
7 inline double length(const гео::Poly&r,geo::Pt start){double out=0;for(auto q:r){out+=geo::dist(start,q);
    start=q;}return out;}
8 inline гео::Poly optimize(geo::Poly route,const std::vector<geo::Poly>&cells,geo::Pt start){
9 if(route.size()!=cells.size())throw std::runtime_error("neighborhood size mismatch");
10 for(int pass=0;pass<5;++pass)for(size_t ii=0;ii<route.size();++ii){size_t i=pass%2?route.size()-1-ii:ii;if
    (cells[i].empty())continue;
11 auto prev=i?route[i-1]:start;auto next=i+1<route.size()?route[i+1]:prev;
12 auto cost=[&](geo::Pt q){return geo::dist(prev,q)+(i+1<route.size()?geo::dist(q,next):0);};
13 auto best=route[i];double value=cost(best);auto trial=[&](geo::Pt q){q=project(q,cells[i]);if(valid(q,
    cells[i])&&cost(q)<value-1e-8){best=q;value=cost(q);}};
14 auto d=next-prev;double fraction=geo::dot(d,d)>0?std::clamp(geo::dot(best-prev,d)/geo::dot(d,d),0.,1.):0;
15 trial(prev+d*fraction);trial((prev+next)/2);
16 for(int step=0;step<12;++step){auto a=best-prev,b=best-next;auto grad=a/std::max(1e-9,geo::norm(a));if(i
    +1<route.size())grad=grad+b/std::max(1e-9,geo::norm(b));trial(best-grad*(8./(1+step)));}
17 route[i]=best;
18 }
19 return route;
20 }
21 inline double expected(const гео::Poly&r,geo::Pt start,const std::vector<dynamic_selection::Hypothesis>&h){
    double mass=0,out=0;for(auto s:h){double t=0;auto q=start;bool found=false;for(auto p:r){t+=geo::dist(
    q,p)/5;q=p;if(geo::dist(p,s.z)<=20){t+=5;found=true;break;}t+=3;}if(!found)return INFINITY;mass+=s.
    weight;out+=s.weight*t;}return mass>0?out/mass:INFINITY;}
22 inline гео::Poly optical(const гео::Poly&base,const гео::Poly&centers,const std::vector<geo::Poly>&
    original_cells,geo::Pt start,const std::vector<dynamic_selection::Hypothesis>&h){
23 std::vector<geo::Poly>cells;std::vector<bool>used(centers.size());
24 for(auto q:base){bool mapped=false;for(size_t i=0;i<centers.size();++i)if(!used[i]&&geo::dist(q,centers[i]
    )<1e-7){used[i]=true;cells.push_back(original_cells[i]);mapped=true;break;}if(!mapped)return base;}
25 auto best=base;double bestlen=length(base,start),bestexp=expected(base,start,h);
26 auto consider=[&](geo::Poly candidate,const std::vector<geo::Poly>&regions){candidate=optimize(candidate,
    regions,start);double l=length(candidate,start),e=expected(candidate,start,h);if(l<bestlen-1e-6&&e<
    bestexp-1e-6){best=candidate;bestlen=l;bestexp=e;}};
27 consider(base,cells);
28 // Jointly reorder the neighborhoods, then resolve their visit positions.
29 auto tour=base;auto regions=cells;
30 for(int pass=0;pass<2;++pass){double gain=0;size_t ai=0,aj=0;for(size_t i=0;i<tour.size();++i)for(size_t j
    =i+1;j<tour.size();++j){auto a=i?tour[i-1]:start;double old=geo::dist(a,tour[i]),fresh=geo::dist(a,
    tour[j]);if(j+1<tour.size()){old+=geo::dist(tour[j],tour[j+1]);fresh+=geo::dist(tour[i],tour[j+1]);}if
    (old-fresh>gain){gain=old-fresh;ai=i;aj=j;}}if(gain<1e-7)break;std::reverse(tour.begin()+ai,tour.begin
    ()+aj+1);std::reverse(regions.begin()+ai,regions.begin()+aj+1);tour=optimize(tour,regions,start);
    consider(tour,regions);}
31 return best;
32 }
33 }

```

src/optical_order.hpp

```

1 #pragma once
2 #include "bearing_types.hpp"
3 #include <numeric>
4 #include <stdint>
5 #include <stdexcept>
6 #define B_SEARCH_ORDER 1
7 namespace optical_order {
8 inline geo::Poly choose(const geo::Poly&p,geo::Pt now,const std::vector<dynamic_selection::Hypothesis>&
    original,double mixture=1.){
9     if(p.empty())return p;
10    auto fallback=p;if(geo::dist(now,fallback.back())<geo::dist(now,fallback.front()))std::reverse(fallback.
        begin(),fallback.end());
11    if(original.empty())return fallback;
12    auto hypotheses=original;
13    if(mixture<1.){double mass=0;for(auto h:hypotheses)mass+=h.weight;for(auto &h:hypotheses)h.weight*=mixture
        /mass;for(auto q:p)hypotheses.push_back({q,1000,false,0,(1-mixture)/p.size()});}
14    double total=0;for(auto h:hypotheses)total+=h.weight;double best=1e100;auto chosen=fallback;
15    int n=p.size();std::vector<int>offsets{0,n-1};for(int i=1;i<=12;++i)offsets.push_back((i*n)/13);
16    for(int reverse=0;reverse<2;++reverse)for(int offset:offsets){geo::Poly route;for(int k=0;k<n;++k){int j=
        reverse?(offset-k+n)%n:(offset+k)%n;route.push_back(p[j]);}
17    std::vector<bool>done(hypotheses.size());int pending=hypotheses.size();double cost=0,expected=0;auto
        position=now;
18    for(auto q:route){cost+=geo::dist(position,q)/5;position=q;
19        for(int i=0;i<(int)hypotheses.size();++i)if(!done[i]&&geo::dist(q,hypotheses[i].z)<=20+1e-7){done[i]=
            true;--pending;expected+=hypotheses[i].weight/total*(cost+5);}
20        if(!pending)break;cost+=3;
21    }
22    if(pending)continue;if(expected<best){best=expected;chosen=route;}
23 }return chosen;
24 }
25 // Minimize expected time to first successful clear, retaining the entire cover.
26 // The cell-center mixture is a ranking regularizer, never a geometric proof.
27 inline geo::Poly search(const geo::Poly&p,geo::Pt now,const std::vector<dynamic_selection::Hypothesis>&
    original,double mixture=1.){
28     if(p.empty()||original.empty())return choose(p,now,original);
29     if(!(mixture>=0&&mixture<=1))throw std::invalid_argument("invalid search mixture");
30     auto h=original;double mass=0;for(auto x:h)mass+=x.weight;
31     for(auto &x:h)x.weight*=(1-mixture)/mass;
32     if(mixture>0)for(auto q:p)h.push_back({q,1000,false,0,mixture/p.size()});
33     const int n=p.size(),chunks=(h.size()+63)/64;
34     std::vector<std::vector<uint64_t>> hits(n,std::vector<uint64_t>(chunks));
35     std::vector<std::vector<double>> travel(n,std::vector<double>(n));std::vector<double>entry(n);
36     for(int i=0;i<n;++i){entry[i]=geo::dist(now,p[i])/5;for(int j=0;j<n;++j)travel[i][j]=geo::dist(p[i],p[j])
        /5;
37     for(size_t j=0;j<h.size();++j)if(geo::dist(p[i],h[j].z)<=20+1e-7)hits[i][j/64]|=uint64_t(1)<<(j%64);
38 }
39 std::vector<uint64_t>full(chunks,~uint64_t(0));if(h.size()%64)full.back()=(uint64_t(1)<<(h.size()%64))-1;
40 auto cost=[&](const std::vector<int>&order)->double{auto remaining=full;size_t pending=h.size();double
    elapsed=0,expected=0;int previous=-1;
41     for(int j:order){elapsed+=previous<0?entry[j]:travel[previous][j];previous=j;
42         for(int k=0;k<chunks;++k){uint64_t found=remaining[k]&hits[j][k];remaining[k]&=~found;
43             while(found){int bit=__builtin_ctzll(found);expected+=h[k*64+bit].weight*(elapsed+5);--pending;found&=
                found-1;}
44         }if(!pending)return expected;elapsed+=3;
45     }return INFINITY;
46 };
47 auto cyclic=choose(p,now,h);std::vector<int>best_order;std::vector<bool>mapped(n);
48 for(auto q:cyclic)for(int i=0;i<n;++i)if(!mapped[i]&&geo::dist(q,p[i])<1e-8){mapped[i]=true;best_order.
    push_back(i);break;}
49 if(best_order.size()!=p.size())throw std::runtime_error("search route permutation mismatch");
50 double best=cost(best_order);
51 for(double power:{.5,1.,1.5}){std::vector<int>order;std::vector<bool>used(n);auto remaining=full;int
    previous=-1;
52     for(int step=0;step<n;++step){int chosen=-1;double score=-1;

```

```

53   for(int j=0;j<n;++j)if(!used[j]){double probability=0;
54   for(int k=0;k<chunks;++k){uint64_t found=remaining[k]&hits[j][k];while(found){int bit=__builtin_ctzll(
    found);probability+=h[k*64+bit].weight;found&=found-1;}}
55   double seconds=3+(previous<0?entry[j]:travel[previous][j]);double value=(probability+1e-12)/std::pow(
    seconds,power);
56   if(value>score){score=value;chosen=j;}
57   }used[chosen]=true;order.push_back(chosen);previous=chosen;for(int k=0;k<chunks;++k)remaining[k]&=~hits[
    chosen][k];
58   }double value=cost(order);if(value<best){best=value;best_order=std::move(order);}
59   }
60   for(int pass=0;pass<2;++pass){auto chosen=best_order;double improved=best;
61   for(int i=0;i<n-1;++i)for(int j=i+1;j<n;++j){auto candidate=best_order;std::reverse(candidate.begin()+i,
    candidate.begin()+j+1);double value=cost(candidate);if(value<improved-1e-10){improved=value;chosen=std
    ::move(candidate);}}
62   if(improved>=best-1e-10)break;best=improved;best_order=std::move(chosen);
63   }
64   geo::Poly route;for(int i:best_order)route.push_back(p[i]);return route;
65   }
66   }

```

src/optical_portfolio.hpp

```

1 #pragma once
2 #include "optical_terminal.hpp"
3 #define B_OPTICAL_PORTFOLIO 1
4 namespace optical_portfolio {
5 inline geo::Poly choose(const geo::Poly& incumbent,const geo::Poly&candidate,geo::Pt now,const std::vector<
    dynamic_selection::Hypothesis>&h,double fraction,double seconds){
6   if(h.empty()||candidate.empty())return incumbent;
7   double before=optical_terminal::expected(incumbent,now,h),after=optical_terminal::expected(candidate,now,h
    );
8   return after<before*(1-fraction)&&before-after>seconds?candidate:incumbent;
9   }
10 }

```

src/optical_terminal.hpp

```

1 #pragma once
2 #include "adaptive_optical.hpp"
3 #include "optical_order.hpp"
4 #include <map>
5 #include <memory>
6 #define B_PREPARED_SWEEP 1
7 namespace optical_terminal {
8 // Geometric branch regions remain separate from finite ranking hypotheses.
9 // Prediction bins currently group noiseless bearings, not noisy report draws.
10 // Their angular envelope bounds geometry; branch probabilities remain approximate.
11 inline geo::Poly update(geo::Poly region,geo::Pt q,int outcome){
12   if(outcome<10)return region;
13   double angle=2.*(outcome-10)+1.;
14   region=geo::observe(region,q,angle,2.005);
15   region=geo::disk_clip(region,q,1500);
16   auto u=geo::polar(1,angle);
17   return geo::clip(region,u,geo::dot(u,q)+1500);
18   }
19 inline double expected(const geo::Poly& route,geo::Pt from,const std::vector<dynamic_selection::Hypothesis
    >& hypotheses){
20   double total=0,cost=0,value=0;for(auto h:hypotheses)total+=h.weight;
21   if(hypotheses.empty())return 0;
22   std::vector<bool> done(hypotheses.size());size_t left=hypotheses.size();
23   for(auto q:route){cost+=geo::dist(from,q)/5;from=q;
24   for(size_t i=0;i<hypotheses.size();++i)if(!done[i]&&geo::dist(q,hypotheses[i].z)<=20+1e-7){done[i]=true
    ;--left;value+=hypotheses[i].weight/total*(cost+5);}
25   if(!left)return value;cost+=3;

```

```

26 }
27 throw std::runtime_error("terminal route does not cover conditional hypotheses");
28 }
29 class PreparedSweep {
30     geo::Poly points;
31     std::vector<uint64_t> hits;
32     std::vector<std::vector<double>>> travel;
33     std::vector<double> weights;
34 public:
35     PreparedSweep(const geo::Poly& p, const std::vector<dynamic_selection::Hypothesis>& h): points(p){
36         if(p.empty() || h.empty() || h.size() > 63) throw std::invalid_argument("invalid prepared sweep");
37         for(auto x: h) weights.push_back(x.weight);
38         hits.assign(p.size(), 0); travel.assign(p.size(), std::vector<double>(p.size()));
39         for(size_t i=0; i<p.size(); ++i){ for(size_t j=0; j<h.size(); ++j) if(geo::dist(p[i], h[j].z) <= 20+1e-7) hits[i] |=
40             uint64_t(1) << j;
41         for(size_t j=0; j<p.size(); ++j) travel[i][j] = geo::dist(p[i], p[j])/5;
42     }
43     double value(uint64_t mask, geo::Pt now, bool match_entry=true) const{
44         if(!mask) return 0;
45         if(mask >> weights.size()) throw std::invalid_argument("invalid sweep mask");
46         double total=0; for(size_t i=0; i<weights.size(); ++i) if(mask >> i & 1) total += weights[i];
47         int n=points.size(); bool flip=match_entry && geo::dist(now, points.back()) < geo::dist(now, points.front());
48         std::vector<int> offsets(0, n-1); for(int i=1; i<=12; ++i) offsets.push_back(i*n/13);
49         std::vector<double> entry; for(auto p: points) entry.push_back(geo::dist(now, p)/5);
50         double best=INFINITY;
51         for(int reverse=0; reverse<2; ++reverse) for(int offset: offsets){
52             uint64_t remaining=mask; double cost=0, value=0; int previous=-1;
53             for(int k=0; k<n; ++k){ int logical=reverse?(offset-k+n)%n: (offset+k)%n; int j=flip?n-1-logical: logical;
54                 cost += previous < 0 ? entry[j]: travel[previous][j]; previous=j;
55                 uint64_t found=remaining & hits[j]; remaining &= ~found;
56                 while(found){ int i=__builtin_ctzll(found); value += weights[i]/total*(cost+5); found &= found-1; }
57                 if(!remaining) break; cost += 3;
58             }
59             if(!remaining && value < best) best=value;
60         }
61         if(!std::isfinite(best)) throw std::runtime_error("terminal route does not cover conditional hypotheses");
62         return best;
63     }
64 };
65 class Contexts {
66     struct State { geo::Poly region; std::unique_ptr<PreparedSweep> sweep; std::map<std::pair<int, int>, int>
67         children; };
68     std::vector<State> states;
69     const std::vector<geo::Pt> & locations;
70     const std::vector<bool> & sensing;
71     const std::vector<dynamic_selection::Hypothesis> & hypotheses;
72     geo::Pt first; double angle; bool match_entry;
73 public:
74     Contexts(const geo::Poly& p, const std::vector<geo::Pt> & q, const std::vector<bool> & m, const std::vector<
75         dynamic_selection::Hypothesis> & h, geo::Pt f, double a, bool match=false): locations(q), sensing(m),
76         hypotheses(h), first(f), angle(a), match_entry(match){ states.push_back({p, {}, {}}); }
77     int advance(int context, int action, int outcome){
78         // Negative readings and failed clears do not yet have a continuous set update.
79         if(!sensing.at(action) || outcome < 10) return context;
80         auto key=std::make_pair(action, outcome); auto it=states.at(context).children.find(key);
81         if(it!=states[context].children.end()) return it->second;
82         auto region=update(states[context].region, locations.at(action), outcome);
83         if(region.empty()) throw std::runtime_error("empty predicted continuous region");
84         int id=states.size(); states[context].children[key]=id; states.push_back({std::move(region), {}, {}}); return
85             id;
86     }
87     double value(uint64_t mask, geo::Pt now, int context){
88         auto& state=states.at(context);

```

```

85  if(!state.sweep)state.sweep=std::make_unique<PreparedSweep>(adaptive_optical::cover(state.region,first,
    angle),hypotheses);
86  return state.sweep->value(mask,now,match_entry);
87  }
88  };
89  }

```

src/policy.hpp

```

1  #pragma once
2  #include "geometry.hpp"
3  #include "routing.hpp"
4  #include "certified_route.hpp"
5  #include "position_uncertainty.hpp"
6  #include "lns.hpp"
7  #include "active.hpp"
8  #include "dynamic_selection.hpp"
9  #include "optical_order.hpp"
10 #include "discovery.hpp"
11 #include "residual_cover.hpp"
12 #include "neighborhood_tour.hpp"
13 #include "adaptive_optical.hpp"
14 #include "optical_portfolio.hpp"
15 #include "../review/certified_layout21_hex.hpp"
16 #include <array>
17 #include <stdexcept>
18 using geo::Pt;using geo::Poly;
19 struct Reading {int kind=0;double angle=0;}; // no_signal=0, near=1, direction=2
20 struct Sensor {virtual Reading measure(Pt,int)=0;virtual bool clear(Pt,int)=0;virtual void audit(int,const
    Poly&){};virtual void phase(int){};virtual ~Sensor()=default;};
21 #include "rollout_sensor.hpp"
22 struct Track {Poly p;Pt first;double angle=0;int obs=0;bool cleared=false;std::vector<dynamic_selection::
    Event>history;int dynamic_used=0,aux_used=0;};
23 struct Options {double spacing=950,early=35;bool square=false,dynamic=true;int routing=0;double shiftA=0,
    shiftB=0;bool shifted=false,includeOrigin=true,opportunistic=false;int fixedLayout=0;double secondA
    =750,secondB=300,eta=0;bool stop_when_found16=false;int route_model=0;bool use_lns=false;int
    active_mode=0;int dynamic_depth=0,dp_node_limit=6000;bool joint_actions=false;int auxiliary=0,
    aux_limit=4;double start_residual=0;bool retreat=false,candidate_menu=false,complete_source=false,
    direct_optical=false,optical_rotate=false;int planning_samples=32;double q3_radius=1200;bool
    clear_neighborhood=false,defer_single=false,weighted_optical=false;double optical_weight=1,
    immediate_fee_weight=0,discovery_weight=0;int dynamic_limit=6;bool adaptive_optical=false;int
    terminal_mode=0;bool variable_optical=false;int optical_search=0;bool budget_aware=false;int
    refinement=0;int bearing_bin=2;int optical_portfolio=0;bool shortcut=false;int robust_mode=0;bool
    residual_cover=false,neighborhood_tour=false,residual_opportunistic=false,neighborhood_variable=false
    };
24 class Policy {
25  Sensor& io;int problem;Options opt;std::array<Track,21> t;bool auxiliary_busy=false;discovery::Belief
    unseen;residual_cover::State remaining_cover;
26  public:Pt pos;int channel=1,cleared=0,fallbacks=0,plans=0,exact_plans=0,local_plans=0,nonlocal_plans=0,
    covered_skips=0;double max_route_gap=0,total_route_gap=0,total_route_upper=0,max_relative_gap=0;std::
    string certificate="incomplete";
27  int dynamic_plans=0,dynamic_actions=0,dynamic_fallbacks=0,dp_expanded=0,dp_budget_hits=0;
28  int auxiliary_calls=0,auxiliary_positive=0,joint_steps=0,residual_calls=0,residual_replaced=0,
    residual_retired=0,rollout_calls=0,rollout_replaced=0;
29  Policy(Sensor& s,int prob,Options o={}) :io(s),problem(prob),opt(o){if(opt.discovery_weight>0)unseen=
    discovery::Belief(prob);}
30  Reading measure(Pt p,int k){auto r=io.measure(p,k);pos=p;channel=k;
31  if(opt.dynamic_depth>0)t[k].history.push_back({p,r.kind,r.angle});
32  if(r.kind==1){if(!clear(p,k))throw std::runtime_error("near followed by failed clear");}
33  else if(r.kind==2){auto &a=t[k];if(a.obs==0){a.p=geo::outer_disk({0,0},1800);a.first=p;a.angle=r.angle;}
34  if(opt.eta>0)a.p=position_error::update(a.p,p,r.angle,opt.eta);
35  else {a.p=geo::observe(a.p,p,r.angle);a.p=geo::disk_clip(a.p,p,1500);}
36  // Valid axial bound additionally prevents circumscribed-disk overshoot along bearing.
37  Pt u=geo::polar(1,r.angle);a.p=geo::clip(a.p,u,geo::dot(u,p)+1500+opt.eta);
38  ++a.obs;if(a.p.empty())throw std::runtime_error("empty feasible region");io.audit(k,a.p);}

```

```

39  if(opt.auxiliary>=3&&!auxiliary_busy)supplement(p,k);
40  return r;
41  }
42  bool clear(Pt p,int k){bool ok=io.clear(p,k);pos=p;if(opt.dynamic_depth>0)t[k].history.push_back({p,ok
    ?4:3,0});if(ok&&!t[k].cleared){t[k].cleared=true;++cleared;}if(opt.auxiliary>=3&&!auxiliary_busy)
    supplement(p,k);return ok;}
43  void optical(int k){io.phase(2);++fallbacks;auto &a=t[k];Pt u=geo::polar(1,a.angle),v={-u.y,u.x};
44  double xmin=1e99,xmax=-1e99,ymin=1e99,ymax=-1e99;
45  for(auto p:a.p){Pt d=p-a.first;double x=geo::dot(d,u),y=geo::dot(d,v);xmin=std::min(xmin,x);xmax=std::max
    (xmax,x);ymin=std::min(ymin,y);ymax=std::max(ymax,y);}
46  double step=opt.eta>0?position_error::grid_side(opt.eta):28;
47  int nx=std::max(1,(int)ceil((xmax-xmin)/step)),ny=std::max(1,(int)ceil((ymax-ymin)/step));
48  Poly route;for(int i=0;i<nx;++i)for(int jj=0;jj<ny;++jj){int j=i%2?ny-1-jj:jj;route.push_back(a.first+u*(
    xmin+(i+.5)*(xmax-xmin)/nx)+v*(ymin+(j+.5)*(ymax-ymin)/ny));}
49  if(opt.adaptive_optical&&opt.eta==0){auto candidate=adaptive_optical::cover(a.p,a.first,a.angle);if(!
    candidate.empty())route=candidate;}
50  if(opt.variable_optical&&opt.eta==0)route=adaptive_optical::variable_cover(a.p,a.first,a.angle);
51  if(geo::dist(pos,route.back())<geo::dist(pos,route.front()))std::reverse(route.begin(),route.end());
52  if(opt.optical_rotate){size_t best=0;for(size_t i=1;i<route.size();++i)if(geo::dist(pos,route[i])<geo::
    dist(pos,route[best]))best=i;std::rotate(route.begin(),route.begin()+best,route.end());}
53  if(opt.weighted_optical&&opt.eta==0)route=optical_order::choose(route,pos,dynamic_selection::
    make_hypotheses(a.p,a.history,problem,63),opt.optical_weight);
54  if(opt.optical_search&&opt.eta==0)route=optical_order::search(route,pos,dynamic_selection::
    make_hypotheses(a.p,a.history,problem,63),opt.optical_search==1?0:opt.optical_search==2?.1:.3);
55  if(opt.optical_portfolio&&opt.eta==0){
56  auto h=dynamic_selection::make_hypotheses(a.p,a.history,problem,63);
57  auto candidate=adaptive_optical::variable_cover(a.p,a.first,a.angle);
58  if(geo::dist(pos,candidate.back())<geo::dist(pos,candidate.front()))std::reverse(candidate.begin(),
    candidate.end());
59  candidate=optical_order::choose(candidate,pos,h);
60  if(opt.optical_portfolio==2)candidate=optical_order::search(candidate,pos,h,.1);
61  double fraction=opt.optical_portfolio==1?.05:opt.optical_portfolio==2?.1:opt.optical_portfolio==3?.2:0;
62  route=optical_order::choose(route,candidate,pos,h,fraction,100*fraction);
63  }
64  if(opt.neighborhood_tour&&opt.eta==0){std::vector<Poly>cells;auto centers=adaptive_optical::cover(a.p,a.
    first,a.angle,&cells);route=neighborhood_tour::optical(route,centers,cells,pos,dynamic_selection::
    make_hypotheses(a.p,a.history,problem,63));}
65  if(opt.neighborhood_variable&&opt.eta==0){std::vector<Poly>cells;auto centers=adaptive_optical::
    variable_cover(a.p,a.first,a.angle,&cells);auto h=dynamic_selection::make_hypotheses(a.p,a.history,
    problem,63);auto order=optical_order::choose(centers,pos,h);auto candidate=neighborhood_tour::optical(
    order,centers,cells,pos,h);
66  double oldcost=neighborhood_tour::expected(route,pos,h),newcost=neighborhood_tour::expected(candidate,
    pos,h);
67  if(newcost+5<oldcost&&newcost<.95*oldcost&&neighborhood_tour::length(candidate,pos)<neighborhood_tour::
    length(route,pos))route=candidate;
68  }
69  for(auto p:route)if(clear(p,k))return;
70  throw std::runtime_error("continuous optical coverage exhausted without success");
71  }
72  dynamic_selection::Choice rollout_choice(int k,dynamic_selection::Choice base){
73  auto &a=t[k];if(!base.available||geo::mec(a.p).r<=35)return base;
74  std::vector<dynamic_selection::Alternative>candidate{{base.point,base.measure,base.value}};
75  auto fresh=robust_geometry::candidate(a.p,pos,a.first,a.angle,a.history,problem,1);
76  if(fresh.valid){double oldrho=base.measure&&robust_geometry::reception(a.p,base.point,a.history,problem)?
    robust_geometry::worst_radius(a.p,base.point,128).upper:geo::mec(a.p).r;
77  double extra=std::max(0.,geo::dist(pos,fresh.q)-geo::dist(pos,base.point))/5+5;
78  if(.4*(oldrho-fresh.rho)>extra&&fresh.rho<geo::mec(a.p).r*.85)candidate.push_back({fresh.q,true,0});
79  }
80  if(candidate.size()==1)return base;
81  auto h=dynamic_selection::make_hypotheses(a.p,a.history,problem,4);
82  // Include polygon boundary extrema, not just interior particles, for Q3.
83  if(problem==3){auto far=a.p.front();for(auto z:a.p)if(geo::dist(z,a.first)>geo::dist(far,a.first))far=z;
84  for(auto z:{a.p.front(),far}){dynamic_selection::Hypothesis x{z,1500,false,0,1};if(dynamic_selection::
    consistent(x,a.history))h.push_back(x);}
85  if(h.empty())return base;

```

```

86  std::vector<std::vector<double>>>costs(candidate.size());
87  for(size_t j=0;j<candidate.size();++j)for(auto source:h)for(double error:{-1.,1.}){
88      RolloutSensor sensor(source,pos,channel,error,a.history);Options continuation=opt;continuation.
        robust_mode=0;continuation.residual_cover=false;continuation.neighborhood_tour=false;continuation.
        auxiliary=0;
89      Policy shadow(sensor,problem,continuation);shadow.pos=pos;shadow.channel=channel;shadow.t[k]=a;
90      auto d=candidate[j];++shadow.t[k].dynamic_used;
91      try{if(d.measure)shadow.measure(d.point,k);else shadow.clear(d.point,k);if(!shadow.t[k].cleared)shadow.
        locate(k);if(!sensor.cleared)throw std::runtime_error("rollout incomplete");costs[j].push_back(sensor.
        time);}
92      catch(const std::exception&){costs[j].push_back(INFINITY);}++rollout_calls;
93  }
94  auto mean=[](const std::vector<double>&v){double s=0;for(auto x:v)s+=x;return s/v.size();};
95  double best=mean(costs[0]);int selected=0;
96  for(size_t j=1;j<candidate.size();++j){double value=mean(costs[j]);double worst_regret=-INFINITY;for(
        size_t i=0;i<costs[j].size();++i)worst_regret=std::max(worst_regret,costs[j][i]-costs[0][i]);
97      if(value+5<best&&worst_regret<=5){best=value;selected=j;}}
98  if(selected){++rollout_replaced;base.point=candidate[selected].point;base.measure=candidate[selected].
        measure;base.value=best;base.alternatives={{base.point,base.measure,best}};}
99  return base;
100 }
101 void locate(int k){io.phase(1);auto &a=t[k];if(a.cleared)return;
102 if(opt.dynamic_depth>0&&opt.eta==0){
103     for(int step=a.dynamic_used;step<opt.dynamic_limit&&!a.cleared;++step){
104         int depth=opt.budget_aware?dynamic_selection::remaining_depth(opt.dynamic_depth,opt.dynamic_limit,a.
            dynamic_used):opt.dynamic_depth;
105         auto decision=dynamic_selection::plan(a.p,pos,a.first,a.angle,a.history,problem,depth,channel==k,opt.
            dp_node_limit,opt.retreat,opt.planning_samples,opt.clear_neighborhood,opt.terminal_mode,opt.refinement
            ,opt.bearing_bin,opt.robust_mode==3?0:opt.robust_mode);
106         if(opt.robust_mode==3)decision=rollout_choice(k,decision);
107         ++dynamic_plans;dp_expanded+=decision.expanded;dp_budget_hits+=decision.budget_hit;
108         if(!decision.available)break;
109         ++dynamic_actions;++a.dynamic_used;if(decision.measure)measure(decision.point,k);else clear(decision.
            point,k);
110     }
111     if(a.cleared)return;++dynamic_fallbacks;
112 }
113 if(opt.direct_optical)optical(k);else locate_baseline(k);
114 }
115 void locate_baseline(int k){auto &a=t[k];if(a.cleared)return;
116 if(opt.opportunistic&&a.obs==1){Pt u=geo::polar(1,a.angle);if(std::abs(geo::cross(u,pos-a.first))>=50){
        measure(pos,k);if(a.cleared)return;}}
117 if(a.obs==1){Pt u=geo::polar(1,a.angle),v={-u.y,u.x};Pt q1=a.first+u*opt.secondA+v*opt.secondB,q2=a.first
        +u*opt.secondA-v*opt.secondB;
118 if(geo::dist(pos,q2)<geo::dist(pos,q1))std::swap(q1,q2);
119 if(opt.active_mode)q1=active::choose(a.p,pos,a.first,a.angle,opt.secondA,opt.secondB,opt.active_mode);
120 auto r=measure(q1,k);if(a.cleared)return;
121 if(r.kind==0){measure(q2,k);if(a.cleared)return;}
122 }
123 for(int attempt=0;attempt<3&&!a.cleared;++attempt){auto c=geo::mec(a.p);
124 if(c.r<=opt.early || c.r<=20-opt.eta-1e-6){if(clear(c.c,k))return;}
125 if(a.obs<2)break;
126 Pt q=c.c;
127 if(geo::dist(pos,q)<1){Pt u=geo::polar(1,a.angle);q=q+Pt{-u.y,u.x}*(attempt%2?-50:50);}
128 auto r=measure(q,k);if(a.cleared)return;
129 if(r.kind==0){Pt u=geo::polar(1,a.angle),v={-u.y,u.x};measure(c.c+v*50,k);if(a.cleared)return;}
130 }
131 if(!a.cleared)optical(k);
132 }
133 Poly covering_points(){
134 if(problem==3&&opt.q3_radius!=1200){Poly p{{0,0}};for(int i=0;i<6;++i)p.push_back(geo::polar(opt.
        q3_radius,60*i));return p;}
135 if(problem==4&&opt.fixedLayout==21){Poly p;for(auto x:certified_layout21::points)p.push_back({x.x,x.y});
        return p;}
136 if(problem==3||!opt.shifted)return geo::coverage(problem,opt.square?600:opt.spacing,opt.square);

```

```

137 Poly points;if(opt.includeOrigin)points.push_back({0,0});double h=opt.spacing;auto v=[&](int i,int j){
    return Pt{h*(i+opt.shiftA+.5*(j+opt.shiftB)),h*sqrt(3.)/2*(j+opt.shiftB)};};
138 for(int i=-6;i<6;++i)for(int j=-6;j<6;++j){Pt a=v(i,j),b=v(i+1,j),c=v(i,j+1),d=v(i+1,j+1);
139     for(auto triangle:{Poly{a,b,c},Poly{d,c,b}})if(geo::meets_disk(triangle,1800))for(auto q:triangle){bool
        duplicate=false;for(auto p:points)if(geo::dist(p,q)<1e-6)duplicate=true;if(!duplicate)points.push_back
            (q);}}
140 return points;
141 }
142 void supplement(Pt q,int exclude=0){
143     if(opt.auxiliary==0||opt.eta!=0||auxiliary_busy)return;
144     auxiliary_busy=true;
145     int start=channel;
146     for(int j=0;j<20;++j){int k=1+(start-1+j)%20;auto &a=t[k];
147         if(k==exclude||a.cleared||a.obs==0||a.aux_used>opt.aux_limit)continue;
148         auto c=geo::mec(a.p);if(c.r<=20||geo::dist(q,c.c)>1000)continue;
149         auto u=geo::polar(1,a.angle);if(std::abs(geo::cross(u,q-a.first))<50)continue;
150         bool repeated=false;for(auto e:a.history)if(e.kind<=2&&geo::dist(e.q,q)<1e-6)repeated=true;if(repeated)
            continue;
151         if(opt.auxiliary>=4){auto sample=dynamic_selection::make_hypotheses(a.p,a.history,problem,32);double
            total=0,heard=0;for(auto h:sample){total+=h.weight;if(dynamic_selection::receives(h,q))heard+=h.weight
                ;}if(total==0||heard<.6*total)continue;}
152         ++a.aux_used;++auxiliary_calls;auto r=measure(q,k);if(r.kind>0)++auxiliary_positive;
153     }
154     auxiliary_busy=false;
155 }
156 void scan(Pt q){io.phase(0);int start=channel;for(int a=0;a<20;++a){int k=1+(start-1+a)%20;if(!t[k].
    cleared&&t[k].obs==0)measure(q,k);}supplement(q);if(opt.discovery_weight>0)unseen.negative_scan(q);}
157 int known()const{int n=0;for(int k=1;k<=20;++k)n+=(t[k].cleared||t[k].obs>0);return n;}
158 double scan_fee()const{int n=0;for(int k=1;k<=20;++k)if(!t[k].cleared&&t[k].obs==0)++n;return n*6*n-(t[
    channel].cleared&&t[channel].obs==0?1:0):0;}
159 double service_distance(Pt from,Pt center,int kind){
160     if(opt.route_model!=2||kind<0||t[kind].obs!=1)return geo::dist(from,center);
161     auto &tr=t[kind];auto u=geo::polar(1,tr.angle);Pt v={-u.y,u.x};auto a=tr.first+u*opt.secondA+v*opt.
        secondB,b=tr.first+u*opt.secondA-v*opt.secondB;
162     // Entry side follows the production nearest-side rule. The exit remains
163     // a belief-center approximation, not a secretly observed source position.
164     Pt entry=geo::dist(from,a)<=geo::dist(from,b)?a:b;
165     return geo::dist(from,entry)+geo::dist(entry,center);
166 }
167 void joint_run(const Poly&initial_points){Poly points=initial_points;std::vector<bool>visited(points.size
    ());
168 while(cleared<16){
169     if(opt.residual_opportunistic&&opt.eta==0){bool scanned_before=false;for(auto q:remaining_cover.past)if(
        geo::dist(q,pos)<1)scanned_before=true;
170     if(!scanned_before)for(int i=0;i<(int)points.size();++i)if(!visited[i]&&geo::dist(pos,points[i])>1){
171         if(remaining_cover.verify(remaining_cover.sites(points,visited,i,pos,true),true)){visited[i]=true;scan
            (pos);remaining_cover.scanned(pos);++remaining_cover.retired;remaining_cover.retire(points,visited);
            residual_retired=remaining_cover.retired;break;}
172     }
173 }
174 Poly nodes;std::vector<int> kind;std::vector<dynamic_selection::Choice>choices;
175 bool complete_discovery=opt.stop_when_found16&&known()==16;
176 for(int i=0;i<(int)points.size();++i)if(!visited[i]){if(complete_discovery){visited[i]=true;++
    covered_skips;}else{nodes.push_back(points[i]);kind.push_back(-1-i);choices.push_back({});}}
177 bool coverage_left=!nodes.empty();
178 for(int k=1;k<=20;++k)if(t[k].obs&&!t[k].cleared){
179     if(opt.defer_single&&coverage_left&&t[k].obs==1)continue;
180     dynamic_selection::Choice decision;auto &a=t[k];
181     if(opt.joint_actions&&opt.dynamic_depth>0&&opt.eta==0&&a.dynamic_used<opt.dynamic_limit){
182         int depth=opt.budget_aware?dynamic_selection::remaining_depth(opt.dynamic_depth,opt.dynamic_limit,a.
            dynamic_used):opt.dynamic_depth;
183         decision=dynamic_selection::plan(a.p,pos,a.first,a.angle,a.history,problem,depth,channel==k,opt.
            dp_node_limit,opt.retreat,opt.planning_samples,opt.clear_neighborhood,opt.terminal_mode,opt.refinement
            ,opt.bearing_bin,opt.robust_mode==3?0:opt.robust_mode);
184         if(opt.robust_mode==3)decision=rollout_choice(k,decision);

```

```

185     ++dynamic_plans; dp_expanded+=decision.expanded; dp_budget_hits+=decision.budget_hit;
186 }
187 nodes.push_back(decision.available&&!opt.candidate_menu?decision.point:geo::mec(a.p).c); kind.push_back(
    k); choices.push_back(decision);
188 }
189 if(nodes.empty())break; std::vector<int>order;
190 if(opt.route_model){cert_route::Problem task; int n=nodes.size(); task.start.resize(n); task.edge.assign(n,
    std::vector<double>(n));
191 for(int j=0; j<n; ++j){task.start[j]=opt.joint_actions?geo::dist(pos,nodes[j]):service_distance(pos,nodes
    [j],kind[j]); for(int i=0; i<n; ++i)if(i!=j)task.edge[i][j]=opt.joint_actions?geo::dist(nodes[i],nodes[j]
    ):service_distance(nodes[i],nodes[j],kind[j]);}
192 if(opt.candidate_menu){auto cost=[&](Pt from,int j){if(!choices[j].available)return geo::dist(from,
    nodes[j]); double best=1e100; for(auto d:choices[j].alternatives)best=std::min(best,geo::dist(from,d.
    point)+std::max(0.,5*d.value-geo::dist(pos,d.point))); return best;}; for(int j=0; j<n; ++j){task.start[j]
    =cost(pos,j); for(int i=0; i<n; ++i)if(i!=j)task.edge[i][j]=cost(nodes[i],j);}}
193 if(opt.joint_actions&&opt.start_residual>0)for(int j=0; j<n; ++j)if(kind[j]>0&&choices[j].available)task.
    start[j]+=opt.start_residual*std::max(0.,choices[j].value*5-geo::dist(pos,nodes[j]));
194 if(opt.immediate_fee_weight>0)for(int j=0; j<n; ++j){double fee=kind[j]<0?scan_fee():choices[j].
    available&&choices[j].measure?5+(channel!=kind[j]):4; task.start[j]+=5*opt.immediate_fee_weight*fee;}
195 if(opt.discovery_weight>0){int left=0; for(int k:kind)left+=k<0; double missing=std::max(0,std::min(16-
    known(),std::max(1,13-known()))); for(int j=0; j<n; ++j)if(kind[j]<0){double reward=opt.discovery_weight*
    missing*unseen.probability(nodes[j])*2.5*left; task.start[j]=std::max(0.,task.start[j]-5*reward);}}
196 auto solution=opt.use_lns?new_route::solve(task):cert_route::solve(task); order=solution.path; ++plans; if
    (solution.exact)++exact_plans; if(solution.local_optimal)++local_plans; else ++nonlocal_plans;
197 double gap=std::max(0.,solution.upper-solution.lower); total_route_gap+=gap; max_route_gap=std::max(
    max_route_gap,gap); total_route_upper+=solution.upper; max_relative_gap=std::max(max_relative_gap,
    solution.upper>0?gap/solution.upper:0.);
198 }else order=route::optimize(pos,nodes);
199 if(opt.neighborhood_tour&&opt.eta==0){Poly path; std::vector<Poly>cells; bool any=false;
200 for(int j:order){path.push_back(nodes[j]); if(kind[j]>0&&geo::mec(t[kind[j]].p).r<20-1e-4){cells.
    push_back(t[kind[j]].p); path.back()=geo::mec(cells.back()).c; any=true;} else cells.push_back({});}
201 if(any){auto moved=neighborhood_tour::optimize(path,cells,pos); for(size_t jj=0; jj<order.size(); ++jj)if
    (!cells[jj].empty()){int j=order[jj]; nodes[j]=moved[jj]; choices[j].available=true; choices[j].measure=
    false; choices[j].point=moved[jj];}}
202 }
203 int chosen=order[0], act=kind[chosen]; if(opt.joint_actions)++joint_steps;
204 if(act<0){int i=-1-act;
205 if(opt.residual_cover&&opt.eta==0){points[i]=remaining_cover.replace(points,visited,i,pos);}
206 visited[i]=true; scan(points[i]);
207 if(opt.residual_cover&&opt.eta==0){remaining_cover.scanned(points[i]); remaining_cover.retire(points,
    visited); residual_calls=remaining_cover.calls; residual_replaced=remaining_cover.accepted;
    residual_retired=remaining_cover.retired;}
208 }
209 else{
210 if(opt.joint_actions){io.phase(1); auto d=choices[chosen]; if(d.available&&t[act].dynamic_used<opt.
    dynamic_limit){++t[act].dynamic_used; ++dynamic_actions; if(d.measure)measure(d.point,act); else clear(d.
    point,act); if(opt.complete_source&&!t[act].cleared)locate(act);} else{++dynamic_fallbacks; if(opt.
    direct_optical)optical(act); else locate_baseline(act);}}
211 else locate(act);
212 if(opt.auxiliary>=2)supplement(pos,act);
213 }
214 }
215 }
216 void run(){Poly points=covering_points(); std::vector<bool> visited(points.size());
217 if(opt.routing==2){joint_run(points); if(cleared==16){certificate="upper_bound_16"; return;} certificate=
    problem=="3?"seven_point_per_channel:"convex_mesh_per_channel"; return;}
218 for(size_t step=0; step<points.size(); ++step){size_t j=points.size(); double best=1e99;
219 if(opt.routing==1){Poly nodes; std::vector<int>ids; for(int i=0; i<(int)points.size(); ++i)if(!visited[i]){
    nodes.push_back(points[i]); ids.push_back(i);} j=ids[route::optimize(pos,nodes)[0]];
220 else for(size_t i=0; i<points.size(); ++i)if(!visited[i]){double score=opt.dynamic?geo::dist(pos,points[i]
    ):double(i); if(score<best){best=score; j=i;}}
221 visited[j]=true; scan(points[j]);
222 while(cleared<16){int kbest=0; best=1e99; Poly nodes; std::vector<int>ids; for(int k=1; k<=20; ++k)if(t[k].obs
    >0&&!t[k].cleared){auto center=geo::mec(t[k].p).c; nodes.push_back(center); ids.push_back(k); double d=
    opt.dynamic?geo::dist(pos,center):k; if(d<best){best=d; kbest=k;}} if(!kbest)break; if(opt.routing==1)

```

```

        kbest=ids[route::optimize(pos,nodes)[0]];locate(kbest);}
223   if(cleared==16){certificate="upper_bound_16";return;}
224   }
225   for(int k=1;k<=20;++k)if(t[k].obs&&!t[k].cleared)throw std::runtime_error("pending source at exit");
226   certificate=problem=="seven_point_per_channel":"convex_mesh_per_channel";
227   }
228 };

```

src/position_uncertainty.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 namespace position_error {
4 inline geo::Poly update(geo::Poly p,geo::Pt s,double angle,double eta){
5   double eps=1.005*geo::pi/180,t=tan(eps);auto u=geo::polar(1,angle);geo::Pt v={-u.y,u.x};
6   for(auto n:{v-u*t,v*(-1)-u*t})p=geo::clip(p,n,geo::dot(n,s)+eta*geo::norm(n));
7   p=geo::disk_clip(p,s,1500+eta);p=geo::clip(p,u,geo::dot(u,s)+1500+eta);p=geo::clip(p,u*(-1),-geo::dot(u,s)+eta);
8   for(auto n:{v,v*(-1)})p=geo::clip(p,n,geo::dot(n,s)+1500*sin(eps)+eta);
9   return p;
10 }
11 inline double grid_side(double eta){return std::min(28.,sqrt(2.)*(20-eta)-1e-6);}
12 }

```

src/residual_cover.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include <array>
4 namespace residual_cover {
5 // Outward-rounded long-double intervals certify every accepted mutation.
6 // Input binary64 site coordinates are represented exactly in long double.
7 struct I{long double lo,hi;I(long double a=0):lo(a),hi(a){}I(long double a,long double b):lo(a),hi(b){}};
8 inline long double down(long double x){return std::nextafter(x,-INFINITY);}
9 inline long double up(long double x){return std::nextafter(x,INFINITY);}
10 inline I operator+(I a,I b){return {down(a.lo+b.lo),up(a.hi+b.hi)};}
11 inline I operator-(I a,I b){return {down(a.lo-b.hi),up(a.hi-b.lo)};}
12 inline I operator*(I a,I b){std::array<long double,4> v{a.lo*b.lo,a.lo*b.hi,a.hi*b.lo,a.hi*b.hi};return {
    down(*std::min_element(v.begin(),v.end())),up(*std::max_element(v.begin(),v.end()))};}
13 inline I det(geo::Pt a,geo::Pt b,geo::Pt c){return (I(b.x)-I(a.x))*(I(c.y)-I(a.y))-(I(b.y)-I(a.y))*(I(c.x)-I(a.x));}
14 inline I distance2(geo::Pt a,geo::Pt b){I x=I(a.x)-I(b.x),y=I(a.y)-I(b.y);return x*x+y*y;}
15 struct Box{double x0,y0,x1,y1;int depth=0;geo::Poly corners()const{return {{x0,y0},{x1,y0},{x1,y1},{x0,y1}};}};
16 inline bool outside(Box b){double x=b.x0>0?b.x0:b.x1<0?b.x1:0,y=b.y0>0?b.y0:b.y1<0?b.y1:0;return distance2({x,y},{0,0}).lo>1800.L*1800.L;}
17 inline bool safe_box(Box b,const geo::Poly&sites){
18   auto vertices=b.corners();geo::Poly safe;
19   for(auto q:sites){bool in=true;for(auto z:vertices)if(distance2(q,z).hi>999.999L*999.999L){in=false;break};if(in)safe.push_back(q);}
20   auto h=geo::hull(safe);if(h.size()<3)return false;
21   for(size_t i=0;i<h.size();++i){if(det(h[i],h[(i+1)%h.size()],h[(i+2)%h.size()]).lo<=0)return false;
22     for(auto z:vertices)if(det(h[i],h[(i+1)%h.size()],z).lo<0)return false;}
23   return true;
24 }
25 struct Certificate{bool pass=false;int checks=0;std::vector<Box>leaves;};
26 inline Certificate certify(const geo::Poly&sites,const std::vector<Box>&prior={},int budget=100000){
27   Certificate out;std::vector<Box>todo=prior.empty()?std::vector<Box>{{{-1800,-1800,1800,1800,0}}}:prior;
28   while(!todo.empty()){
29     auto b=todo.back();todo.pop_back();if(outside(b))continue;
30     if(++out.checks>budget)return out;
31     if(safe_box(b,sites)){out.leaves.push_back(b);continue;}
32     if(b.depth>=30)return out;
33     // An uncovered center is a quick rejection, never an acceptance test.

```

```

34 geo::Pt c{(b.x0+b.x1)/2,(b.y0+b.y1)/2};
35 if(geo::norm(c)<1800){geo::Poly near;for(auto p:sites)if(geo::dist(c,p)<=1000)near.push_back(p);if(!geo::
    contains(geo::hull(near),c))return out;}
36 if(b.x1-b.x0>=b.y1-b.y0){double m=(b.x0+b.x1)/2;todo.push_back({b.x0,b.y0,m,b.y1,b.depth+1});todo.
    push_back({m,b.y0,b.x1,b.y1,b.depth+1});}
37 else{double m=(b.y0+b.y1)/2;todo.push_back({b.x0,b.y0,b.x1,m,b.depth+1});todo.push_back({b.x0,m,b.x1,b.y1
    ,b.depth+1});}
38 }
39 out.pass=true;return out;
40 }
41 class State {
42 public:geo::Poly past;std::vector<Box>leaves;int calls=0,accepted=0,retired=0;double saved=0;
43 geo::Poly sites(const geo::Poly&points,const std::vector<bool>&visited,int omit=-1,geo::Pt replacement={},
    bool add=false)const{
44 geo::Poly out=past;for(int i=0;i<(int)points.size();++i)if(!visited[i]&&i!=omit)out.push_back(points[i]);
    if(add)out.push_back(replacement);return out;
45 }
46 bool verify(const geo::Poly&s,bool commit){++calls;auto c=certify(s,leaves);if(c.pass&&commit)leaves=std::
    move(c.leaves);return c.pass;}
47 void scanned(geo::Pt q){past.push_back(q);}
48 void retire(geo::Poly&points,std::vector<bool>&visited){
49 // With no extra/replaced site, deleting an essential static point cannot
50 // become sound merely because other static points gave negative readings.
51 for(int i=0;i<(int)points.size();++i)if(!visited[i]&&verify(sites(points,visited,i),true)){visited[i]=
    true;++retired;}
52 }
53 geo::Pt replace(const geo::Poly&points,const std::vector<bool>&visited,int i,geo::Pt now){
54 auto old=points[i];if(geo::dist(old,now)<5)return old;
55 auto valid=[&](geo::Pt q,bool commit){return verify(sites(points,visited,i,q,true),commit);};
56 if(valid(now,true)){++accepted;saved+=geo::dist(old,now);return now;}
57 // Closest certified point on this segment; feasibility is checked at every
58 // accepted point. Bisection is a heuristic search, not a monotonicity claim.
59 double low=0,high=1;geo::Pt best=old;
60 for(int it=0;it<7;++it){double m=(low+high)/2;auto q=old+(now-old)*m;if(valid(q,false)){low=m;best=q;}
    else high=m;}
61 if(geo::dist(best,old)>=2&&valid(best,true)){++accepted;saved+=geo::dist(best,old);return best;}return
    old;
62 }
63 };
64 }

```

src/robust_geometry.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include "bearing_types.hpp"
4 #include <array>
5 #include <queue>
6 namespace robust_geometry {
7 constexpr double eps=1.005; // ±1 degree plus the public 0.01-degree readout rounding.
8 struct Bound {double lower=0,upper=0;int cells=0;};
9 inline bool reception(const geo::Poly&p,geo::Pt q,const std::vector<dynamic_selection::Event>&h,int problem
    ,double range=950.7){
10 if(p.empty())return false;
11 for(auto z:p)if(geo::dist(q,z)>range-1e-5)return false;
12 if(problem==3)return true;
13 // For every z, the positive observations constrain heading by homogeneous
14 // halfplanes. q in the convex hull of heard sites is a sufficient dual cone
15 // certificate simultaneously for all z. No heading particles certify this.
16 geo::Poly heard;for(auto e:h)if(e.kind==1||e.kind==2)heard.push_back(e.q);
17 return geo::contains(geo::hull(heard),q);
18 }
19 inline std::pair<double,double> angles(const geo::Poly&p,geo::Pt q){
20 if(geo::contains(p,q))return {0,360};

```

```

21 std::vector<double>a;for(auto z:p){double b=atan2(z.y-q.y,z.x-q.x)*180/geo::pi;if(b<0)b+=360;a.push_back(b);}
22 std::sort(a.begin(),a.end());double gap=-1,start=0;
23 for(size_t i=0;i<a.size();++i){double b=i+1<a.size()?a[i+1]:a[0]+360;if(b-a[i]>gap){gap=b-a[i];start=b;}}
24 return {start-eps,start+360-gap+eps};
25 }
26 inline Bound worst_radius(const geo::Poly&p,geo::Pt q,int budget=96,double tolerance=.10){
27 if(p.empty())return {};auto old=geo::mec(p);
28 auto range=angles(p,q);
29 struct Cell{double a,b,upper,lower;bool operator<(const Cell&o)const{return upper<o.upper;}};
30 auto cell=[&](double a,double b){double mid=(a+b)/2;
31 auto outer=geo::observe(p,q,mid,eps+(b-a)/2+1e-9);
32 auto exact=geo::observe(p,q,mid,eps);
33 double up=outer.empty()?0:std::min(old.r,geo::mec(outer).r+1e-6);
34 double lo=exact.empty()?0:std::max(0.,geo::mec(exact).r);
35 return Cell{a,b,up,lo};
36 };
37 std::priority_queue<Cell>todo;int initial=std::max(1,int(ceil((range.second-range.first)/12.)));double
lower=0;int used=0;
38 for(int i=0;i<initial;++i){auto x=cell(range.first+(range.second-range.first)*i/initial,range.first+(range
.second-range.first)*(i+1)/initial);lower=std::max(lower,x.lower);todo.push(x);++used;}
39 while(used+2<=budget&&!todo.empty()&&todo.top().upper>lower+tolerance){auto x=todo.top();todo.pop();double
mid=(x.a+x.b)/2;for(auto y:{cell(x.a,mid),cell(mid,x.b)}){lower=std::max(lower,y.lower);todo.push(y);
++used;}}
40 return {lower,todo.empty()?0:std::max(lower,todo.top().upper)+1e-5,used};
41 }
42 inline geo::Pt project_reception(geo::Pt q,const geo::Poly&p,double range=950.69){
43 for(int pass=0;pass<24;++pass){double maxmove=0;for(auto z:p){double d=geo::dist(q,z);if(d>range){auto n=z
+(q-z)*(range/d);maxmove=std::max(maxmove,geo::dist(q,n));q=n;}}if(maxmove<1e-6)break;}return q;
44 }
45 // Deterministic simplex optimization of the geometric objective. It neither
46 // quantizes bearings nor performs eight-direction particle pattern search.
47 template<class Score> geo::Pt simplex(geo::Pt seed,double scale,Score score,int steps=26){
48 struct V{geo::Pt q;double s;};auto at=[&](geo::Pt q){return V{q,score(q)};};
49 std::array<V,3>x{at(seed),at(seed+geo::Pt{scale,0}),at(seed+geo::Pt{0,scale})};
50 for(int it=0;it<steps;++it){std::sort(x.begin(),x.end(),[](auto a,auto b){return a.s<b.s;});auto c=(x[0].q
+x[1].q)/2;auto r=at(c*2-x[2].q);
51 if(r.s<x[0].s){auto e=at(c+(r.q-c)*2);x[2]=e.s<r.s?e:r;}
52 else if(r.s<x[1].s)x[2]=r;
53 else{auto s=at(c+(r.s<x[2].s?r.q-c:x[2].q-c)*.5);if(s.s<std::min(r.s,x[2].s))x[2]=s;else for(int j=1;j
<3;++j)x[j]=at((x[0].q+x[j].q)/2);}
54 }
55 return std::min_element(x.begin(),x.end(),[](auto a,auto b){return a.s<b.s;})->q;
56 }
57 struct Candidate{bool valid=false;geo::Pt q;double rho=INFINITY,cost=INFINITY;};
58 inline Candidate candidate(const geo::Poly&p,geo::Pt now,geo::Pt first,double angle,const std::vector<
dynamic_selection::Event>&history,int problem,int mode){
59 Candidate best;if(p.empty()||problem==4)return best; // one positive bearing has no new universal heading
guarantee.
60 auto c=geo::mec(p);if(c.r<=20)return best;
61 auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};
62 geo::Poly seeds{project_reception(now,p),project_reception(c.c,p)};
63 for(double side:{-1.,1.})seeds.push_back(project_reception(first+u*750+v*(side*300),p));
64 auto valid=[&](geo::Pt q){if(geo::norm(q)>2671.99||!reception(p,q,history,problem))return false;for(auto e
:history)if(e.kind<=2&&geo::dist(e.q,q)<.5)return false;return true;};
65 auto score=[&](geo::Pt q){if(!valid(q))return 1e8+geo::dist(q,project_reception(q,p))*1000;double r=
worst_radius(p,q,64,.25).upper;
66 return mode==2?geo::dist(now,q)+500*std::max(0.,r-34.8):r;};
67 std::sort(seeds.begin(),seeds.end(),[&](auto a,auto b){return score(a)<score(b);});
68 for(int i=0;i<2;++i){auto q=simplex(seeds[i],75,score,22);if(!valid(q))continue;double r=worst_radius(p,q
,256,.03).upper;
69 if(mode==2&&r>35)continue;double cost=mode==2?geo::dist(now,q):r;
70 if(cost<best.cost)best={true,q,r,cost};}
71 return best;
72 }

```

src/rollout_sensor.hpp

```

1 #pragma once
2 // Included after Sensor/Reading declarations in policy.hpp.
3 struct RolloutSensor:Sensor {
4   dynamic_selection::Hypothesis source;Pt pos;int channel;double error,time=0;bool cleared=false;
5   std::vector<dynamic_selection::Event>events;
6   RolloutSensor(dynamic_selection::Hypothesis h,Pt p,int k,double e,const std::vector<dynamic_selection::
7     Event>&history):source(h),pos(p),channel(k),error(e),events(history){}
8   void move(Pt p){time+=geo::dist(pos,p)/5;pos=p;}
9   Reading measure(Pt p,int k)override{move(p);time+=5+(channel!=k);channel=k;if(cleared)return {};}
10  for(auto e:events)if(e.kind<=2&&geo::dist(p,e.q)<1e-7)return {e.kind,e.angle};
11  Reading r;if(dynamic_selection::receives(source,p)){if(geo::dist(source.z,p)<=5)r.kind=1;else{r.kind=2;r.
12    angle=fmod(atan2(source.z.y-p.y,source.z.x-p.x)*180/geo::pi,error+720,360);}}
13  events.push_back({p,r.kind,r.angle});return r;
14 }
15 bool clear(Pt p,int)override{move(p);bool ok=!cleared&&geo::dist(p,source.z)<=20;time+=ok?5:3;if(ok)
16   cleared=true;return ok;}
17 };

```

src/routing.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include <numeric>
4 namespace route {
5   inline double length(geo::Pt start,const geo::Poly&p,const std::vector<int>&r){double d=0;for(int i:r){d+=
6     geo::dist(start,p[i]);start=p[i];}return d;}
7   inline std::vector<int> nearest(geo::Pt start,const geo::Poly&p,int first=-1){
8     std::vector<int> r;std::vector<bool>used(p.size());
9     if(first>=0){r.push_back(first);used[first]=true;start=p[first];}
10    while(r.size()<p.size()){int best=-1;double d=1e100;for(int i=0;i<(int)p.size();++i)if(!used[i]&&geo::dist
11      (start,p[i])<d){d=geo::dist(start,p[i]);best=i;}r.push_back(best);used[best]=true;start=p[best];}
12    return r;
13 }
14 inline void two_opt(geo::Pt start,const geo::Poly&p,std::vector<int>&r){
15   for(int iter=0;iter<100;++iter){double best=-1e-7;int bi=-1,bj=-1;
16   for(int i=0;i<(int)r.size()-1;++i)for(int j=i+1;j<(int)r.size();++j){auto a=i?p[r[i-1]]:start;double
17     delta=geo::dist(a,p[r[j]])-geo::dist(a,p[r[i]]);
18     if(j+1<(int)r.size())delta+=geo::dist(p[r[i]],p[r[j+1]])-geo::dist(p[r[j]],p[r[j+1]]);
19     if(delta<best){best=delta;bi=i,bj=j;}}
20   if(bi<0)break;std::reverse(r.begin()+bi,r.begin()+bj+1);
21 }
22 inline std::vector<int> optimize(geo::Pt start,const geo::Poly&p){auto best=nearest(start,p);two_opt(start,
23   p,best);double cost=length(start,p,best);
24 // Deterministic starts: all possible first nodes, capped only by small problem size.
25 for(int first=0;first<(int)p.size();++first){auto r=nearest(start,p,first);two_opt(start,p,r);double c=
26   length(start,p,r);if(c<cost-1e-7){cost=c;best=r;}}
27 return best;
28 }
29 }

```

src/shortcut_sensor.hpp

```

1 #pragma once
2 #include "policy.hpp"
3 #define B_SHORTCUT_SENSOR 1
4 // Execution adapter: the controller retains its original route anchors, while
5 // physical clear commands may stop earlier on the incoming segment. Measurements
6 // are never moved. With nominal conservative support, clear outcomes are unchanged.

```

```

7 // Construct only for a fresh session starting at (0,0), never attach mid-session.
8 class ShortcutSensor:public Sensor {
9   Sensor& io;bool enabled;std::array<geo::Poly,21> support;
10  geo::Pt physical,anchor;double physical_length=0,anchor_length=0;
11  bool fits(const geo::Poly& p,geo::Pt q)const{
12   if(p.empty())return false;for(auto z:p)if(geo::dist(z,q)>20-1e-4)return false;return true;
13  }
14  void record(geo::Pt requested,geo::Pt executed){
15   anchor_length+=geo::dist(anchor,requested);physical_length+=geo::dist(physical,executed);anchor=requested;
    physical=executed;
16   if(physical_length+geo::dist(physical,anchor)>anchor_length+1e-5)throw std::runtime_error("shortcut
    violated anchored path dominance");
17  }
18  public:int shortcuts=0;
19  explicit ShortcutSensor(Sensor& sensor,bool active=true):io(sensor),enabled(active){}
20  Reading measure(geo::Pt q,int k)override{auto r=io.measure(q,k);record(q,q);return r;}
21  bool clear(geo::Pt q,int k)override{
22   geo::Pt target=q;
23   if(enabled&&k>=1&&k<=20&&fits(support[k],q)){
24    if(fits(support[k],physical))target=physical;
25    else{double low=0,high=1;for(int i=0;i<60;++i){double mid=(low+high)/2;auto trial=physical+(q-physical)*
      mid;if(fits(support[k],trial))high=mid;else low=mid;}target=physical+(q-physical)*high;}
26    // A fresh endpoint check handles arithmetic roundoff without assuming success.
27    if(!fits(support[k],target))target=q;
28   }
29   bool result=io.clear(target,k);record(q,target);
30   if(geo::dist(q,target)>1e-7){++shortcuts;if(!result)throw std::runtime_error("guaranteed shortcut clear
    failed");}
31   if(result&&k>=1&&k<=20)support[k].clear();return result;
32  }
33  void audit(int k,const geo::Poly&p)override{if(k>=1&&k<=20)support[k]=p;io.audit(k,p);}
34  void phase(int stage)override{io.phase(stage);}
35  double saved_distance()const{return anchor_length-physical_length;}
36 };

```

src/simulator.hpp

```

1 #pragma once
2 #include "policy.hpp"
3 #include <stdint>
4 #include <fstream>
5 #include <iomanip>
6 #include <random>
7 struct Jammer {int ch;Pt p;double radius;bool directional;double heading;bool removed=false;};
8 inline uint64_t mix(uint64_t z){z+=0x9e3779b97f4a7c15ULL;z=(z^(z>>30))*0xbf58476d1ce4e5b9ULL;z=(z^(z>>27))
  *0x94d049bb133111ebULL;return z^(z>>31);}
9 inline std::vector<Jammer> make_case(int problem,int seed,int stress=0){
10  std::mt19937_64 rng(seed);auto U=[&]() {return std::generate_canonical<double,53>(rng);};
11  int n=10+rng()%7;std::vector<int>ch;for(int k=1;k<=20;++k)ch.push_back(k);std::shuffle(ch.begin(),ch.end()
  ,rng);
12  int nd=1+rng()%(n-1);std::vector<Jammer> j;
13  for(int k=0;k<n;++k){Pt p;do{p=geo::polar(1800*sqrt(U()),360*U());}while(std::any_of(j.begin(),j.end()
  ,[&](Jammer a){return geo::dist(a.p,p)<40;}));
14   double r=1000+500*U(),heading=360*U();bool dir=problem==4&&k<nd;
15   if(stress==1){r=1000;p=geo::polar(1800,(k+.137)*360/n);heading=atan2(p.y,p.x)*180/geo::pi;dir=problem
    ==4&&k<n-1;}
16   if(stress==2){r=1000;p={-1700.+3400.*k/(n-1),double(k%2)*.01};heading=k%2?90:270;dir=problem==4&&k<n-1;}
17   if(stress==3){r=1000;p={double(k%5)*5,0};heading=k%2?0:180;dir=problem==4&&k<n-1;}
18   j.push_back({ch[k],p,r,dir,heading});}return j;
19 }
20 class Sim:public Sensor {
21  std::vector<Jammer> truth;int seed,errmode;std::ofstream trace;
22  public:Pt pos;int channel=1,measures=0,miss=0,success=0,switches=0,audits=0;double time=0,length=0,
    max_violation=0;

```

```

23 Sim(std::vector<Jammer> j,int s,int e,const std::string&file=""):truth(j),seed(s),errmode(e){if(!file.
    empty()){trace.open(file);trace<<"kind,channel,x,y,result,angle,time\n";}}
24 void move(Pt q){double d=geo::dist(q,pos);length+=d;time=round((time+d/5)*1e6)/1e6;pos=q;}
25 Jammer* pending(int k){for(auto &j:truth)if(j.ch==k&&!j.removed)return &j;return nullptr;}
26 void check(){if(time>=360000)throw std::runtime_error("virtual deadline");}
27 Reading measure(Pt p,int k)override {check();move(p);++measures;if(k!=channel){++switches;time+=1;}channel
    =k;time+=5;Reading r;
28 auto j=pending(k);if(j&&geo::dist(p,j->p)<=j->radius&&(!j->directional||geo::dot(p-j->p,geo::polar(1,j->
    heading))>=-1e-9)){
29     if(geo::dist(p,j->p)<=5)r.kind=1;else{r.kind=2;double a=atan2(j->p.y-p.y,j->p.x-p.x)*180/geo::pi;
30     uint64_t v=mix((uint64_t)llround(p.x*100))~mix((uint64_t)llround(p.y*100)+1234567)^mix(seed);
31     double e=2*(mix(v)>>11)*0x1.0p-53-1;
32     if(errmode==1)e=1;if(errmode==2)e=-1;if(errmode==3)e=sin(p.x/350)*cos(p.y/350);if(errmode==4)e=(p.x
        >=0?1:-1);
33     r.angle=fmod(round(fmod(a+e+720,360)*100)/100,360);}}
34 if(trace)trace<<std::setprecision(12)<<"measure,"<<k<<","<<p.x<<","<<p.y<<","<<r.kind<<","<<r.angle
    <<","<<time<<"\n";return r;
35 }
36 bool clear(Pt p,int k)override{check();move(p);auto j=pending(k);bool ok=j&&geo::dist(p,j->p)<=20;time+=ok
    ?5:3;if(ok){j->removed=true;++success;}else ++miss;
37 if(trace)trace<<std::setprecision(12)<<"clear,"<<k<<","<<p.x<<","<<p.y<<","<<ok<<","<<time<<"\n";return
    ok;}
38 void audit(int k,const Poly&p)override{++audits;auto j=pending(k);if(j&&!geo::contains(p,j->p))throw std::
    runtime_error("truth excluded from conservative polygon");}
39 int total()const{return(int)truth.size();}
40 };

```

review/certified_layout21_hex.hpp

```

1 // Exactly the binary64 coordinates independently certified in cover21_995_binary64.json.
2 #pragma once
3 #include <array>
4 namespace certified_layout21 {
5 struct Point { double x,y; };
6 inline constexpr std::array<Point,21> points = {{
7     {0x0.0p+0, 0x0.0p+0},
8     {0x1.f180000000000p+9, 0x0.0p+0},
9     {0x1.5fc91ea18215dp+9, 0x1.5fc91ea18215dp+9},
10    {0x0.0p+0, 0x1.f180000000000p+9},
11    {-0x1.5fc91ea18215dp+9, 0x1.5fc91ea18215dp+9},
12    {-0x1.f180000000000p+9, 0x0.0p+0},
13    {-0x1.5fc91ea18215dp+9, -0x1.5fc91ea18215dp+9},
14    {0x0.0p+0, -0x1.f180000000000p+9},
15    {0x1.5fc91ea18215dp+9, -0x1.5fc91ea18215dp+9},
16    {0x1.d200000000000p+10, 0x0.0p+0},
17    {0x1.93915dd785dc9p+10, 0x1.d200000000000p+9},
18    {0x1.d200000000000p+9, 0x1.93915dd785dc9p+10},
19    {0x0.0p+0, 0x1.d200000000000p+10},
20    {-0x1.d200000000000p+9, 0x1.93915dd785dc9p+10},
21    {-0x1.93915dd785dc9p+10, 0x1.d200000000000p+9},
22    {-0x1.d200000000000p+10, 0x0.0p+0},
23    {-0x1.93915dd785dc9p+10, -0x1.d1fffffffffff7p+9},
24    {-0x1.d200000000000p+9, -0x1.93915dd785dc4p+10},
25    {0x0.0p+0, -0x1.d200000000000p+10},
26    {0x1.d200000000000p+9, -0x1.93915dd785dc9p+10},
27    {0x1.93915dd785dc4p+10, -0x1.d200000000000p+9},
28 }};
29 }

```

src/robot.cpp

```

1 #include "policy.hpp"
2 #include "configurations.hpp"
3 #include "joint_config.hpp"

```

```

4 #include "shortcut_sensor.hpp"
5 #include <iostream>
6 #include <iomanip>
7 // Solver receives observations only; no simulator headers or case data are linked.
8 struct PipeSensor:Sensor {
9     Reading measure(Pt p,int k)override{std::cout<<std::setprecision(17)<<"measure "<<k<<" "<<p.x<<" "<<p.y<<
        std::endl;Reading r;if(!(std::cin>>r.kind>>r.angle))throw std::runtime_error("bridge ended");return r
        ;}
10     bool clear(Pt p,int k)override{std::cout<<std::setprecision(17)<<"clear "<<k<<" "<<p.x<<" "<<p.y<<std::
        endl;int ok;if(!(std::cin>>ok))throw std::runtime_error("bridge ended");return ok==1;}
11 };
12 int main(int argc,char**argv){try{
13     int prob=argc>1?atoi(argv[1]):3;if(prob!=3&&prob!=4)throw std::runtime_error("problem must be 3 or 4");
14     std::string strategy=argc>5?argv[5]:"joint";if(strategy!="geometric"&&strategy!="joint"&&strategy!="
        previous"&&strategy!="baseline"&&strategy!="stage1"&&strategy!="stage2")throw std::runtime_error("
        strategy must be geometric, joint, previous, stage1, stage2 or baseline");
15     Options o=joint_configuration(strategy=="geometric"?84:strategy=="joint"?78:strategy=="previous"?56:
        strategy=="stage2"?39:strategy=="stage1"?38:0,prob);if(argc>2)o.early=atof(argv[2]);if(argc>3)o.eta=
        atof(argv[3]);
16     int depth=argc>4?std::stoi(argv[4]):2;if(depth<0||depth>3)throw std::runtime_error("planner depth must be
        in [0,3]");
17     if(!(o.eta>=0&&o.eta<=.4))throw std::runtime_error("position error must be in [0,0.4]");
18     if(o.eta==0){if(depth==0){double early=o.early;o=joint_configuration(0,prob);o.early=early;}o.
        dynamic_depth=depth;}
19     else{double early=o.early,eta=o.eta;o=configuration(5);o.early=early;o.eta=eta;}
20     PipeSensor pipe;ShortcutSensor s(pipe,o.shortcut&&o.eta==0);Policy bot(s,prob,o);bot.run();
21     std::cerr<<"strategy="<<strategy<<" planner_depth="<<o.dynamic_depth<<" dp_calls="<<bot.dynamic_plans<<"
        dp_actions="<<bot.dynamic_actions<<" dp_fallbacks="<<bot.dynamic_fallbacks<<" budget_hits="<<bot.
        dp_budget_hits<<" shortcuts="<<s.shortcuts<<" saved_m="<<s.saved_distance()<<"\n";
22     std::cout<<"exit "<<bot.cleared<<" "<<bot.certificate<<std::endl;
23 }catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}

```

src/benchmark.cpp

```

1 #include "simulator.hpp"
2 #include "configurations.hpp"
3 #include "joint_config.hpp"
4 #include "shortcut_sensor.hpp"
5 #include <chrono>
6 #include <iostream>
7 Options benchmark_options(int method,int problem){return joint_configuration(method,problem);}
8 int main(int argc,char**argv){
9     if(argc<4){std::cerr<<"benchmark OUTPUT START COUNT [STRESS] [LAST_METHOD]\n";return 2;}
10     int start=atoi(argv[2]),count=atoi(argv[3]),stress=argc>4?atoi(argv[4]):0,last=argc>5?atoi(argv[5]):6;
11     int first=argc>6?atoi(argv[6]):0;
12     if(count<1||start<1||first<0||last<first||last>86||stress<0||stress>5)return 2;
13     std::ofstream f(argv[1]);if(!f)return 2;
14     f<<"problem,seed,method,stress,n,cleared,time_s,avg_s,move_m,measures,switches,miss,fallbacks,dp_calls,
        dp_actions,dp_fallbacks,dp_expanded,dp_budget_hits,aux_calls,aux_positive,joint_steps,runtime_s,
        certificate,shortcuts,shortcut_saved_m,residual_calls,residual_replaced,residual_retired,rollout_calls
        ,rollout_replaced\n";
15     for(int prob:{3,4})for(int seed=start;seed<start+count;++seed){auto environment=make_case(prob,seed,stress
        );
16     for(int method=first;method<=last;++method){auto options=benchmark_options(method,prob);Sim sim(
        environment,seed,stress==2?2:stress==4?3:stress==5?4:0);ShortcutSensor sensor(sim,options.shortcut&&
        options.eta==0);Policy bot(sensor,prob,options);
17     auto begin=std::chrono::steady_clock::now();
18     try{bot.run();if(sim.success!=sim.total())throw std::runtime_error("incomplete");if(bot.dynamic_actions>
        options.dynamic_limit*sim.total())throw std::runtime_error("dynamic action limit");
19     double cost=sim.length/5+sim.switches+5*sim.measures+3*sim.miss+5*sim.success;if(std::abs(sim.time-cost
        )>.001)throw std::runtime_error("accounting");}
20     catch(const std::exception&e){std::cerr<<"FAIL problem="<<prob<<" seed="<<seed<<" method="<<method<<"
        "<<e.what()<<std::endl;return 1;}
21     double seconds=std::chrono::duration<double>(std::chrono::steady_clock::now()-begin).count();

```

```

22     f<<std::setprecision(13)<<prob<<"<<seed<<"<<method<<"<<stress<<"<<sim.total()<<"<<sim.success
    <<"<<sim.time<<"<<sim.time/sim.total()<<"<<sim.length<<"<<sim.measures<<"<<sim.switches
    <<"<<sim.miss<<"<<bot.fallbacks<<"<<bot.dynamic_plans<<"<<bot.dynamic_actions<<"<<bot.
    dynamic_fallbacks<<"<<bot.dp_expanded<<"<<bot.dp_budget_hits<<"<<bot.auxiliary_calls<<"<<bot.
    auxiliary_positive<<"<<bot.joint_steps<<"<<seconds<<"<<bot.certificate<<"<<sensor.shortcuts
    <<"<<sensor.saved_distance()<<"<<bot.residual_calls<<"<<bot.residual_replaced<<"<<bot.
    residual_retired<<"<<bot.rollout_calls<<"<<bot.rollout_replaced<<"\n";
23 }
24 if((seed-start+1)%10==0){f.flush();std::cout<<"p="<<prob<<" cases="<<seed-start+1<<std::endl;}
25 }
26 }

```

src/bridge.py

```

1  """Serial HTTP transport only. Model and decisions reside in robot.exe."""
2  import argparse
3  import json
4  import queue
5  import subprocess
6  import sys
7  import threading
8  import time
9  from pathlib import Path
10
11  ROOT = Path(__file__).resolve().parents[1]
12  sys.path.insert(0, str(ROOT / 'vendor'))
13  from mocksim.client import RobotClient, Rejected, TransportFailure, ProtocolError
14
15
16  def _until(fn, deadline_at):
17      """Bound the whole operation, including a stalled HTTP body or child pipe.
18
19      A timed-out worker may still be receiving its old response. The caller must
20      abort the run without sending any new action (including /exit) in that case.
21      """
22      remaining = deadline_at - time.perf_counter()
23      if remaining <= 0:
24          raise TimeoutError('real deadline: incomplete run')
25      result = queue.Queue(maxsize=1)
26      def work():
27          try:
28              result.put((True, fn()))
29          except Exception as exc:
30              result.put((False, exc))
31      threading.Thread(target=work, daemon=True).start()
32      try:
33          ok, value = result.get(timeout=max(0, deadline_at-time.perf_counter()))
34      except queue.Empty:
35          raise TimeoutError('real deadline during operation: incomplete run') from None
36      if not ok:
37          raise value
38      return value
39
40
41  class DeadlineClient(RobotClient):
42      """Reuse original serialization/logging, but cap every attempt and retry.
43
44      New IDs are still generated by RobotClient action methods. All retries call
45      its post() with the identical payload. Only one request is in flight.
46      """
47      deadline_at = float('inf')
48
49      def __init__(self, **kwargs):
50          super().__init__(**kwargs)
51          self.retry_limit = self.retries

```

```

52     self.retries = 0 # Remain zero even if an old worker outlives its deadline.
53     self.aborted = False
54
55     def _position_action(self, kind, x, y, channel):
56         # Preserve the exact binary64 coordinate received from robot stdout.
57         # Original RobotClient._num rounds to 1e-6 m, which can invalidate a
58         # closed angular boundary in a strict set-membership calculation.
59         payload = self._base(kind)
60         payload.update(position={'x':float(x), 'y':float(y)}, channel=int(channel))
61         return self.post('/'+kind, payload)
62
63     def measure(self, x, y, channel):
64         return self._position_action('measure', x, y, channel)
65
66     def clear(self, x, y, channel):
67         return self._position_action('clear', x, y, channel)
68
69     def post(self, path, payload):
70         if self.aborted:
71             raise TimeoutError('run already aborted with an uncertain request')
72         timeout, retry_limit = self.timeout_s, self.retry_limit
73         try:
74             for attempt in range(retry_limit + 1):
75                 remaining = self.deadline_at-time.perf_counter()
76                 if remaining <= 0:
77                     raise TimeoutError('real deadline before request: incomplete run')
78                 self.timeout_s = min(timeout, remaining)
79                 try:
80                     if self.deadline_at == float('inf'):
81                         response = super().post(path, payload)
82                     else:
83                         response = _until(lambda: super(DeadlineClient, self).post(path, payload), self.
deadline_at)
84                     status = self.log[-1]['http_status']
85                     if status != 200:
86                         raise ProtocolError(status, response)
87                     return response
88                 except TransportFailure:
89                     if attempt == retry_limit:
90                         raise
91                     delay = self.retry_backoff_s*(attempt+1)
92                     remaining = self.deadline_at-time.perf_counter()
93                     if remaining <= delay:
94                         raise TimeoutError('real deadline before retry: incomplete run') from None
95                     self.stats.retries += 1
96                     time.sleep(delay)
97             except TimeoutError:
98                 self.aborted = True
99                 raise
100         finally:
101             self.timeout_s = timeout
102
103
104 def drive(problem, client, early=35, deadline=1200, position_error=0.0, planner_depth=2, strategy='joint'):
105     if strategy not in ('geometric', 'joint', 'previous', 'stage1', 'stage2', 'baseline'):
106         raise ValueError('strategy must be geometric, joint, previous, stage1, stage2 or baseline')
107     if type(planner_depth) is not int or not 0 <= planner_depth <= 3:
108         raise ValueError('planner depth must be an integer in [0,3]')
109     if not 0 <= position_error <= .4:
110         raise ValueError('position error must be in [0, 0.4] metres')
111     start = time.perf_counter()
112     hard_stop = start + deadline
113     def call(fn, cutoff):
114         if isinstance(client, DeadlineClient):
115             client.deadline_at = cutoff

```

```

116         return fn()
117     return _until(fn, cutoff)
118     enter = call(client.enter, hard_stop)
119     if enter.get('accepted') is not True:
120         raise Rejected(enter)
121     # Start-based accounting is conservative about time spent receiving /enter.
122     hard_stop = start + min(deadline, enter['remaining_real_duration_s'])
123     remaining = hard_stop-time.perf_counter()
124     if remaining <= 0:
125         raise TimeoutError('no real duration remains after enter: incomplete run')
126     action_stop = hard_stop-min(2.0, remaining*.1)
127     proc = subprocess.Popen([str(ROOT / 'robot.exe'), str(problem), str(early), str(position_error), str(
128         planner_depth), strategy],
129                             stdin=subprocess.PIPE, stdout=subprocess.PIPE,
130                             stderr=subprocess.PIPE, text=True, bufsize=1)
131     virtual, final = 0, None
132     lines = queue.Queue()
133     errors = []
134     def read_output():
135         try:
136             for line in proc.stdout:
137                 lines.put(line)
138         finally:
139             lines.put(None)
140     def read_errors():
141         for line in proc.stderr:
142             errors.append(line)
143     reader = threading.Thread(target=read_output, daemon=True)
144     err_reader = threading.Thread(target=read_errors, daemon=True)
145     reader.start(); err_reader.start()
146     try:
147         while True:
148             # Safe /exit only between completed actions, never after an uncertain
149             # timed-out request, which might still be registered on the server.
150             remaining = action_stop-time.perf_counter()
151             if remaining <= 0:
152                 final = call(client.exit, hard_stop)
153                 raise TimeoutError('real deadline guard: incomplete run')
154             try:
155                 line = lines.get(timeout=remaining)
156             except queue.Empty:
157                 final = call(client.exit, hard_stop)
158                 raise TimeoutError('solver output deadline: incomplete run') from None
159             if line is None:
160                 raise RuntimeError('solver exited without certificate: ' + ''.join(errors))
161             if virtual >= 359900:
162                 final = call(client.exit, hard_stop)
163                 raise TimeoutError('virtual deadline guard: incomplete run')
164             fields = line.split()
165             if fields[0] == 'exit':
166                 final = call(client.exit, hard_stop)
167                 if final.get('accepted') is not True:
168                     raise Rejected(final)
169                 return dict(cleared=int(fields[1]), certificate=fields[2],
170                             planner_depth=planner_depth if position_error == 0 else 0,
171                             strategy=strategy if position_error==0 and planner_depth>0 else 'legacy',
172                             virtual_time_s=final['virtual_time_s'],
173                             runtime_s=time.perf_counter()-start, response=final)
174             action, channel, x, y = fields
175             if action not in ('measure', 'clear'):
176                 raise ValueError('unknown solver action')
177             response = call(lambda: getattr(client, action)(float(x), float(y), int(channel)), action_stop)
178             if response.get('accepted') is not True:
179                 raise Rejected(response)
180             virtual = response['virtual_time_s']

```

```

180         if action == 'measure':
181             kind = {'no_signal': 0, 'near': 1, 'direction': 2}[response['measure_result']]
182             reply = f"{kind} {response.get('svd_deg', 0)}\n"
183         else:
184             reply = f"{int(response['clear_result'] == 'success')}\n"
185         proc.stdin.write(reply)
186         proc.stdin.flush()
187     finally:
188         if proc.poll() is None:
189             proc.terminate()
190             proc.wait(timeout=5)
191             reader.join(timeout=1)
192             err_reader.join(timeout=1)
193         for stream in (proc.stdin, proc.stdout, proc.stderr):
194             stream.close()
195
196
197 def main():
198     p = argparse.ArgumentParser()
199     p.add_argument('--problem', type=int, choices=[3,4], required=True)
200     p.add_argument('--team-id', required=True)
201     p.add_argument('--url', default='http://127.0.0.1:2026')
202     p.add_argument('--log', required=True)
203     p.add_argument('--early', type=float, default=35)
204     p.add_argument('--position-error', type=float, default=0.0)
205     p.add_argument('--strategy', choices=['geometric', 'joint', 'previous', 'stage1', 'stage2', 'baseline'],
206                     default='joint')
207     p.add_argument('--planner-depth', type=int, choices=[0,1,2,3], default=2,
208                     help='0: previous controller; 1/2/3: receding-horizon decision depth (default 2)')
209     a = p.parse_args()
210     # This program never launches or purchases a formal test session.
211     with open(a.log, 'x', encoding='utf-8') as f:
212         def log(row):
213             f.write(json.dumps(row, ensure_ascii=False) + '\n')
214             f.flush()
215         client = DeadlineClient(base_url=a.url, robot_id=a.team_id, log_sink=log)
216         result = drive(a.problem, client, a.early, position_error=a.position_error, planner_depth=a.
217                       planner_depth, strategy=a.strategy)
218         print(json.dumps(result, ensure_ascii=False, indent=2))
219
220 if __name__ == '__main__':
221     main()

```

src/q2_full_domain_certificate.py

```

1 """Exact-rational minimax audit on the FULL robust reception domain.
2
3 Trigonometric enclosures: Machin pi + Taylor alternating-series remainders.
4 Floating-point centers are proposals only; every leaf is accepted by exact
5 rational squared-distance checks. No library floating trig is trusted.
6 """
7 from fractions import Fraction as F
8 from functools import lru_cache
9 from math import hypot, isqrt
10 import json
11 import sys
12
13 S = 10**20
14
15 def rounded_interval(lo, hi):
16     return F((lo*S).__floor__(), S), F((hi*S).__ceil__(), S)
17
18 def atan_interval(inv, n):
19     x = F(1, inv)
20     v = sum((-1)**k*x**(2*k+1)/F(2*k+1) for k in range(n)), F(0)

```

```

21     tail = (-1)**n*x**(2*n+1)/F(2*n+1)
22     return min(v,v+tail),max(v,v+tail)
23
24 a,b=atan_interval(5,45)
25 c,d=atan_interval(239,14)
26 PI_LO,PI_HI=rounded_interval(16*a-4*d,16*b-4*c)
27
28 @lru_cache(None)
29 def trig(degrees, cosine=False):
30     degrees=F(degrees)
31     lo,hi=sorted((degrees*PI_LO/180,degrees*PI_HI/180))
32     x=F(((lo+hi)*S/2).__floor__(),S)
33     err=max(abs(x-lo),abs(x-hi))
34     term=F(1) if cosine else x
35     v=term
36     for k in range(1,32):
37         term *= -x*x/F((2*k-1)*(2*k) if cosine else (2*k)*(2*k+1))
38         v += term
39     k=32
40     tail=term*(-x*x)/F((2*k-1)*(2*k) if cosine else (2*k)*(2*k+1))
41     # |x|<4 here, so the omitted alternating tail decreases in magnitude.
42     assert abs(x)<4
43     return rounded_interval(min(v,v+tail)-err,max(v,v+tail)+err)
44
45 class I:
46     def __init__(self,a,b=None):self.a=F(a);self.b=F(a if b is None else b)
47     def __add__(x,y):
48         y=as_i(y);return I(x.a+y.a,x.b+y.b)
49     __radd__=__add__
50     def __neg__(x):return I(-x.b,-x.a)
51     def __sub__(x,y):return x+-as_i(y)
52     def __rsub__(x,y):return as_i(y)+-x
53     def __mul__(x,y):
54         y=as_i(y);v=[x.a*y.a,x.a*y.b,x.b*y.a,x.b*y.b];return I(min(v),max(v))
55     __rmul__=__mul__
56     def __truediv__(x,y):
57         y=as_i(y);assert y.a*y.b>0;return x*I(1/y.b,1/y.a)
58     def sq(x):
59         assert x.a>=0;return I(x.a*x.a,x.b*x.b)
60     def pair(x):return [float(x.a),float(x.b)]
61
62 def as_i(x):return x if isinstance(x,I) else I(x)
63 def sin(x):return I(*trig(x))
64 def cos(x):return I(*trig(x,True))
65 def sqrt_bounds(x):
66     lo=isqrt((x.a*S*S).__floor__())
67     hi=isqrt((x.b*S*S).__ceil__()+1)
68     return I(F(lo,S),F(hi,S))
69
70 def global_lower(e1,e2,t_upper,claimed_floor):
71     R=1500; eps=F('1.005'); T=F(t_upper)
72     k=sin(e1+2*e2);h=cos(e1+2*e2);s=sin(2*e2);l=sin(2*e1+2*e2)
73     # Every q in G has 0<=a<=1005, |b|<=1005, norm(q)<=1005.
74     # b<=30 is handled by the fixed normal-readout witness pair (1340,0),(1500,0).
75     # Their angle difference has positive dot product and tan(delta)<=4800/165825.
76     assert F(4800,165825)<(sin(2*e2)/cos(2*e2)).a
77     # b>=30: denominator positive, B on the forward rotated ray, r_B>5,
78     # and |B-q|>5 so neither witness triggers the near response.
79     denlo=(R*l-1005*k).a
80     denhi=(R*l+1005*h).b
81     tlo=(R*h*30-s*1005**2).a/denhi
82     lambda_lo=(30*cos(e1)-1005*sin(e1)).a/denhi
83     assert denlo>0 and tlo>5
84     assert (R*h-1005).a>0 # x-component of rotated A-q is positive
85     assert lambda_lo*(R-1005)>5

```

```

86     assert T<(R*cos(2*e1)).a
87     # N-T*D = -s*||q||^2 + w.q - T*R*1. G is contained in
88     # disk(center=(5*cos(eps),-5*sin(eps)), radius=1000).
89     cx,cy=5*cos(eps),-5*sin(eps)
90     wx,wy=k*(R+T),h*(R-T)
91     gx,gy=wx-2*s*cx,wy-2*s*cy
92     assert gx.a>0 and gy.a>0
93     gnorm=sqrt_bounds(gx.sq()+gy.sq())
94     assert gnorm.a>(2000*s).b
95     # Norm(center)^2=25 exactly. The quadratic maximum occurs at radius 1000.
96     quad_max=-s*(1000000+25)+wx*cx+wy*cy-T*R*1+1000*gnorm
97     assert quad_max.b<=0, ('increase t_upper',quad_max.pair())
98     rad=sqrt_bounds(I(R*R)+I(T*T)-2*R*T*cos(2*e1))/2
99     assert rad.a>=F(claimed_floor)
100    assert F(claimed_floor)<80
101    return {'certified_floor_m':claimed_floor,'global_lower_m':rad.pair(),
102            't_upper_m':t_upper,'quadratic_max_upper':float(quad_max.b),
103            'positive_denominator_lower':float(denlo),'witness_r_lower_m':float(tlo),
104            'witness_second_distance_lower_m':float(lambda_lo*(R-1005)),
105            'small_b_witness_radius_m':80}
106
107 def reception(q):
108     a,b=map(F,q); eps=F('1.005')
109     d=a*cos(eps)-abs(b)*sin(eps); normsq=a*a+b*b
110     c1=2000*d-normsq;c2=999975+10*d-normsq
111     assert c1.a>=0 and c2.a>=0
112     return {'q':[str(a),str(b)],'r1000_constraint_margin_m2':c1.pair(),
113            'r5_constraint_margin_m2':c2.pair()}
114
115 def clip(poly, normal, rhs):
116     out=[]
117     for i,x in enumerate(poly):
118         y=poly[(i+1)%len(poly)]
119         f=sum(normal[j]*x[j] for j in range(2))-rhs
120         g=sum(normal[j]*y[j] for j in range(2))-rhs
121         if f<=0:out.append(x)
122         if (f<=0)!=(g<=0):
123             t=f/(f-g);out.append(tuple(x[j]+t*(y[j]-x[j]) for j in range(2)))
124     return out
125
126 def center_proposal(poly):
127     p=[tuple(map(float,z)) for z in poly]
128     if not p:return (F(0),F(0))
129     # Any center is permitted. Select best among pair midpoints and circumcenters.
130     candidates=list(p)
131     for i,a in enumerate(p):
132         for j,b in enumerate(p[:i]):
133             candidates.append(((a[0]+b[0])/2,(a[1]+b[1])/2))
134             for c in p[:j]:
135                 ux,uy=b[0]-a[0],b[1]-a[1];vx,vy=c[0]-a[0],c[1]-a[1]
136                 d=2*(ux*vy-uy*vx)
137                 if abs(d)>1e-15:
138                     uu=ux*ux+uy*uy;vv=vx*vx+vy*vy
139                     candidates.append((a[0]+(uu*vy-vv*uy)/d,a[1]+(ux*vv-vx*uu)/d))
140     center=min(candidates,key=lambda c:max(hypot(c[0]-z[0],c[1]-z[1]) for z in p))
141     return tuple(F(f"{v:.12f}") for v in center)
142
143 def certify_fixed_q(bound=F('55.739')):
144     q=(F('843.1178'),F('545.4003'));eps=F('1.005')
145     # Outer triangle plus the exact disk tangents at +/-eps (with rational
146     # coefficient error allowances). This contains every point of the arc.
147     tangent=sin(eps).b/cos(eps).a
148     first=[(F(0),F(0)),(F(1500),-1500*tangent),(F(1500),1500*tangent)]
149     for angle in [-eps,eps]:
150         nx,ny=cos(angle),sin(angle)

```

```

151     normal=((nx.a+nx.b)/2,(ny.a+ny.b)/2)
152     pad=3000*((nx.b-nx.a)+(ny.b-ny.a))/2
153     first=clip(first,normal,F(1500)+pad)
154     stack=[(F(-182+23*k),F(-182+23*(k+1))) for k in range(8)]
155     leaves=[];nodes=0
156     while stack:
157         lo,hi=stack.pop();mid=(lo+hi)/2;half=(hi-lo)/2
158         poly=first
159         for angle,sign in [(mid+eps+half,1),(mid-eps-half,-1)]:
160             si,co=sin(angle),cos(angle)
161             nx,ny=-si*sign,co*sign
162             normal=((nx.a+nx.b)/2,(ny.a+ny.b)/2)
163             # Every source has |z_x-q_x|, |z_y-q_y|<3000.
164             pad=3000*((nx.b-nx.a)+(ny.b-ny.a))/2
165             rhs=sum(normal[j]*q[j] for j in range(2))+pad
166             poly=clip(poly,normal,rhs)
167         nodes+=1
168         center=center_proposal(poly)
169         sq=max((sum((z[j]-center[j])**2 for j in range(2)) for z in poly),default=F(0))
170         if sq<=bound*bound:
171             leaves.append({"lo":str(lo),"hi":str(hi),"center":[str(v) for v in center],"vertices":len(poly)
172 })
173         else:
174             assert hi-lo>F(1,10**12),"Requested radius is too small for this outer triangle"
175             stack.extend([(lo,mid),(mid,hi)])
176         leaves.sort(key=lambda a:F(a['lo']))
177         assert F(leaves[0]['lo'])==-182 and F(leaves[-1]['hi'])==2
178         assert all(x['hi']==y['lo'] for x,y in zip(leaves,leaves[1:]))
179     return {"q":[str(v) for v in q],"upper_m":str(bound),"verified_by":"exact Fraction squared-distance
180 checks", "nodes":nodes,"leaves":leaves}
181
182 if __name__=='__main__':
183     # Tiny inward margins avoid first-reading rounding ties and ensure the
184     # physical quantized witnesses are feasible under an unknown tie rule.
185     e1=F('1.004999999')
186     result={"reception":reception(['843.1178','545.4003']),
187           "continuous_model_lower":global_lower(e1,F('1.004999999'),'1400.796371','55.73823'),
188           "physical_quantized_lower":global_lower(e1,F('0.999999999'),'1400.99','55.6528'),
189           "continuous_and_quantized_upper":certify_fixed_q()}
190     with open(sys.argv[1] if len(sys.argv)>1 else 'q2_rational_certificate.json','w',encoding='utf-8') as f:
191         json.dump(result,f,ensure_ascii=False,indent=2)
192     print(json.dumps({k:({kk:vv for kk,vv in v.items() if kk!='leaves'}) for k,v in result.items()}),
193           ensure_ascii=False,indent=2))

```

src/q4_area_arc_certificate.py

```

1 """Exact-rational certificate: fixed universal Q4 designs need at least 15 sites.
2
3 Uses upper Riemann sums for two-disk overlap area and rational Taylor bounds
4 for the boundary arc. No floating point is used in any acceptance condition.
5 """
6 from fractions import Fraction as F
7 from math import isqrt
8 import json
9 import sys
10
11 R,L=1800,1000
12 K=F(138,100)
13 SCALE=10**12
14
15 def atan_bounds(inv,n):
16     x=F(1,inv)
17     v=sum((-1)**j*x**(2*j+1)/F(2*j+1) for j in range(n)),F(0))
18     tail=(-1)**n*x**(2*n+1)/F(2*n+1)

```

```

19     return min(v,v+tail),max(v,v+tail)
20
21 a,b=atan_bounds(5,25)
22 c,d=atan_bounds(239,8)
23 PI_LO=16*a-4*d
24 PI_HI=16*b-4*c
25 assert PI_LO>F('3.14159')
26
27 def cos_bounds(x):
28     x=F(x);assert 0<=x<=F('0.508')
29     term=F(1);total=term
30     for j in range(1,24):
31         term*=-x*x/F((2*j-1)*(2*j))
32         total+=term
33     tail=term*(-x*x)/F(47*48)
34     return min(total,total+tail),max(total,total+tail)
35
36 def sqrt_upper(n):
37     assert n>=0
38     v=n*SCALE*SCALE
39     root=isqrt(v)
40     if root*root<v:root+=1
41     return F(root,SCALE)
42
43 def overlap_upper(r):
44     # f(y)=[sqrt(R^2-y^2)+sqrt(L^2-y^2)-r]_+ is nonincreasing on [0,L].
45     # Thus the unit-spaced LEFT rectangle sum is an UPPER integral bound.
46     total=F(0)
47     for y in range(L):
48         total+=max(F(0),sqrt_upper(R*R-y*y)+sqrt_upper(L*L-y*y)-r)
49     return 2*total
50
51 def main():
52     # On  $r \leq \sqrt{R^2+L^2}$ ,  $\delta(r) = \arccos(R/r)$ . For all  $r$ ,  $\delta(r) \leq \arctan(L/R)$ .
53     edges=[1800,1900,1950,2000,2025,2050,2800]
54     angle_upper=[F('0.326'),F('0.395'),F('0.452'),F('0.476'),F('0.4991'),F('0.508')]
55     records=[]
56     for lo,hi,u in zip(edges,edges[1:],angle_upper):
57         clo,chi=cos_bounds(u)
58         assert clo>0 and u<PI_LO/2
59         if hi<=2050:
60             assert hi*hi<R*R+L*L
61             assert chi<=F(R,hi)
62             angle_method='cos(upper)<=R/right_endpoint'
63         else:
64             assert chi*chi<=F(R*R,R*R+L*L)
65             angle_method='global delta<=atan(L/R)<upper'
66         area=overlap_upper(lo)
67         deficit=1-area/(PI_LO*L*L)
68         slack=deficit-K*u
69         assert slack>0
70         records.append({'r_interval_m':[lo,hi],'overlap_area_upper_m2':float(area),
71                         'deficit_lower':float(deficit),'delta_upper_rad':str(u),
72                         'verified_slack_lower':float(slack),'angle_method':angle_method,
73                         'exact_area_upper':str(area)})
74     n_lower=F(3*R*R,L*L)+K*PI_LO
75     assert n_lower>14
76     result={'statement':'N >= 15 for every finite fixed universal directional-reception site set',
77            'source_disk_radius_m':R,'minimum_reception_radius_m':L,'k':str(K),
78            'pi_lower':str(PI_LO),'riemann_step_m':1,'intervals':records,
79            'N_real_lower':float(n_lower),'N_integer_lower':15,
80            'scope':'All source positions in the radius-1800 disk and all closed 180-degree emission
81            orientations; fixed finite sites; no optical fallback or adaptation counted as sites.'}
82     with open(sys.argv[1] if len(sys.argv)>1 else 'q4_area_arc_certificate.json','w',encoding='utf-8') as f
83         :

```

```

82         json.dump(result,f,ensure_ascii=False,indent=2)
83     print(json.dumps({k:v for k,v in result.items() if k!='pi_lower'},ensure_ascii=False,indent=2))
84
85 if __name__=='__main__':main()

```

src/run_experiments.py

```

1 """Frozen paired experiments; raw seeds are never selected from final results."""
2 import argparse,concurrent.futures,hashlib,json,platform,subprocess,time
3 from pathlib import Path
4 R=Path(__file__).resolve().parents[1]
5 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
6 def run(job):
7     name,start,n,stress,method=job;cmd=[str(R/'bin/benchmark_final'),str(R/'results'/f'{name}.csv'),str(start),
8         str(n),str(stress),str(method),str(method)]
9     begin=time.time();out=subprocess.run(cmd,capture_output=True,text=True);record={'command':cmd,'exit_code':
10         out.returncode,'started_unix':begin,'runtime_s':time.time()-begin,'stdout':out.stdout,'stderr':out.
11         stderr,'output':f'results/{name}.csv'}
12     if out.returncode==0:record['output_sha256']=sha(R/record['output'])
13     (R/'artifacts'/f'run-{name}.json').write_text(json.dumps(record,ensure_ascii=False,indent=2));print(name,
14         out.returncode,round(record['runtime_s'],2),flush=True)
15     if out.returncode:raise RuntimeError(record)
16     return record
17 if __name__=='__main__':
18     p=argparse.ArgumentParser();p.add_argument('--suite',choices=['iid','stress','dev'],required=True);p.
19         add_argument('--workers',type=int,default=3);a=p.parse_args()
20     if a.suite=='iid':jobs=[(f'iid{m}',2610001,200,0,m) for m in [56,78,84]]
21     elif a.suite=='stress':jobs=[(f'stress{s}_{m}',2620001+(s-1)*10000,40,s,m) for s in range(1,6) for m in
22         [56,84]]
23     else:jobs=[(f'dev{m}_replay',2110001,40,0,m) for m in [56,78,84]]
24     with concurrent.futures.ThreadPoolExecutor(max_workers=a.workers) as pool:records=list(pool.map(run,jobs))
25     (R/'artifacts'/f'execution-{a.suite}.json').write_text(json.dumps({'suite':a.suite,'environment':platform.
26         platform(),'binary_sha256':sha(R/'bin/benchmark_final'),'records':records},ensure_ascii=False,indent
27         =2))

```

src/analyze_revision.py

```

1 import csv,json,math,statistics as st
2 from pathlib import Path
3 R=Path(__file__).resolve().parents[1]
4 def rows(name):
5     p=R/'results'/name
6     return list(csv.DictReader(p.open()))
7 def paired(a,b,q):
8     a={int(x['seed']):x for x in a if int(x['problem'])==q};b={int(x['seed']):x for x in b if int(x['problem'])==q}
9     assert set(a)==set(b) and a
10     for k in a:
11         assert a[k]['n']==b[k]['n']==a[k]['cleared']==b[k]['cleared']
12     x=[float(a[k]['time_s']) for k in sorted(a)];y=[float(b[k]['time_s']) for k in sorted(a)];d=[i-j for i,j
13         in zip(x,y)];n=len(d);mean=st.mean(d);se=st.stdev(d)/math.sqrt(n) if n>1 else 0
14     from scipy.stats import t
15     critical=float(t.ppf(.975,n-1))
16     out={'n':n,'baseline_mean_s':st.mean(x),'candidate_mean_s':st.mean(y),'saving_s':mean,'saving_pct':100*
17         mean(st.mean(x),'ci95_s':[mean-critical*se,mean+critical*se]),'faster':sum(v>1e-4 for v in d),'slower':
18         sum(v<-1e-4 for v in d),'ties':sum(abs(v)<=1e-4 for v in d),'max_regression_s':-min(d),'max_saving_s':
19         max(d),'all_cleared':True,'mean_source_s':st.mean(float(i['avg_s']) for i in b.values()),'cpu_mean_s':
20         st.mean(float(i['runtime_s']) for i in b.values()),'cpu_max_s':max(float(i['runtime_s']) for i in b.
21         values()),'baseline_source_s':st.mean(float(i['avg_s']) for i in a.values())}
22     out['decomposition_s']={k:st.mean((float(a[s][field])-float(b[s][field]))*factor for s in a) for k,field,
23         factor in [(('move','move_m',.2),('measure','measures',5),('switch','switches',1),('failed_clear','miss',3))]}
24     out['candidate_counts']={field:st.mean(float(i.get(field,0)) for i in b.values()) for field in ['move_m','
25         measures','switches','miss','fallbacks','residual_replaced','residual_retired','rollout_calls','

```

```

        rollout_replaced'}}
18 assert abs(sum(out['decomposition_s'].values())-mean)<.001
19 out['pairs']=[{'seed':s,'baseline_s':float(a[s]['time_s']),'candidate_s':float(b[s]['time_s']),'saving_s':
        float(a[s]['time_s'])-float(b[s]['time_s'])} for s in sorted(a)]
20 return out
21 if __name__=='__main__':
22     development={};base=rows('dev56.csv')
23     for method,name in [(79,'dev79.csv'),(80,'dev80.csv'),(81,'dev81b.csv'),(82,'dev82.csv'),(83,'dev83.csv')
        ,(84,'dev84.csv'),(85,'dev85.csv'),(86,'dev86.csv')]:
24         development[str(method)]={'q':paired(base,rows(name),q) for q in [3,4]}
25     out={'development':development,'iid':{},'stress':{}}
26     for q in [3,4]:
27         out['iid'][str(q)]={'vs56':paired(rows('iid56.csv'),rows('iid84.csv'),q),'vs78':paired(rows('iid78.csv'),
            rows('iid84.csv'),q)}
28     for group in range(1,6):out['stress'][str(group)]={'q':paired(rows(f'stress{group}_56.csv'),rows(f'
        stress{group}_84.csv'),q) for q in [3,4]}
29     for q in ['3','4']:
30         assert out['iid'][q]['vs56']['n']==200
31         for group in out['stress']:assert out['stress'][group][q]['n']==40
32         (R/'artifacts/summary.json').write_text(json.dumps(out,ensure_ascii=False,indent=2))
33     print(json.dumps({k:{f:v for f,v in d.items() if f not in ['pairs']} for k,d in m.items()} for q,m in
        out['iid'].items()),ensure_ascii=False,indent=2))

```

tests/replay_residual_fraction.py

```

1 """Independent exact-rational replay of accepted spatial witnesses.
2
3 No production interval predicate imported. JSON numbers are converted through
4 binary64, matching the robot's actual coordinates exactly, then to Fraction.
5 """
6 import json,sys,time,hashlib
7 from fractions import Fraction as F
8 from pathlib import Path
9 start=time.monotonic();root=Path(__file__).resolve().parents[1]
10 witness=Path(sys.argv[1]) if len(sys.argv)>1 else root/'artifacts/residual-witness.json'
11 a=json.loads(witness.read_text())
12 P=[tuple(F(float(v)) for v in z) for z in a['sites']]
13 def cross(a,b,c):return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
14 def hull(p):
15     p=sorted(set(p));lo=[];hi=[]
16     for chain,points in [(lo,p),(hi,p[::-1])]:
17         for q in points:
18             while len(chain)>1 and cross(chain[-2],chain[-1],q)<=0:chain.pop()
19             chain.append(q)
20     return lo[::-1]+hi[::-1]
21 for raw in a['boxes']:
22     x0,y0,x1,y1=map(lambda x:F(float(x)),raw);v=[(x0,y0),(x1,y0),(x1,y1),(x0,y1)]
23     safe=[p for p in P if all(sum((p[i]-z[i])**2 for i in [0,1])<=F('999.999'))**2 for z in v)]
24     h=hull(safe);assert len(h)>=3
25     assert all(cross(h[i],h[(i+1)%len(h)],z)>=0 for i in range(len(h)) for z in v)
26 # Rebuild the dyadic partition to prove every disk-intersecting box is present,
27 # instead of trusting a list of successful leaves that might have gaps.
28 leaves={tuple(F(float(x)) for x in b) for b in a['boxes']};seen=0
29 stack=[(F(-1800),F(-1800),F(1800),F(1800))]
30 while stack:
31     b=stack.pop();x0,y0,x1,y1=b
32     if b in leaves:seen+=1;continue
33     x=x0 if x0>0 else x1 if x1<0 else 0;y=y0 if y0>0 else y1 if y1<0 else 0
34     if x*x+y*y>1800**2:continue
35     assert max(x1-x0,y1-y0)>F(1,10000),'missing leaf'
36     if x1-x0>=y1-y0:
37         m=(x0+x1)/2;stack.extend([(x0,y0,m,y1),(m,y0,x1,y1)])
38     else:
39         m=(y0+y1)/2;stack.extend([(x0,y0,x1,m),(x0,m,x1,y1)])
40 assert seen==len(leaves)

```

```

41 out={'status':'PASS','arithmetic':'Python Fraction, independent of C++ interval code','sites':len(P),'
    verified_leaf_boxes':seen,'full_disk_partition':True,'witness_sha256':hashlib.sha256(witness.
    read_bytes()).hexdigest(),'runtime_s':time.monotonic()-start}
42 (root/'artifacts/residual-fraction-replay.json').write_text(json.dumps(out,indent=2));print(json.dumps(out)
    )

```

tests/q3_coverage_bound.py

```

1 from fractions import Fraction as F
2 import json
3 from pathlib import Path
4 root=Path(__file__).resolve().parents[1]
5 sqrt3lo=F('1.7320508075688772935');sqrt3hi=F('1.7320508075688772936')
6 assert sqrt3lo**2<3<sqrt3hi**2
7 R=F(1800);reception=F('999.6');a=F(1124)
8 # Radius r <= reception is covered by the origin. In every angular sector
9 # |delta| <=30 degrees, squared distance to its ring point is convex in r.
10 # Its maximum on [reception,1800] is at an endpoint.
11 boundary=R*R+a*a-R*a*sqrt3lo
12 inner=reception*reception+a*a-reception*a*sqrt3lo
13 assert max(boundary,inner)<reception**2
14 points=json.loads((root/'results/q3_production_points.json').read_text())
15 assert len(points)==7
16 ideal=[(F(0),F(0)),(a,F(0)),(a/2,a*sqrt3lo/2),(-a/2,a*sqrt3lo/2),(-a,F(0)),(-a/2,-a*sqrt3lo/2),(a/2,-a*
    sqrt3lo/2)]
17 for actual,reference in zip(points,ideal):
18     error=sum((F.from_float(float(x))-y)**2 for x,y in zip(actual,reference))
19     assert error<F('0.0000000001')**2
20 assert F('0.0000000001')+a*(sqrt3hi-sqrt3lo)/2<F('0.0000000001')
21 result={'ring_radius':1124,'certified_ideal_cover_radius':'999.6','boundary_squared_upper':str(boundary),'
    inner_squared_upper':str(inner),'sqrt3_interval':[str(sqrt3lo),str(sqrt3hi)],'
    coordinate_perturbation_allowance_m':'0.4','scope':'ideal sixfold ring + origin; implementation
    floating coordinates must be within 0.4m'}
22 result['production_binary64_error_bound_m']='0.0000000001'
23 (root/'results/q3_coverage_bound.json').write_text(json.dumps(result,indent=2))
24 print('PASS rational radial/angular proof: 1124m seven-point ring covers the 1800m disk within 999.6m')

```

tests/test_geometry_revision.cpp

```

1 #include "../src/policy.hpp"
2 #include <cassert>
3 #include <fstream>
4 #include <iomanip>
5 #include <iostream>
6 int main(int argc,char**argv){
7     if(argc!=2){std::cerr<<"usage: test_geometry_revision OUTPUT_WITNESS.json\n";return 2;}
8     auto p=geo::observe(geo::outer_disk({0,0},1800),{0,0},0);p=geo::disk_clip(p,{0,0},1500);p=geo::clip(p
    ,{1,0},1500);
9     int checks=0;double gap=0;
10    for(auto q:geo::Poly{{750,300},{810,-480},{300,0},{0,600},{1400,10}}){auto b=robust_geometry::worst_radius
    (p,q,512,.01);auto range=robust_geometry::angles(p,q);double sampled=0;
11    for(int i=0;i<=5000;++i){double angle=range.first+(range.second-range.first)*i/5000;auto region=geo::
    observe(p,q,angle);if(!region.empty())sampled=std::max(sampled,geo::mec(region).r);++checks;}
12    assert(sampled<=b.upper+1e-6);assert(b.lower<=b.upper);gap=std::max(gap,b.upper-sampled);
13 }
14 std::vector<dynamic_selection::Event>history{{{0,0},2,0}};
15 assert(robust_geometry::reception(p,{750,300},history,3));
16 assert(!robust_geometry::reception(p,{750,300},history,4));
17 assert(!robust_geometry::candidate(p,{0,0},{0,0},0,history,3,2).valid);
18 std::vector<geo::Poly>cells;auto centers=adaptive_optical::cover(p,{0,0},0,&cells);assert(cells.size()==
    centers.size());
19 auto moved=neighborhood_tour::optimize(centers,cells,{0,200});assert(neighborhood_tour::length(moved
    ,{0,200})<=neighborhood_tour::length(centers,{0,200})+1e-7);

```

```

20 for(size_t i=0;i<cells.size();++i){assert(neighborhood_tour::valid(moved[i],cells[i]));for(auto z:cells[i]
    ])assert(gео::dist(moved[i],z)<20);}
21 geo::Poly sites;for(auto q:certified_layout21::points)sites.push_back({q.x,q.y});auto certificate=
    residual_cover::certify(sites);assert(certificate.pass);
22 auto missing=sites;missing.erase(missing.begin()+9);assert(!residual_cover::certify(missing,certificate.
    leaves).pass);
23 residual_cover::State state;std::vector<bool>visited(sites.size());state.leaves=certificate.leaves;
24 geo::Pt shifted=state.replace(sites,visited,1,{850,50});assert(gео::dist(shifted,sites[1])>1);sites[1]=
    shifted;auto recert=residual_cover::certify(sites);assert(recert.pass);
25 // Export actual accepted coordinates and interval-certified leaf boxes for
26 // independent Fraction replay; doubles are round-trip decimal encoded.
27 std::ofstream f(argv[1]);if(!f){std::cerr<<"cannot open witness output\n";return 3;}
28 f<<std::setprecision(17)<<{"\nsites\":";for(size_t i=0;i<sites.size();++i){if(i)f<<",";f<<["<<sites[i].x
    <<","<<sites[i].y<<"]};f<<"},"boxes\":";for(size_t i=0;i<recert.leaves.size();++i){auto b=recert.
    leaves[i];if(i)f<<",";f<<["<<b.x0<<","<<b.y0<<","<<b.x1<<","<<b.y1<<"]};f<<"]}\n";
29 f.close();if(!f){std::cerr<<"cannot finish witness output\n";return 4;}
30 std::cout<<{"\nstatus\":"<<"PASS\","<<"angle_checks\":"<<checks<<","<<"max_upper_gap_m\":"<<gap<<","<<
    optical_cells\":"<<cells.size()<<","<<"static_leaves\":"<<certificate.leaves.size()<<","<<
    replacement_leaves\":"<<recert.leaves.size()<<","<<"replacement_shift_m\":"<<gео::dist(shifted,geo::Pt
    {995,0})<<"}<<"}\n";
31 }

```

tests/run_checks.py

```

1 """Compile and run checks; the rational replay must consume this run's witness."""
2 import hashlib,json,os,shutil,subprocess,sys,tempfile,time
3 from pathlib import Path
4 R=Path(__file__).resolve().parents[1];records=[]
5 cc=os.environ.get('CXX') or shutil.which('clang++') or shutil.which('g++')
6 assert cc,'C++17 compiler required'
7 (R/'bin').mkdir(exist_ok=True);(R/'artifacts').mkdir(exist_ok=True)
8 digest=lambda p:hashlib.sha256(Path(p).read_bytes()).hexdigest()
9 witness=R/'artifacts/residual-witness.json'
10 for name in ['test_geometry_revision','test_joint','test_controller','test_adaptive_optical',
    'test_shortcut_controller','test_action_budget','test_scan_fee','test_discovery','test_variable_cover
    ']:
11 source=R/'tests'/f'{name}.cpp';binary=R/'bin'/name
12 cmd=[cc,'-O2','-std=c++17',str(source),'-o',str(binary)];begin=time.time();out=subprocess.run(cmd,
    capture_output=True,text=True)
13 if out.returncode:raise RuntimeError(out.stderr)
14 run_cmd=[str(binary)];extra={}
15 if name=='test_geometry_revision':
16 witness.unlink(missing_ok=True)
17 with tempfile.TemporaryDirectory(prefix='geometry-export-') as tmp:
18 bad=subprocess.run([str(binary),str(Path(tmp)/'absent'/'witness.json')],cwd=tmp,capture_output=True,text
    =True)
19 assert bad.returncode!=0 and 'PASS' not in bad.stdout
20 run_cmd.append(str(witness));out=subprocess.run(run_cmd,cwd=tmp,capture_output=True,text=True)
21 assert out.returncode==0 and witness.is_file() and witness.stat().st_mtime>=begin
22 extra={'source_sha256':digest(source),'binary_sha256':digest(binary),'fresh_witness_sha256':digest(
    witness),'invalid_output_rejected':True,'execution_cwd':'fresh temporary directory'}
23 else:out=subprocess.run(run_cmd,cwd=R,capture_output=True,text=True)
24 records.append({'name':name,'compile_command':cmd,'command':run_cmd,'exit_code':out.returncode,'runtime_s
    ':time.time()-begin,'stdout':out.stdout,'stderr':out.stderr,**extra})
25 if out.returncode:raise RuntimeError(records[-1])
26 print(name,'PASS',flush=True)
27 for name in ['replay_residual_fraction.py','test_bridge_deadline.py']:
28 cmd=[sys.executable,str(R/'tests'/name)]
29 if name.startswith('replay'):cmd.append(str(witness))
30 begin=time.time();out=subprocess.run(cmd,cwd=R,capture_output=True,text=True)
31 records.append({'name':name,'command':cmd,'exit_code':out.returncode,'runtime_s':time.time()-begin,'stdout
    ':out.stdout,'stderr':out.stderr})
32 if out.returncode:raise RuntimeError(records[-1])
33 print(name,'PASS',flush=True)

```

```

34 assert json.loads((R/'artifacts/residual-fraction-replay.json').read_text())['witness_sha256']==records
    [0]['fresh_witness_sha256']
35 (R/'artifacts/regression-tests.json').write_text(json.dumps({'status': 'PASS', 'records': records},
    ensure_ascii=False, indent=2))

```

src/validate_mock.py

```

1 """Independent nonofficial public-protocol Python simulator kernel + real HTTP tests of C++ solver."""
2 import argparse
3 import json
4 import sys
5 import threading
6 import time
7 from pathlib import Path
8 from bridge import ROOT, drive, DeadlineClient
9 from mocksim.arena import Session, make_case
10 from mocksim.client import RobotClient
11 from mocksim.server import MockSimulator, serve
12
13 class KernelClient:
14     def __init__(self, problem, seed):
15         self.session = Session(make_case(problem=problem, seed=seed))
16     def enter(self):
17         self.session.enter()
18         return dict(accepted=True, remaining_real_duration_s=1200)
19     def measure(self, x, y, k):
20         o=self.session.measure(x, y, k)
21         return dict(accepted=True, virtual_time_s=o.virtual_time_s, **o.payload)
22     def clear(self, x, y, k):
23         o=self.session.clear(x, y, k)
24         return dict(accepted=True, virtual_time_s=o.virtual_time_s, **o.payload)
25     def exit(self):
26         o=self.session.exit()
27         return dict(accepted=True, virtual_time_s=o.virtual_time_s)
28
29 def main():
30     p=argparse.ArgumentParser();p.add_argument('--count', type=int, default=30);p.add_argument('--early', type
    =float, default=35);p.add_argument('--planner-depth', type=int, choices=[0,1,2,3], default=2);p.
    add_argument('--strategy', choices=['geometric', 'joint', 'previous', 'stage1', 'stage2', 'baseline'],
    default='joint')
31     a=p.parse_args();rows=[]
32     for problem in [3,4]:
33         for seed in range(1610001,1610001+a.count):
34             client=KernelClient(problem, seed)
35             out=drive(problem, client, a.early, planner_depth=a.planner_depth, strategy=a.strategy)
36             assert out['cleared']==client.session.case.total
37             rows.append(dict(layer='original_kernel', problem=problem, seed=seed, strategy=out['strategy'],
    planner_depth=out['planner_depth'], **client.session.statistics(out['runtime_s']), certificate=out['
    certificate']))
38         for seed in [1620001,1620002,1620003]:
39             sim=MockSimulator(problem=problem, mode='practice', seed=seed, team_id='LOCAL_RESEARCH', port=0)
40             server=serve(sim);threading.Thread(target=server.serve_forever, daemon=True).start();sim.start()
41             file=ROOT/'results'/f'LOCAL_HTTP_{a.strategy}_D{a.planner_depth}_P{problem}_{seed}.jsonl'
42             try:
43                 with file.open('w', encoding='utf-8') as f:
44                     client=DeadlineClient(base_url=f'http://127.0.0.1:{server.server_address[1]}', robot_id
    ='LOCAL_RESEARCH', log_sink=lambda r:f.write(json.dumps(r, ensure_ascii=False)+'\n'))
45                     out=drive(problem, client, a.early, planner_depth=a.planner_depth, strategy=a.strategy)
46                     assert out['cleared']==sim.case.total
47                     rows.append(dict(layer='local_HTTP', problem=problem, seed=seed, total=sim.case.total, **out))
48             finally:
49                 server.shutdown();server.server_close()
50             (ROOT/'results'/f'original_mock_{a.strategy}_depth{a.planner_depth}.json').write_text(json.dumps(rows,
    ensure_ascii=False, indent=2), encoding='utf-8')
51     print(f'validated {len(rows)} nonofficial simulator cases (including 6 HTTP runs)')

```

```

52
53 if __name__=='__main__':main()

```

tests/test_bridge_deadline.py

```

1 """Transport failure tests: local HTTP only, no formal simulator session."""
2 import json
3 import math
4 import sys
5 import threading
6 import time
7 import unittest
8 import subprocess
9 from pathlib import Path
10 from unittest.mock import patch
11 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
12
13 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / 'src'))
14 import bridge
15 from mocksim.client import ProtocolError
16
17 class FakeClient:
18     def __init__(self, remaining=1200, rejected=False):
19         self.remaining=remaining; self.rejected=rejected; self.actions=[]
20     def enter(self): return {'accepted':True,'remaining_real_duration_s':self.remaining}
21     def measure(self,*args):
22         self.actions.append('measure')
23         return {'accepted':not self.rejected,'virtual_time_s':0,'measure_result':'no_signal'}
24     def exit(self):
25         self.actions.append('exit')
26         return {'accepted':True,'virtual_time_s':0}
27
28 class DeadlineTests(unittest.TestCase):
29     def test_non_200_success_status_is_rejected(self):
30         class Handler(BaseHTTPRequestHandler):
31             def do_POST(self):
32                 self.rfile.read(int(self.headers['Content-Length']))
33                 data=json.dumps({'accepted':True,'virtual_time_s':0}).encode()
34                 self.send_response(201);self.send_header('Content-Length',str(len(data)));self.end_headers
35                 ();self.wfile.write(data)
36             def log_message(self,*args):pass
37         server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
38         threading.Thread(target=server.serve_forever,daemon=True).start()
39         try:
40             c=bridge.DeadlineClient(base_url=f'http://127.0.0.1:{server.server_port}',robot_id='REVIEW')
41             with self.assertRaises(ProtocolError):c.enter()
42             finally:server.shutdown();server.server_close()
43         def test_remaining_zero_starts_no_action(self):
44             c=FakeClient(remaining=0)
45             with self.assertRaises(TimeoutError): bridge.drive(3,c)
46             self.assertEqual(c.actions,[])
47
48         def test_rejected_action_never_counts_as_observation(self):
49             c=FakeClient(rejected=True)
50             with self.assertRaises(bridge.Rejected): bridge.drive(3,c)
51             self.assertEqual(c.actions,['measure'])
52
53         def test_solver_without_stdout_honors_short_deadline(self):
54             c=FakeClient(remaining=.25)
55             original=subprocess.Popen
56             def silent(*args,**kwargs):
57                 return original([sys.executable,'-c','import time; time.sleep(2)'],**kwargs)
58             start=time.perf_counter()
59             with patch.object(bridge.subprocess,'Popen',side_effect=silent):
60                 with self.assertRaises(TimeoutError): bridge.drive(3,c)

```

```

60     self.assertLess(time.perf_counter()-start,1.0)
61     self.assertNotIn('measure',c.actions)
62
63 def test_retry_reuses_identical_id_and_body(self):
64     records=[]
65     class Handler(BaseHTTPRequestHandler):
66         def do_POST(self):
67             raw=self.rfile.read(int(self.headers['Content-Length']));records.append(raw)
68             if len(records)==1:
69                 self.close_connection=True; self.connection.close();return
70             data=json.dumps({'accepted':True,'virtual_time_s':6,'measure_result':'no_signal'}).encode()
71             self.send_response(200);self.send_header('Content-Length',str(len(data)));self.end_headers
72             ();self.wfile.write(data)
73         def log_message(self,*args): pass
74     server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
75     threading.Thread(target=server.serve_forever,daemon=True).start()
76     try:
77         cls=getattr(bridge,'DeadlineClient',bridge.RobotClient)
78         c=cls(base_url=f'http://127.0.0.1:{server.server_port}',robot_id='REVIEW',retry_backoff_s=.001)
79         c.deadline_at=time.perf_counter()+1
80         self.assertTrue(c.measure(0,0,1)['accepted'])
81         self.assertEqual(len(records),2)
82         self.assertEqual(records[0],records[1])
83         self.assertEqual(c.counter,1)
84     finally: server.shutdown();server.server_close()
85
86 def test_expired_transport_deadline_sends_nothing(self):
87     cls=getattr(bridge,'DeadlineClient',bridge.RobotClient)
88     c=cls(base_url='http://127.0.0.1:1',robot_id='REVIEW',timeout_s=.01,retries=0)
89     c.deadline_at=time.perf_counter()-1
90     with self.assertRaises(TimeoutError):c.measure(0,0,1)
91     self.assertEqual(c.stats.requests,0)
92
93 def test_position_is_not_rounded_before_geometry_observation(self):
94     c=bridge.DeadlineClient(robot_id='REVIEW')
95     captures=[]
96     c.post=lambda path,payload:captures.append((path,payload)) or {'accepted':True}
97     x,y=0.,-.00000049
98     c.measure(x,y,1);c.clear(x,y,1)
99     for _,payload in captures:
100         self.assertEqual(payload['position'],{'x':x,'y':y})
101     eps=math.radians(1.005)
102     z=(1000*math.cos(eps-1e-12),1000*math.sin(eps-1e-12))
103     # If the old serializer moved the sensor to (0,0), a legal reading of
104     # 0.00 degrees excludes this true source from the solver's retained cone.
105     excess=-math.tan(eps)*z[0]+z[1]-y
106     self.assertGreater(excess,1e-9)
107     # Full float transport keeps physical and solver coordinates identical.
108     self.assertEqual(captures[0][1]['position']['y'],y)
109
110 def test_slow_http_body_cannot_extend_total_deadline(self):
111     records=[]
112     class Handler(BaseHTTPRequestHandler):
113         def do_POST(self):
114             records.append(self.rfile.read(int(self.headers['Content-Length'])))
115             data=json.dumps({'accepted':True,'virtual_time_s':6,'measure_result':'no_signal'}).encode()
116             self.send_response(200);self.send_header('Content-Length',str(len(data)));self.end_headers
117             ()
118         try:
119             for value in data:
120                 self.wfile.write(bytes([value]));self.wfile.flush();time.sleep(.015)
121             except OSError:pass
122         def log_message(self,*args):pass
123     server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
124     threading.Thread(target=server.serve_forever,daemon=True).start()

```

```

123         try:
124             c=bridge.DeadlineClient(base_url=f'http://127.0.0.1:{server.server_port}',robot_id='REVIEW',
retries=5)
125             start=time.perf_counter();c.deadline_at=start+.15
126             with self.assertRaises(TimeoutError):c.measure(0,0,1)
127             self.assertLess(time.perf_counter()-start,.5)
128             with self.assertRaises(TimeoutError):c.exit()
129             self.assertEqual(len(records),1)
130             finally:server.shutdown();server.server_close()
131
132     def test_retry_backoff_is_limited_by_remaining_duration(self):
133         records=[]
134         class Handler(BaseHTTPRequestHandler):
135             def do_POST(self):
136                 records.append(self.rfile.read(int(self.headers['Content-Length'])))
137                 self.close_connection=True;self.connection.close()
138             def log_message(self,*args):pass
139         server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
140         threading.Thread(target=server.serve_forever,daemon=True).start()
141         try:
142             c=bridge.DeadlineClient(base_url=f'http://127.0.0.1:{server.server_port}',robot_id='REVIEW',
retry_backoff_s=1)
143             start=time.perf_counter();c.deadline_at=start+.1
144             with self.assertRaises(TimeoutError):c.measure(0,0,1)
145             self.assertLess(time.perf_counter()-start,.5)
146             self.assertEqual(len(records),1)
147             finally:server.shutdown();server.server_close()
148
149 if __name__=='__main__':unittest.main(verbosity=2)

```

run_all.py

```
1 """Portable frozen reproduction entry point. Never starts official tests."""
2 from pathlib import Path
3 import subprocess,sys,argparse,shutil,json,time,os
4 R=Path(__file__).resolve().parent
5 p=argparse.ArgumentParser();p.add_argument('--full',action='store_true');p.add_argument('--paper',action='
    store_true');a=p.parse_args()
6 cc=os.environ.get('CXX') or shutil.which('clang++') or shutil.which('g++');assert cc,'C++17 compiler
    required'
7 (R/'bin').mkdir(exist_ok=True);records=[]
8 def run(cmd,cwd=R):
9     begin=time.time();out=subprocess.run(cmd,cwd=cwd,capture_output=True,text=True);rec={'command':cmd,'cwd':
        str(cwd.relative_to(R)) if cwd.is_relative_to(R) else str(cwd),'exit_code':out.returncode,'runtime_s':
        time.time()-begin,'stdout':out.stdout,'stderr':out.stderr};records.append(rec)
10    (R/'artifacts/reproduction-log.json').write_text(json.dumps(records,ensure_ascii=False,indent=2))
11    if out.returncode:raise RuntimeError(rec)
12    print('PASS',cmd[0],cmd[-1],flush=True)
13    run([cc,'-O2','-std=c++17','src/robot.cpp','-o','robot.exe'])
14    run([cc,'-O2','-std=c++17','src/benchmark.cpp','-o','bin/benchmark_final'])
15    run([sys.executable,'tests/run_checks.py'])
16    run([cc,'-O2','-std=c++17','tests/test_q3_radius.cpp','-o','bin/test_q3_radius']);run([str(R/'bin/
        test_q3_radius')]);run([sys.executable,'tests/q3_coverage_bound.py'])
17    run([sys.executable,'src/q2_full_domain_certificate.py','artifacts/q2-certificate.json'])
18    run([sys.executable,'src/q4_area_arc_certificate.py','artifacts/q4-lower-bound.json'])
19    if a.full:
20        run([sys.executable,'src/run_experiments.py','--suite','iid'])
21        run([sys.executable,'src/run_experiments.py','--suite','stress'])
22    run([sys.executable,'src/analyze_revision.py'])
23    if a.paper:
24        run([sys.executable,'src/build_paper_assets.py']);run([sys.executable,'src/write_solution.py'])
25        for _ in range(2):run(['xelatex','-interaction=nonstopmode','-halt-on-error','main.tex'],R/'paper')
26        run(['pandoc','B题几何策略优化_题解.md','--pdf-engine=xelatex','-V','documentclass=ctexart','-V','mainfont=
            Songti SC','-V','CJKmainfont=Songti SC','-V','geometry:margin=2.5cm','-V','papersize:a4','-V','
            fontsize=11pt','-o','B题几何策略优化_题解.pdf'])
27    run([os.environ.get('PDF_PYTHON',sys.executable),'src/finalize_pdfs.py'])
28    print('Frozen results reproduced; geometric=84 experimental, joint=78 recommended.')
```